THE ULTIMATE

# DOCKER COMMAND REFERENCE

602+ distinct practical Docker CLI command forms

Flags, options, arguments, examples, expected behavior, troubleshooting, and safety notes

From first container to image publishing, Compose, networking, storage, BuildKit, security, and Swarm operations.

**How each entry is structured**
Purpose • syntax • options used • positional arguments • example • expected result • troubleshooting • related commands • safety note.

Compatibility note: Docker evolves continuously. This reference includes 600+ practical CLI forms and defines the flags used in each example, but it is not a guarantee that every flag across every Engine release, plugin, operating system, and platform is covered. Use `docker COMMAND --help` and the official documentation to verify your installed version before production use.

# How to use this book

Entries are grouped by workflow, not alphabetically. Docker CLI commands are distinguished from Dockerfile instructions and shell utilities. The options section defines flags used in the example; the syntax section gives the general form. Version- and platform-dependent features are explicitly called out where relevant.

## Safety conventions

**DESTRUCTIVE entries can remove resources or data. HIGH RISK entries affect credentials, cluster membership, security boundaries, or service availability. Port publishing can expose services beyond the host depending on bind address and firewall policy.**

## Example conventions

Replace sample names, registry hosts, IPs, paths, tags, and tokens with values from your environment. Addresses from documentation ranges are placeholders. Shell pipelines, variables, and redirection are host-shell syntax, not Docker flags. Linux shell examples are used unless stated otherwise.

Coverage: 602 executable Docker CLI command forms, plus 20 Dockerfile reference entries and 8 combined workflow recipes. Options are version-sensitive; consult local help for exhaustive installed-command details.

# Contents


01. 01 · Fundamentals, installation & CLI — 29 entries · p. 4
02. 02 · Containers: run, create & lifecycle — 177 entries · p. 11
03. 03 · Images, registries & distribution — 57 entries · p. 56
04. 04 · Build, Dockerfile & BuildKit — 49 entries · p. 71
05. 06 · Networking — 43 entries · p. 84
06. 08 · Docker Compose — 70 entries · p. 95
07. 09 · Contexts & remote Engines — 12 entries · p. 113
08. 10 · Swarm, services, nodes & stacks — 87 entries · p. 116
09. 11 · Maintenance, security, plugins & advanced tooling — 78 entries · p. 140
10. 12 · Dockerfile syntax (not CLI commands) — 20 entries · p. 160
11. 13 · Practical combined workflows (CLI recipes) — 8 entries · p. 165


Quick path: verify installation → run a container → inspect logs/state → build/tag/push → networks/volumes → Compose → Buildx/Swarm → cleanup and hardening.

# 01 · Fundamentals, installation & CLI

29 reference entries. Follow the examples in order and substitute environment-specific values.

## 001. docker version — Compare client and server versions

**Purpose.** Confirm the CLI can reach the Engine and compare versions.

**Syntax.** `docker version [OPTIONS]`

**Options used.** No flag required; `--format TEMPLATE` selects fields.

**Arguments.** No positional arguments.

**Example.**

```
docker version
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 002. docker version — Extract client version as JSON

**Purpose.** Use structured output in support scripts.

**Syntax.** `docker version [OPTIONS]`

**Options used.** `--format` accepts a Go template; `json` serializes the client object.

**Arguments.** No positional arguments.

**Example.**

```
docker version --format '{{json .Client}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 003. docker version — Extract server API version

**Purpose.** Check API compatibility when options appear unsupported.

**Syntax.** `docker version [OPTIONS]`

**Options used.** `--format` selects the server API field.

**Arguments.** No positional arguments.

**Example.**

```
docker version --format '{{.Server.APIVersion}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 004. docker version — Compare client and server API values

**Purpose.** Print both API versions on one line for incident notes.

**Syntax.** `docker version [OPTIONS]`

**Options used.** `--format` interpolates client/server API fields.

**Arguments.** No positional arguments.

**Example.**

```
docker version --format 'client={{.Client.APIVersion}} server={{.Server.APIVersion}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 005. docker version — Check server reachability

**Purpose.** Use the Server section to determine whether the selected Engine is reachable.

**Syntax.** `docker version [OPTIONS]`

**Options used.** No option; inspect whether the Server section appears.

**Arguments.** No positional arguments.

**Example.**

```
docker version
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 006. docker info — Display Engine configuration

**Purpose.** Review storage driver, runtimes, plugins, security options, and daemon summary.

**Syntax.** `docker info [OPTIONS]`

**Options used.** `--format TEMPLATE` can select fields.

**Arguments.** No positional arguments.

**Example.**

```
docker info
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 007. docker info — Extract storage driver

**Purpose.** Get the active storage driver for a report.

**Syntax.** `docker info [OPTIONS]`

**Options used.** `--format` selects `.Driver`.

**Arguments.** No positional arguments.

**Example.**

```
docker info --format '{{.Driver}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 008. docker info — List configured runtimes

**Purpose.** Inspect configured OCI runtime names.

**Syntax.** `docker info [OPTIONS]`

**Options used.** `--format` plus `json` emits structured runtime metadata.

**Arguments.** No positional arguments.

**Example.**

```
docker info --format '{{json .Runtimes}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 009. docker info — Inspect security options

**Purpose.** Review the security options reported by the daemon.

**Syntax.** `docker info [OPTIONS]`

**Options used.** `--format` selects `.SecurityOptions`.

**Arguments.** No positional arguments.

**Example.**

```
docker info --format '{{json .SecurityOptions}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 010. docker info — Check daemon root directory

**Purpose.** Find the Engine data root as reported by the daemon.

**Syntax.** `docker info [OPTIONS]`

**Options used.** `--format` selects `.DockerRootDir`.

**Arguments.** No positional arguments.

**Example.**

```
docker info --format '{{.DockerRootDir}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 011. docker --help — List top-level commands

**Purpose.** Discover command groups supported by this installation.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `--help` prints usage and exits.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 012. docker --help — Inspect run options

**Purpose.** Ask the installed CLI for the options supported by `run`.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `--help` belongs to the subcommand and reflects the local CLI version.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker run --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 013. docker --help — Inspect Compose options

**Purpose.** Check the installed Compose V2 plugin command surface.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `compose --help` prints Compose subcommands and options.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker compose --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 014. docker --help — Use a specific client config directory

**Purpose.** Run the client with a selected config directory.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `--config DIR` is a global option selecting the client config directory.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker --config "$HOME/.docker" info
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 015. docker --help — Target a named context for one command

**Purpose.** Run a diagnostic without changing the saved default context.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `--context NAME` selects an endpoint for this invocation.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker --context staging info
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 016. docker --help — Target a Docker host endpoint

**Purpose.** Select an explicit daemon endpoint for one command.

**Syntax.** `docker [OPTIONS] COMMAND`

**Options used.** `-H, --host ENDPOINT` selects a daemon endpoint.

**Arguments.** COMMAND is a top-level command such as `run`, `image`, or `compose`.

**Example.**

```
docker -H unix:///var/run/docker.sock info
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 017. docker system df — Summarize Docker disk usage

**Purpose.** Measure image, container, volume, and build-cache usage.

**Syntax.** `docker system df [OPTIONS]`

**Options used.** No option required.

**Arguments.** No positional arguments.

**Example.**

```
docker system df
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 018. docker system df — Show verbose resource usage

**Purpose.** Identify individual resources and reclaimable space.

**Syntax.** `docker system df [OPTIONS]`

**Options used.** `-v, --verbose` expands per-resource details.

**Arguments.** No positional arguments.

**Example.**

```
docker system df --verbose
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 019. docker events — Stream all Engine events

**Purpose.** Watch resource lifecycle events until interrupted.

**Syntax.** `docker events [OPTIONS]`

**Options used.** No option required.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 020. docker events — Filter to container events

**Purpose.** Narrow the event stream to container lifecycle activity.

**Syntax.** `docker events [OPTIONS]`

**Options used.** `--filter type=container` limits the event type.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events --filter type=container
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 021. docker events — Filter to die events

**Purpose.** Focus on container exit events.

**Syntax.** `docker events [OPTIONS]`

**Options used.** `--filter event=die` selects matching event attributes.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events --filter event=die
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 022. docker events — Limit events to recent interval

**Purpose.** Review recent activity instead of streaming indefinitely.

**Syntax.** `docker events [OPTIONS]`

**Options used.** `--since` accepts supported timestamps or relative durations.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events --since 30m
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 023. docker events — Bound event query with until

**Purpose.** End the query at a specified time.

**Syntax.** `docker events [OPTIONS]`

**Options used.** `--since` and `--until` set time bounds; replace illustrative timestamps.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events --since 2025-01-01T10:00:00Z --until 2025-01-01T10:15:00Z
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 024. docker events — Filter by image reference

**Purpose.** Watch events associated with an image where event attributes support it.

**Syntax.** `docker events [OPTIONS]`

**Options used.** `--filter image=VALUE` narrows events.

**Arguments.** Optional filters and time bounds.

**Example.**

```
docker events --filter image=nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 025. docker system prune — Prune unused resources conservatively

**Purpose.** Remove stopped containers, unused networks, dangling images, and eligible build cache.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** Interactive confirmation is shown by default.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove containers, images, networks, cache, and possibly volumes. Review inventory and backups first.**

## 026. docker system prune — Include all unused images

**Purpose.** Expand cleanup beyond dangling images.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `-a, --all` includes all unused images.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove containers, images, networks, cache, and possibly volumes. Review inventory and backups first.**

## 027. docker system prune — Skip confirmation in automation

**Purpose.** Run a reviewed cleanup without a prompt.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `-f, --force` skips confirmation.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove containers, images, networks, cache, and possibly volumes. Review inventory and backups first.**

### 028. docker system prune — Prune with age filter

**Purpose.** Restrict eligible resources by age where supported.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `--filter until=...` applies a supported age filter; verify local semantics.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --filter 'until=168h'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove containers, images, networks, cache, and possibly volumes. Review inventory and backups first.**

### 029. docker system prune — Include unused volumes

**Purpose.** Include eligible unused volumes in cleanup where supported.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `--volumes` expands cleanup to eligible volumes.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --volumes
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove containers, images, networks, cache, and possibly volumes. Review inventory and backups first.**

# 02 · Containers: run, create & lifecycle

177 reference entries. Follow the examples in order and substitute environment-specific values.

### 030. docker run — Detached service

**Purpose.** Run a long-lived service in the background.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-d, --detach` backgrounds the process; `--name` assigns a stable name.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --name web nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 031. docker run — Auto-remove one-shot task

**Purpose.** Remove container metadata after its main process exits.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--rm` removes the container after exit.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm alpine:3.20 echo hello
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 032. docker run — Interactive shell

**Purpose.** Open an interactive terminal in a temporary Linux container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-i` keeps stdin open; `-t` allocates a pseudo-terminal.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm -it alpine:3.20 sh
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 033. docker run — Publish a host port

**Purpose.** Forward host port 8080 to container port 80.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-p HOST:CONTAINER` publishes TCP by default.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --name web -p 8080:80 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 034. docker run — Bind published port to loopback

**Purpose.** Expose a development service only on the local host interface.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-p HOST_IP:HOST_PORT:CONTAINER_PORT` limits the host-side bind address.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d -p 127.0.0.1:8080:80 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 035. docker run — Publish all exposed ports

**Purpose.** Publish image-declared ports to dynamically assigned host ports.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-P, --publish-all` publishes ports declared by image metadata.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d -P nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 036. docker run — Set one environment variable

**Purpose.** Configure the application without rebuilding the image.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-e, --env KEY=VALUE` sets an environment variable.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm -e APP_MODE=development alpine:3.20 env
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 037. docker run — Read environment from file

**Purpose.** Load multiple environment settings from a file.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--env-file FILE` reads `KEY=VALUE` entries.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --env-file ./app.env alpine:3.20 env
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 038. docker run — Mount a named volume

**Purpose.** Persist database data outside the container writable layer.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-v NAME:PATH` mounts a named volume and creates it if absent.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --name db -v dbdata:/var/lib/postgresql/data postgres:16
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 039. docker run — Use explicit volume mount syntax

**Purpose.** Express volume type/source/destination with key-value syntax.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--mount type=volume,src=...,dst=...` defines the mount explicitly.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --mount type=volume,src=dbdata,dst=/var/lib/postgresql/data postgres:16
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 040. docker run — Connect to custom network

**Purpose.** Attach a new container to an application network.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--network NAME` selects the network at creation.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --name api --network app-net nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 041. docker run — Set restart policy

**Purpose.** Restart a service after daemon restart or process exit.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--restart` accepts policies such as `no`, `on-failure`, `always`, and `unless-stopped`.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --restart unless-stopped --name worker alpine:3.20 sleep 3600
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 042. docker run — Limit memory

**Purpose.** Set a memory ceiling for the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--memory SIZE` sets a memory limit.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --memory 512m nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 043. docker run — Limit CPU quota

**Purpose.** Restrict CPU usage to an approximate CPU count.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--cpus NUMBER` sets CPU quota in CPU units.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --cpus 1.5 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 044. docker run — Pin to selected CPUs

**Purpose.** Restrict execution to specified host CPU IDs.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--cpuset-cpus` accepts CPU IDs/ranges such as `0-3` or `0,2`.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --cpuset-cpus 0,2 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 045. docker run — Read-only root filesystem

**Purpose.** Block writes to the root filesystem while retaining a temporary writable path.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--read-only` makes root filesystem read-only; `--tmpfs PATH` adds temporary storage.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --read-only --tmpfs /tmp nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 046. docker run — Run as non-root UID

**Purpose.** Run a process with a specific UID/GID.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--user UID[:GID]` overrides the runtime user.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --user 10001:10001 alpine:3.20 id
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 047. docker run — Set working directory

**Purpose.** Set the process working directory inside the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-w, --workdir DIR` sets working directory.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --workdir /tmp alpine:3.20 pwd
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 048. docker run — Override entrypoint

**Purpose.** Run a different executable than the image entrypoint.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--entrypoint EXEC` overrides ENTRYPOINT; trailing words become its arguments.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --entrypoint sh alpine:3.20 -c 'echo diagnostic'
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 049. docker run — Override default command

**Purpose.** Replace the image CMD with a one-off command.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** Words after the image are command/arguments, subject to ENTRYPOINT.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm alpine:3.20 uname -a
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 050. docker run — Add a host mapping

**Purpose.** Resolve a hostname to a chosen IP inside the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--add-host HOST:IP` adds a host mapping.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --add-host api.local:192.0.2.10 alpine:3.20 getent hosts api.local
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 051. docker run — Set DNS resolver

**Purpose.** Specify a DNS server for the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--dns IP` sets a resolver and may be repeated.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --dns 1.1.1.1 alpine:3.20 cat /etc/resolv.conf
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 052. docker run — Set DNS search domain

**Purpose.** Add a DNS search suffix.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--dns-search DOMAIN` sets a search suffix.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --dns-search corp.example alpine:3.20 cat /etc/resolv.conf
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 053. docker run — Set hostname

**Purpose.** Assign a hostname visible inside the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--hostname NAME` sets the container hostname.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --hostname batch-01 alpine:3.20 hostname
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 054. docker run — Add metadata labels

**Purpose.** Attach searchable metadata to a container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--label KEY=VALUE` adds metadata; repeat for multiple labels.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --label team=platform --label tier=dev nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 055. docker run — Configure health check

**Purpose.** Define a periodic command used to report container health.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--health-cmd` sets the probe; `--health-interval` sets its interval.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --health-cmd='wget -q -O /dev/null http://127.0.0.1/ || exit 1' \
--health-interval=30s nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 056. docker run — Disable inherited health check

**Purpose.** Disable a health check inherited from image metadata.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--no-healthcheck` disables the image health check for this container.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --no-healthcheck nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 057. docker run — Set stop grace period

**Purpose.** Allow a process time to shut down before forced termination.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--stop-timeout SECONDS` sets the stop timeout.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --stop-timeout 30 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 058. docker run — Set PID limit

**Purpose.** Limit process creation inside the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--pids-limit N` sets a PID ceiling on supported platforms.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --pids-limit 256 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 059. docker run — Drop all capabilities and add one

**Purpose.** Start with minimal Linux capabilities and add a required capability.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--cap-drop` removes capabilities; `--cap-add` adds selected ones.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --cap-drop ALL --cap-add NET_BIND_SERVICE nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 060. docker run — Enable no-new-privileges

**Purpose.** Prevent processes from gaining additional privileges through exec transitions on supported Linux hosts.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--security-opt KEY=VALUE` sets supported runtime security options.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --security-opt no-new-privileges:true alpine:3.20 id
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 061. docker run — Use host networking

**Purpose.** Share the host network namespace where supported.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--network host` uses host networking on supported Linux Engine setups.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --network host alpine:3.20 ip addr
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 062. docker run — Increase shared memory

**Purpose.** Increase `/dev/shm` for browser/test/database workloads.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--shm-size SIZE` configures shared-memory capacity.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --shm-size 1g alpine:3.20 df -h /dev/shm
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 063. docker run — Use init process

**Purpose.** Add a small init process to forward signals and reap children.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--init` enables a small init process as PID 1.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --init alpine:3.20 ps
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 064. docker run — Select image platform

**Purpose.** Request a specific OS/architecture variant.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--platform OS/ARCH[/VARIANT]` selects a platform.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --platform linux/arm64 alpine:3.20 uname -m
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 065. docker run — Select logging driver

**Purpose.** Choose a logging driver for a container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--log-driver DRIVER` selects a supported logging driver.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --log-driver local nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 066. docker run — Configure JSON log rotation

**Purpose.** Bound log file size and number of retained files.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--log-opt KEY=VALUE` passes driver-specific options.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --log-driver json-file --log-opt max-size=10m --log-opt max-file=3 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 067. docker run — Request GPU devices

**Purpose.** Expose compatible GPU devices to a container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--gpus all` requests all available GPUs; host runtime integration is required.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 068. docker run — Constrain memory and swap together

**Purpose.** Set memory limit and total memory-plus-swap policy.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--memory` caps RAM; `--memory-swap` sets RAM plus swap total.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --memory 512m --memory-swap 512m nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 069. docker run — Use a deterministic name

**Purpose.** Assign a unique readable name for stable scripts.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--name NAME` assigns a unique container name.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --name frontend nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 070. docker run — Run a shell expression in container

**Purpose.** Use the image shell for a multi-command one-shot job.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `sh -c` is the command executed inside the container.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm alpine:3.20 sh -c 'date; id'
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 071. docker run — Set custom stop signal

**Purpose.** Choose a signal sent before Docker force-kills a stopped container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--stop-signal SIGNAL` sets the stop signal.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --stop-signal SIGQUIT nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 072. docker run — Configure OOM-killer behavior

**Purpose.** Change OOM-killer behavior on supported Linux Engine setups.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--oom-kill-disable` disables the OOM killer for this container where supported; pair with a memory limit.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --oom-kill-disable --memory 1g nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 073. docker run — Expose an internal port in metadata

**Purpose.** Document a port without publishing it to the host.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--expose PORT[/PROTO]` exposes a port to linked/network peers but does not publish it to the host.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --expose 8080 nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 074. docker run — Set domain name

**Purpose.** Set a container domain name.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--domainname NAME` sets the container domain name.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --domainname corp.example alpine:3.20 hostname -d
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 075. docker run — Set a MAC address

**Purpose.** Request a MAC address on a supported network.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--mac-address MAC` sets the container MAC address where supported.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --mac-address 02:42:ac:11:00:02 alpine:3.20 ip link
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 076. docker run — Set a sysctl

**Purpose.** Configure a supported namespaced kernel parameter.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--sysctl KEY=VALUE` sets supported namespaced sysctls; syntax/allowlist is platform-dependent.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --sysctl 'net.ipv4.ip_local_port_range=1024 65000' alpine:3.20 true
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 077. docker run — Set an IPC namespace mode

**Purpose.** Choose IPC namespace sharing behavior for specialized workloads.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--ipc MODE` sets IPC namespace mode, such as `private` or supported alternatives.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --ipc private alpine:3.20 true
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 078. docker run — Set PID namespace mode

**Purpose.** Choose PID namespace sharing behavior.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--pid MODE` selects the PID namespace mode where supported.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --pid private alpine:3.20 ps
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 079. docker run — Set user namespace mode

**Purpose.** Select user namespace behavior where daemon/runtime supports it.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--userns MODE` selects user namespace behavior; availability is platform-specific.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --userns=host alpine:3.20 id
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 080. docker run — Limit block I/O read rate

**Purpose.** Set a read-rate limit for a block device where supported.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--device-read-bps DEVICE:RATE` sets a block-device read rate limit on supported Linux hosts.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --device-read-bps /dev/sda:10mb nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 081. docker run — Limit number of open files

**Purpose.** Set an `ulimit` for file descriptors.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--ulimit name=soft[:hard]` sets a resource limit.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --ulimit nofile=1024:2048 alpine:3.20 sh -c 'ulimit -n'
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 082. docker run — Set cgroup parent

**Purpose.** Place container processes under a specified cgroup parent where supported.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--cgroup-parent NAME` selects the parent cgroup on supported Linux setups.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --cgroup-parent my.slice nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 083. docker run — Use a custom runtime

**Purpose.** Select an installed runtime by name.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--runtime NAME` selects a configured runtime.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --runtime runc alpine:3.20 true
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 084. docker run — Set detach key sequence

**Purpose.** Change the key sequence used to detach the client from an attached session.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--detach-keys` overrides the client detach sequence.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -it --detach-keys='ctrl-x,x' alpine:3.20 sh
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 085. docker run — Disable signal proxying

**Purpose.** Change whether signals from the client are proxied to the container process.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--sig-proxy=false` disables signal proxying for this attached run.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -it --sig-proxy=false alpine:3.20 sh
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 086. docker run — Write container ID to a file

**Purpose.** Record the newly created container ID for automation.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--cidfile FILE` writes the container ID to a file.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --cidfile ./web.cid nginx:alpine
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 087. docker run — Set container restart maximum

**Purpose.** Limit retries for an on-failure restart policy.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--restart on-failure:5` retries on non-zero exit up to the configured retry count.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run -d --restart on-failure:5 alpine:3.20 sh -c 'exit 1'
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 088. docker run — Mount a bind path with explicit syntax

**Purpose.** Mount a host directory into a container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `type=bind` selects bind mount; `src` is host path and `dst` is container path.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --mount type=bind,src="$PWD",dst=/workspace alpine:3.20 ls /workspace
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 089. docker run — Mount bind path read-only

**Purpose.** Share host config files without permitting writes through the mount.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `readonly` marks the bind mount read-only.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --mount type=bind,src="$PWD/config",dst=/config,readonly alpine:3.20 \
ls /config
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 090. docker run — Use tmpfs with options

**Purpose.** Create ephemeral in-memory storage for temporary files.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--tmpfs PATH:OPTIONS` configures a tmpfs mount on supported platforms.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --tmpfs /tmp:rw,noexec,nosuid,size=64m alpine:3.20 df -h /tmp
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 091. docker run — Use custom entrypoint and args

**Purpose.** Run a shell for a one-off diagnostic while retaining image filesystem.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--entrypoint` sets executable; remaining arguments are passed to it.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm --entrypoint /bin/sh alpine:3.20 -c 'cat /etc/os-release'
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 092. docker run — Set environment variable from host

**Purpose.** Pass an already-exported host variable into the container.

**Syntax.** `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-e KEY` copies the value from the client environment when available.

**Arguments.** IMAGE is a repository/tag or digest; trailing COMMAND/ARG values override CMD subject to ENTRYPOINT.

**Example.**

```
docker run --rm -e BUILD_ID alpine:3.20 printenv BUILD_ID
```

**Expected result.** A new container is created and started. The exact output depends on detached/interactive mode and the image process.

**Troubleshooting.** If it exits immediately, use `docker ps -a`, `docker logs NAME`, and `docker inspect NAME`. Port conflicts, missing image tags, permission issues, and platform mismatches are common.

**Related.** Related: `docker create`, `docker start`, `docker exec`, `docker logs`, `docker inspect`, `docker network`, `docker volume`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 093. docker ps — List running containers

**Purpose.** Show containers currently running.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** Default table shows ID, image, command, status, ports, and name.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 094. docker ps — Include stopped containers

**Purpose.** Review exited containers during triage.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `-a, --all` includes stopped containers.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 095. docker ps — Print container IDs only

**Purpose.** Produce compact IDs for scripts.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `-q, --quiet` prints IDs only.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -q
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 096. docker ps — Filter by name

**Purpose.** Find containers matching a name filter.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter name=VALUE` filters names.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -a --filter name=web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 097. docker ps — Filter by image ancestor

**Purpose.** Find containers created from an image reference or descendant.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter ancestor=IMAGE` filters image ancestry.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -a --filter ancestor=nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 098. docker ps — Filter by status

**Purpose.** List exited containers.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter status=STATUS` supports documented states such as running, exited, created, paused.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -a --filter status=exited
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 099. docker ps — Filter by label

**Purpose.** Find resources marked with metadata.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter label=KEY[=VALUE]` matches labels.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --filter label=team=platform
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 100. docker ps — Show most recently created

**Purpose.** Inspect the latest container record.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `-l, --latest` selects latest; `-a` includes stopped.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -l -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 101. docker ps — Show last five containers

**Purpose.** Review recent container churn.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `-n, --last N` limits the listing to the last N.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -n 5 -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 102. docker ps — Format rows as JSON

**Purpose.** Emit newline-delimited JSON objects for automation.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--format TEMPLATE` uses Go templates; `json` serializes each row.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --format '{{json .}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

### 103. docker ps — Disable output truncation

**Purpose.** Show full IDs and command text.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--no-trunc` disables truncation.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --no-trunc
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

### 104. docker ps — Filter by published port

**Purpose.** Find containers publishing a host/container port.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter publish=PORT[/PROTO]` filters published ports.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --filter publish=8080
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

### 105. docker ps — Filter by network

**Purpose.** List containers attached to a named network.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter network=NETWORK` matches network membership.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --filter network=app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

### 106. docker ps — Filter by exit code

**Purpose.** Find stopped containers with exit code 1.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter exited=CODE` filters stopped containers by exit code.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps -a --filter exited=1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 107. docker ps — Filter by health status

**Purpose.** Find containers reporting unhealthy where supported by the filter.

**Syntax.** `docker ps [OPTIONS]`

**Options used.** `--filter health=STATUS` selects health state on supported CLI/Engine versions.

**Arguments.** No positional arguments; filters use `--filter KEY=VALUE`.

**Example.**

```
docker ps --filter health=unhealthy
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** An empty list may simply mean no container matches; add `-a` to include stopped containers.

**Related.** Related: `docker inspect`, `docker logs`, `docker stats`, `docker events`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 108. docker logs — Read captured logs

**Purpose.** Read stdout/stderr captured by the configured logging driver.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** No option required.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 109. docker logs — Follow logs live

**Purpose.** Stream new lines while reproducing an issue.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `-f, --follow` streams new output.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --follow web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 110. docker logs — Show last 100 lines

**Purpose.** Reduce noise during triage.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `--tail N` prints the last N lines.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --tail 100 web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 111. docker logs — Show logs from last ten minutes

**Purpose.** Focus on a recent time window.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `--since` accepts supported relative durations or timestamps.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --since 10m web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 112. docker logs — Include timestamps

**Purpose.** Correlate log lines with events and monitoring.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `-t, --timestamps` prefixes timestamps.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --timestamps web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 113. docker logs — Follow recent lines with timestamps

**Purpose.** Start with recent history and stream new lines.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `--tail`, `--follow`, and `--timestamps` control initial history, streaming, and timestamps.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --tail 50 --follow --timestamps web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 114. docker logs — Bound logs by start/end time

**Purpose.** Review a precise incident interval.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `--since` and `--until` set time bounds; replace example timestamps.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --since 2025-01-01T10:00:00Z --until 2025-01-01T10:15:00Z web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 115. docker logs — Include logging metadata

**Purpose.** Show extra attributes where the driver provides them.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `--details` adds extra attributes.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs --details web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 116. docker logs — Read logs from stopped container

**Purpose.** Diagnose a process that exited but was not removed.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** Stopped containers can be queried while they still exist.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs failed-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 117. docker logs — Follow only errors by shell filter

**Purpose.** Filter log output using a host shell utility.

**Syntax.** `docker logs [OPTIONS] CONTAINER`

**Options used.** `-f` follows logs; `grep` is a host shell utility, not a Docker option.

**Arguments.** CONTAINER is a name or ID.

**Example.**

```
docker logs -f api | grep -i error
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Some logging drivers do not support `docker logs`; application file logs may live elsewhere. A quiet stream is not proof of failure.

**Related.** Related: `docker inspect`, `docker events`, `docker stats`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 118. docker exec — Run a diagnostic command

**Purpose.** Execute a one-off process inside a running container.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** No option required.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec web nginx -t
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 119. docker exec — Open an interactive shell

**Purpose.** Get a terminal in a running container.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `-i` keeps stdin open; `-t` allocates a terminal.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec -it web sh
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 120. docker exec — Run as a specific user

**Purpose.** Run a diagnostic as a non-root UID/GID.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `-u, --user UID[:GID]` overrides exec user.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec --user 10001:10001 web id
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 121. docker exec — Set working directory

**Purpose.** Run from an application directory.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `-w, --workdir DIR` sets process working directory.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec --workdir /app web pwd
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 122. docker exec — Set temporary environment variable

**Purpose.** Pass a value to this exec process only.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `-e KEY=VALUE` sets an exec-only environment variable.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec -e LOG_LEVEL=debug web printenv LOG_LEVEL
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 123. docker exec — Load environment file

**Purpose.** Supply several temporary environment variables.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `--env-file FILE` loads variables where supported by installed version.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec --env-file ./debug.env web printenv
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 124. docker exec — Run detached exec task

**Purpose.** Start a background task inside a running container.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `-d, --detach` returns without waiting for output.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec -d web sh -c 'sleep 60'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 125. docker exec — Set detach keys

**Purpose.** Change the key sequence used to detach from the exec client.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `--detach-keys` overrides the client detach sequence.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec -it --detach-keys='ctrl-x,x' web sh
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 126. docker exec — Check database readiness

**Purpose.** Run the image-provided readiness probe.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** The executable must exist in the container.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec db pg_isready
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 127. docker exec — Check filesystem usage

**Purpose.** Inspect mounted filesystem usage from the container namespace.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** The `df` executable runs inside the container.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec web df -h
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 128. docker exec — Inspect environment safely by key

**Purpose.** Read one known variable without dumping the entire environment.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `printenv` is the in-container command; no Docker flag required.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec api printenv APP_MODE
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 129. docker exec — Run command as root explicitly

**Purpose.** Use root only for a controlled repair task.

**Syntax.** `docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** `--user 0` runs the exec process as UID 0.

**Arguments.** CONTAINER must be running; COMMAND must exist inside it.

**Example.**

```
docker exec --user 0 api id
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Exec fails when the container is not running or the executable is missing. Minimal images may lack bash, curl, ps, or diagnostic tools.

**Related.** Related: `docker attach`, `docker logs`, `docker top`, `docker cp`

**Safety.** Interactive production access should be controlled; commands inside a container can modify application state.

## 130. docker inspect — Inspect full container JSON

**Purpose.** Read state, config, mounts, and network details.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `--type` disambiguates resource type; `--format` selects fields.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 131. docker inspect — Extract container IP address

**Purpose.** Read IPs assigned on attached networks.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `-f, --format` uses Go templates.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 132. docker inspect — Inspect health status

**Purpose.** Read health status and recent probe output.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `-f` selects health object; it may be absent without a health check.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect -f '{{json .State.Health}}' web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 133. docker inspect — Inspect restart policy

**Purpose.** Verify the configured restart policy.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `-f` selects restart policy metadata.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect -f '{{json .HostConfig.RestartPolicy}}' web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 134. docker inspect — Inspect mount points

**Purpose.** Identify volume and bind mount sources/destinations.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `-f` selects mount metadata.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect -f '{{json .Mounts}}' db
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 135. docker inspect — Inspect labels

**Purpose.** Read labels used by filters and automation.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `-f` selects labels as JSON.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect -f '{{json .Config.Labels}}' web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 136. docker inspect — Inspect image entrypoint and CMD

**Purpose.** Check default executable and arguments configured in an image.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `docker image inspect` inspects image metadata; `-f` formats fields.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker image inspect -f '{{json .Config.Entrypoint}} {{json .Config.Cmd}}' nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 137. docker inspect — Disambiguate object type

**Purpose.** Limit inspection to container objects.

**Syntax.** `docker inspect [OPTIONS] NAME|ID [NAME|ID...]`

**Options used.** `--type TYPE` limits inspection to the selected object type.

**Arguments.** One or more resource names or IDs.

**Example.**

```
docker inspect --type container web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** `No such object` usually means the name/ID is wrong or the resource was removed. Output can include environment variables and other sensitive metadata.

**Related.** Related: `docker ps -a`, `docker image inspect`, `docker network inspect`, `docker volume inspect`

**Safety.** Inspection output may disclose environment variables, paths, and operational metadata.

## 138. docker create — Create without starting

**Purpose.** Prepare a container for later start.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--name` assigns a name; create does not start the container.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name staged nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 139. docker create — Create with port mapping

**Purpose.** Store a port mapping in the container configuration.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-p HOST:CONTAINER` records a published port mapping.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name staged -p 8080:80 nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 140. docker create — Create with persistent volume

**Purpose.** Prepare a container with a named volume.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-v NAME:PATH` attaches a named volume.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name db-staged -v dbdata:/var/lib/postgresql/data postgres:16
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 141. docker create — Create with environment

**Purpose.** Store an environment variable in the container configuration.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `-e KEY=VALUE` sets an environment variable.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name api-staged -e APP_MODE=production nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 142. docker create — Create on custom network

**Purpose.** Configure network attachment before starting.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--network NAME` selects a network.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name api-staged --network app-net nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 143. docker create — Create read-only root filesystem

**Purpose.** Prepare a container whose root filesystem is read-only.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--read-only` prevents writes to root filesystem.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name hardened --read-only nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 144. docker create — Create with memory limit

**Purpose.** Set a resource limit before the container starts.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--memory SIZE` sets the memory limit.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name limited --memory 256m nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 145. docker create — Create a one-shot task

**Purpose.** Configure a container to execute a single shell command later.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** Words after the image are the command and arguments.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name report alpine:3.20 sh -c 'date > /tmp/report.txt'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 146. docker create — Create with restart policy

**Purpose.** Store restart behavior before starting.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--restart POLICY` configures restart behavior.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name worker --restart unless-stopped alpine:3.20 sleep 3600
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 147. docker create — Create with labels

**Purpose.** Attach metadata at creation.

**Syntax.** `docker create [OPTIONS] IMAGE [COMMAND] [ARG...]`

**Options used.** `--label KEY=VALUE` adds metadata.

**Arguments.** IMAGE plus optional command/arguments.

**Example.**

```
docker create --name api --label team=platform nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: `docker run`, `docker start`, `docker inspect`

**Safety.** Routine operation; verify the selected daemon/context and target.

## 148. docker start — Start a stopped container

**Purpose.** Start a container without creating a new one.

**Syntax.** `docker start [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** One or more existing container names/IDs.

**Example.**

```
docker start web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 149. docker start — Start and attach output

**Purpose.** Watch stdout/stderr while starting a container.

**Syntax.** `docker start [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-a, --attach` attaches output.

**Arguments.** One or more existing container names/IDs.

**Example.**

```
docker start -a batch-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 150. docker start — Start with stdin attached

**Purpose.** Reattach stdin for an interactive container.

**Syntax.** `docker start [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-a` attaches output; `-i` attaches stdin.

**Arguments.** One or more existing container names/IDs.

**Example.**

```
docker start -ai shellbox
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 151. docker start — Start several containers

**Purpose.** Start a group of existing containers.

**Syntax.** `docker start [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Each positional argument is a container name/ID.

**Arguments.** One or more existing container names/IDs.

**Example.**

```
docker start api worker scheduler
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 152. docker stop — Gracefully stop a container

**Purpose.** Send the configured stop signal before forced termination.

**Syntax.** `docker stop [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Uses the configured stop signal and default timeout.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker stop web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 153. docker stop — Set a 30-second timeout

**Purpose.** Allow a stateful process more time to shut down.

**Syntax.** `docker stop [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-t, --timeout SECONDS` overrides the grace period.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker stop --timeout 30 db
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 154. docker stop — Stop a group of containers

**Purpose.** Stop several workloads during maintenance.

**Syntax.** `docker stop [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Multiple container names/IDs are accepted.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker stop api worker scheduler
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 155. docker stop — Use short timeout for a disposable task

**Purpose.** Stop a disposable process with a short grace period.

**Syntax.** `docker stop [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-t 2` waits up to two seconds before forced termination.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker stop -t 2 test-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 156. docker restart — Restart a container

**Purpose.** Stop and start using the existing container configuration.

**Syntax.** `docker restart [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** No option required for the default timeout.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker restart api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 157. docker restart — Restart with longer timeout

**Purpose.** Give a stateful process more time to stop.

**Syntax.** `docker restart [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--timeout SECONDS` sets the stop grace period.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker restart --timeout 45 db
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 158. docker restart — Restart multiple containers

**Purpose.** Restart a reviewed group of containers.

**Syntax.** `docker restart [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Each argument names a container.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker restart api worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 159. docker kill — Force-stop a stuck container

**Purpose.** Send SIGKILL immediately.

**Syntax.** `docker kill [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Default signal is SIGKILL.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker kill hung-worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 160. docker kill — Send SIGTERM

**Purpose.** Send a chosen signal instead of the default.

**Syntax.** `docker kill [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-s, --signal SIGNAL` selects a signal.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker kill --signal=SIGTERM api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 161. docker kill — Send HUP for supported reload

**Purpose.** Ask an application designed for it to reload on SIGHUP.

**Syntax.** `docker kill [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--signal=HUP` sends HUP; behavior is application-specific.

**Arguments.** One or more running container names/IDs.

**Example.**

```
docker kill --signal=HUP web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 162. docker rm — Remove a stopped container

**Purpose.** Delete container metadata and writable layer.

**Syntax.** `docker rm [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Stopped containers can be removed without force.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker rm old-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 163. docker rm — Force-remove a running container

**Purpose.** Force removal of a running container.

**Syntax.** `docker rm [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-f, --force` force-removes a running container.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker rm -f test-api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 164. docker rm — Remove associated anonymous volumes

**Purpose.** Remove anonymous volumes attached to a container.

**Syntax.** `docker rm [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-v` removes associated anonymous volumes; named volumes are not removed by this flag.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker rm -v old-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 165. docker rm — Remove several stopped containers

**Purpose.** Remove a reviewed set of disposable containers.

**Syntax.** `docker rm [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Multiple names/IDs are accepted.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker rm test-a test-b test-c
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 166. docker pause — Freeze one container

**Purpose.** Suspend processes using the runtime freezer mechanism.

**Syntax.** `docker pause CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Running container names/IDs.

**Example.**

```
docker pause batch-worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 167. docker pause — Pause a group

**Purpose.** Temporarily freeze several containers.

**Syntax.** `docker pause CONTAINER [CONTAINER...]`

**Options used.** Multiple names/IDs are accepted.

**Arguments.** Running container names/IDs.

**Example.**

```
docker pause worker-a worker-b
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 168. docker unpause — Resume one container

**Purpose.** Allow a paused container process to continue.

**Syntax.** `docker unpause CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Paused container names/IDs.

**Example.**

```
docker unpause batch-worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 169. docker unpause — Resume a group

**Purpose.** Resume multiple paused containers.

**Syntax.** `docker unpause CONTAINER [CONTAINER...]`

**Options used.** Multiple names/IDs are accepted.

**Arguments.** Paused container names/IDs.

**Example.**

```
docker unpause worker-a worker-b
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 170. docker rename — Rename a container

**Purpose.** Give a container a clearer operational name.

**Syntax.** `docker rename CONTAINER NEW_NAME`

**Options used.** Second argument is the new unique name.

**Arguments.** Existing container and unique new name.

**Example.**

```
docker rename web web-blue
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 171. docker rename — Rename a stopped container

**Purpose.** Standardize a stopped container name before restarting.

**Syntax.** `docker rename CONTAINER NEW_NAME`

**Options used.** Container may be stopped or running; new name must be unique.

**Arguments.** Existing container and unique new name.

**Example.**

```
docker rename old-api api-canary
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 172. docker update — Change restart policy

**Purpose.** Update restart behavior without recreating the container.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--restart POLICY` sets a supported restart policy.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --restart unless-stopped api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 173. docker update — Change memory limit

**Purpose.** Adjust a container memory cap.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--memory SIZE` updates memory limit where supported.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --memory 768m api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 174. docker update — Change CPU quota

**Purpose.** Adjust CPU quota for a container.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--cpus NUMBER` sets CPU quota where supported.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --cpus 2 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 175. docker update — Set PID limit

**Purpose.** Limit the number of processes.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--pids-limit N` sets the PID limit where supported.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --pids-limit 512 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 176. docker update — Update CPU shares

**Purpose.** Change relative CPU weight under contention.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--cpu-shares N` sets relative CPU weight, not a hard CPU count.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --cpu-shares 512 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 177. docker update — Update memory reservation

**Purpose.** Set a soft memory reservation where supported.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--memory-reservation SIZE` sets a soft reservation.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --memory-reservation 256m api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 178. docker update — Update stop timeout

**Purpose.** Change the graceful-stop wait period where supported.

**Syntax.** `docker update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--stop-timeout SECONDS` configures stop timeout.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker update --stop-timeout 30 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 179. docker wait — Wait for one job exit code

**Purpose.** Block until a container exits and print its exit code.

**Syntax.** `docker wait CONTAINER [CONTAINER...]`

**Options used.** The command blocks until the container stops.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker wait batch-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 180. docker wait — Wait for multiple jobs

**Purpose.** Collect exit codes from several containers.

**Syntax.** `docker wait CONTAINER [CONTAINER...]`

**Options used.** Each argument names a container.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker wait job-a job-b
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 181. docker top — List container processes

**Purpose.** View processes associated with a container from the host.

**Syntax.** `docker top CONTAINER [ps OPTIONS]`

**Options used.** No option required.

**Arguments.** Container name/ID and optional host `ps` arguments.

**Example.**

```
docker top api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 182. docker top — Show PID, parent PID, and command

**Purpose.** Use host process-list syntax to select columns.

**Syntax.** `docker top CONTAINER [ps OPTIONS]`

**Options used.** `-eo` is passed to host `ps`; flags vary by operating system.

**Arguments.** Container name/ID and optional host `ps` arguments.

**Example.**

```
docker top api -eo pid,ppid,comm
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 183. docker port — List all published mappings

**Purpose.** Find host-to-container port mappings.

**Syntax.** `docker port CONTAINER [PRIVATE_PORT[/PROTO]]`

**Options used.** No second argument lists all mappings.

**Arguments.** Container name/ID and optional private port/protocol.

**Example.**

```
docker port web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 184. docker port — Find a TCP mapping

**Purpose.** Query one private port and protocol.

**Syntax.** `docker port CONTAINER [PRIVATE_PORT[/PROTO]]`

**Options used.** `PRIVATE_PORT/PROTO` selects the mapping.

**Arguments.** Container name/ID and optional private port/protocol.

**Example.**

```
docker port web 80/tcp
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 185. docker stats — Monitor all running containers

**Purpose.** Watch CPU, memory, network, block I/O, and process usage.

**Syntax.** `docker stats [OPTIONS] [CONTAINER...]`

**Options used.** No option required; output streams live.

**Arguments.** Optional container names/IDs.

**Example.**

```
docker stats
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 186. docker stats — Take one snapshot

**Purpose.** Collect one sample for a report.

**Syntax.** `docker stats [OPTIONS] [CONTAINER...]`

**Options used.** `--no-stream` disables continuous updates.

**Arguments.** Optional container names/IDs.

**Example.**

```
docker stats --no-stream
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 187. docker stats — Monitor one container

**Purpose.** Focus metrics on a single workload.

**Syntax.** `docker stats [OPTIONS] [CONTAINER...]`

**Options used.** Container name/ID narrows the target.

**Arguments.** Optional container names/IDs.

**Example.**

```
docker stats api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 188. docker stats — Format selected fields

**Purpose.** Print name, CPU, and memory in a compact row.

**Syntax.** `docker stats [OPTIONS] [CONTAINER...]`

**Options used.** `--format TEMPLATE` selects supported fields.

**Arguments.** Optional container names/IDs.

**Example.**

```
docker stats --no-stream --format '{{.Name}} {{.CPUPerc}} {{.MemUsage}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 189. docker cp — Copy file from container to host

**Purpose.** Retrieve a diagnostic artifact.

**Syntax.** `docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH | SRC_PATH CONTAINER:DEST_PATH`

**Options used.** Container source uses `CONTAINER:PATH`.

**Arguments.** One side is a host path; the other is `CONTAINER:PATH`.

**Example.**

```
docker cp api:/tmp/report.txt ./report.txt
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 190. docker cp — Copy directory into container

**Purpose.** Copy local config into an existing container.

**Syntax.** `docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH | SRC_PATH CONTAINER:DEST_PATH`

**Options used.** Host path is source; container path is destination.

**Arguments.** One side is a host path; the other is `CONTAINER:PATH`.

**Example.**

```
docker cp ./config api:/tmp/config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 191. docker cp — Copy with archive semantics

**Purpose.** Request archive ownership/permission behavior.

**Syntax.** `docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH | SRC_PATH CONTAINER:DEST_PATH`

**Options used.** `-a` requests archive semantics where supported.

**Arguments.** One side is a host path; the other is `CONTAINER:PATH`.

**Example.**

```
docker cp -a ./artifact api:/tmp/artifact
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 192. docker cp — Copy from stopped container

**Purpose.** Extract a file from an exited container that still exists.

**Syntax.** `docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH | SRC_PATH CONTAINER:DEST_PATH`

**Options used.** Container need not be running if it still exists.

**Arguments.** One side is a host path; the other is `CONTAINER:PATH`.

**Example.**

```
docker cp failed-job:/var/log/app.log ./app.log
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 193. docker diff — List writable-layer changes

**Purpose.** Compare the writable layer against the image baseline.

**Syntax.** `docker diff CONTAINER`

**Options used.** No option required; A=added, C=changed, D=deleted.

**Arguments.** One container name/ID.

**Example.**

```
docker diff api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 194. docker diff — Filter changes to configuration paths

**Purpose.** Use a host shell filter to narrow changed paths.

**Syntax.** `docker diff CONTAINER`

**Options used.** `grep` is a host-shell utility, not a Docker subcommand.

**Arguments.** One container name/ID.

**Example.**

```
docker diff web | grep -E '(/etc|/usr/local)'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 195. docker attach — Attach to main process streams

**Purpose.** Connect to PID 1 stdout/stderr and possibly stdin.

**Syntax.** `docker attach [OPTIONS] CONTAINER`

**Options used.** Attaches to the existing process; does not create a shell.

**Arguments.** One running container name/ID.

**Example.**

```
docker attach worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 196. docker attach — Customize detach keys

**Purpose.** Detach from the client without stopping the container.

**Syntax.** `docker attach [OPTIONS] CONTAINER`

**Options used.** `--detach-keys` changes the client detach sequence.

**Arguments.** One running container name/ID.

**Example.**

```
docker attach --detach-keys='ctrl-p,ctrl-q' worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 197. docker commit — Commit filesystem snapshot

**Purpose.** Create an image from a container writable layer for debugging.

**Syntax.** `docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]`

**Options used.** Container is source; second argument names the image.

**Arguments.** Source container and optional destination image reference.

**Example.**

```
docker commit api debug/api:snapshot
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 198. docker commit — Add commit message

**Purpose.** Record why the snapshot image was created.

**Syntax.** `docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]`

**Options used.** `-m, --message TEXT` records a commit message.

**Arguments.** Source container and optional destination image reference.

**Example.**

```
docker commit --message 'capture failed upgrade' api debug/api:incident
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 199. docker commit — Add author metadata

**Purpose.** Record an author string in image metadata.

**Syntax.** `docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]`

**Options used.** `-a, --author AUTHOR` sets author metadata.

**Arguments.** Source container and optional destination image reference.

**Example.**

```
docker commit --author 'Platform Team' api debug/api:incident
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 200. docker export — Export root filesystem to tar

**Purpose.** Create a tar archive of the container filesystem, excluding mounted volume contents.

**Syntax.** `docker export [OPTIONS] CONTAINER`

**Options used.** `-o, --output FILE` writes the archive.

**Arguments.** One container name/ID.

**Example.**

```
docker export api -o api-rootfs.tar
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 201. docker export — Export and gzip through shell

**Purpose.** Compress rootfs archive using host shell tools.

**Syntax.** `docker export [OPTIONS] CONTAINER`

**Options used.** `gzip` and `>` are host-shell features; Docker emits tar to stdout.

**Arguments.** One container name/ID.

**Example.**

```
docker export api | gzip > api-rootfs.tar.gz
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 202. docker import — Import rootfs archive

**Purpose.** Create an image from a filesystem tarball.

**Syntax.** `docker import [OPTIONS] FILE|URL|- [REPOSITORY[:TAG]]`

**Options used.** Archive is source; last argument tags the new image.

**Arguments.** Archive source and optional image reference.

**Example.**

```
docker import api-rootfs.tar.gz local/rootfs:imported
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 203. docker import — Import with default command

**Purpose.** Apply supported image metadata changes while importing.

**Syntax.** `docker import [OPTIONS] FILE|URL|- [REPOSITORY[:TAG]]`

**Options used.** `--change` applies supported Dockerfile-like metadata changes.

**Arguments.** Archive source and optional image reference.

**Example.**

```
docker import --change 'CMD ["/bin/sh"]' rootfs.tar local/rootfs:shell
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 204. docker container prune — Remove all stopped containers

**Purpose.** Reclaim metadata and writable layers from stopped containers.

**Syntax.** `docker container prune [OPTIONS]`

**Options used.** Confirmation is shown by default.

**Arguments.** No positional arguments.

**Example.**

```
docker container prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 205. docker container prune — Prune stopped containers older than 24h

**Purpose.** Restrict cleanup to an age filter.

**Syntax.** `docker container prune [OPTIONS]`

**Options used.** `--filter until=24h` limits eligible containers by age.

**Arguments.** No positional arguments.

**Example.**

```
docker container prune --filter 'until=24h'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 206. docker container prune — Skip prune confirmation

**Purpose.** Run a reviewed cleanup non-interactively.

**Syntax.** `docker container prune [OPTIONS]`

**Options used.** `-f, --force` skips confirmation.

**Arguments.** No positional arguments.

**Example.**

```
docker container prune --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

# 03 · Images, registries & distribution

57 reference entries. Follow the examples in order and substitute environment-specific values.

### 207. docker image ls — List local images

**Purpose.** Show repositories, tags, IDs, creation times, and sizes.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** No option required.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 208. docker image ls — Include intermediate image records

**Purpose.** Show intermediate images as well as tagged images.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `-a, --all` includes intermediate images.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 209. docker image ls — Print image IDs only

**Purpose.** Produce compact image IDs for scripts.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `-q, --quiet` prints IDs only.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls -q
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 210. docker image ls — Filter dangling images

**Purpose.** Find untagged image records.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--filter dangling=true` selects dangling images.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --filter dangling=true
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 211. docker image ls — Filter repository name

**Purpose.** Narrow listing to a repository.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** Positional repository filters the listing.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls nginx
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 212. docker image ls — Show full image IDs

**Purpose.** Avoid truncation when comparing identifiers.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--no-trunc` prints full IDs.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --no-trunc
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 213. docker image ls — Format image listing

**Purpose.** Print selected fields in a compact format.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--format TEMPLATE` uses Go templates.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --format '{{.Repository}}:{{.Tag}} {{.ID}} {{.Size}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 214. docker image ls — Filter images by label

**Purpose.** Find images with a specified label.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--filter label=KEY[=VALUE]` matches image labels.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --filter label=org.opencontainers.image.source
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 215. docker image ls — Filter by reference pattern

**Purpose.** Narrow images using a reference filter.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--filter reference=PATTERN` filters repository/tag references.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --filter reference='nginx:*'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 216. docker image ls — List image digests

**Purpose.** Include repository digests in image output.

**Syntax.** `docker image ls [OPTIONS] [REPOSITORY[:TAG]]`

**Options used.** `--digests` displays digest information where available.

**Arguments.** Optional repository/tag filter.

**Example.**

```
docker image ls --digests
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 217. docker pull — Pull a tagged image

**Purpose.** Download an image from its registry.

**Syntax.** `docker pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** Image reference includes repository and tag.

**Arguments.** Image reference is repository/tag or digest-qualified reference.

**Example.**

```
docker pull nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 218. docker pull — Pull a specific platform

**Purpose.** Select an OS/architecture variant of a multi-platform image.

**Syntax.** `docker pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** `--platform OS/ARCH[/VARIANT]` selects a platform.

**Arguments.** Image reference is repository/tag or digest-qualified reference.

**Example.**

```
docker pull --platform linux/arm64 alpine:3.20
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 219. docker pull — Pull quietly

**Purpose.** Suppress progress output in a script.

**Syntax.** `docker pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** `-q, --quiet` suppresses progress details.

**Arguments.** Image reference is repository/tag or digest-qualified reference.

**Example.**

```
docker pull -q alpine:3.20
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 220. docker pull — Pull from private registry

**Purpose.** Fetch a private image by fully qualified reference.

**Syntax.** `docker pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** Reference includes registry host, namespace, repository, and tag.

**Arguments.** Image reference is repository/tag or digest-qualified reference.

**Example.**

```
docker pull registry.example.com/team/api:1.4.2
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 221. docker pull — Pull by digest

**Purpose.** Use a content-addressed immutable reference.

**Syntax.** `docker pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** Digest syntax is `@sha256:...`; replace the placeholder with a valid digest.

**Arguments.** Image reference is repository/tag or digest-qualified reference.

**Example.**

```
docker pull alpine@sha256:REPLACE_WITH_REAL_DIGEST
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 222. docker image rm — Remove one tagged reference

**Purpose.** Remove a local image tag/reference.

**Syntax.** `docker image rm [OPTIONS] IMAGE [IMAGE...]`

**Options used.** Target is repository:tag.

**Arguments.** Image ID or repository/tag reference.

**Example.**

```
docker image rm local/api:old
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 223. docker image rm — Remove by image ID

**Purpose.** Remove a local image record by identifier.

**Syntax.** `docker image rm [OPTIONS] IMAGE [IMAGE...]`

**Options used.** Argument is a full or unambiguous short image ID.

**Arguments.** Image ID or repository/tag reference.

**Example.**

```
docker image rm 0123456789ab
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 224. docker image rm — Force removal of reference

**Purpose.** Force image reference removal where Docker permits.

**Syntax.** `docker image rm [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-f, --force` forces removal in limited cases.

**Arguments.** Image ID or repository/tag reference.

**Example.**

```
docker image rm --force local/api:old
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 225. docker image rm — Remove multiple references

**Purpose.** Remove a reviewed set of local image references.

**Syntax.** `docker image rm [OPTIONS] IMAGE [IMAGE...]`

**Options used.** Multiple references may be supplied.

**Arguments.** Image ID or repository/tag reference.

**Example.**

```
docker image rm local/api:old local/worker:old
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 226. docker image prune — Remove dangling images

**Purpose.** Reclaim untagged images.

**Syntax.** `docker image prune [OPTIONS]`

**Options used.** Default targets dangling images.

**Arguments.** No positional arguments.

**Example.**

```
docker image prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 227. docker image prune — Remove all unused images

**Purpose.** Expand cleanup to all images unused by containers.

**Syntax.** `docker image prune [OPTIONS]`

**Options used.** `-a, --all` broadens the eligible image set.

**Arguments.** No positional arguments.

**Example.**

```
docker image prune -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 228. docker image prune — Prune images older than a cutoff

**Purpose.** Apply an age filter to eligible images.

**Syntax.** `docker image prune [OPTIONS]`

**Options used.** `--filter until=168h` applies a time cutoff.

**Arguments.** No positional arguments.

**Example.**

```
docker image prune -a --filter 'until=168h'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 229. docker image prune — Skip image-prune confirmation

**Purpose.** Run a reviewed prune in automation.

**Syntax.** `docker image prune [OPTIONS]`

**Options used.** `-f, --force` skips confirmation.

**Arguments.** No positional arguments.

**Example.**

```
docker image prune -f
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 230. docker tag — Tag a local image

**Purpose.** Add another reference to existing image content.

**Syntax.** `docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`

**Options used.** Second argument is the target repository/tag.

**Arguments.** Source and target image references.

**Example.**

```
docker tag local/api:dev local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 231. docker tag — Prepare a registry tag

**Purpose.** Create a local reference for publishing.

**Syntax.** `docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`

**Options used.** Target includes registry hostname and namespace.

**Arguments.** Source and target image references.

**Example.**

```
docker tag local/api:1.0.0 registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 232. docker tag — Create a latest alias

**Purpose.** Add a convenience tag to a release.

**Syntax.** `docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`

**Options used.** `latest` is an ordinary mutable tag, not a guaranteed newest version.

**Arguments.** Source and target image references.

**Example.**

```
docker tag local/api:1.0.0 local/api:latest
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 233. docker push — Push a tagged image

**Purpose.** Upload layers and manifest to a registry.

**Syntax.** `docker push [OPTIONS] NAME[:TAG]`

**Options used.** Argument is destination repository and tag.

**Arguments.** Destination image reference.

**Example.**

```
docker push registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 234. docker push — Push a versioned release

**Purpose.** Publish an explicit version tag.

**Syntax.** `docker push [OPTIONS] NAME[:TAG]`

**Options used.** Tag identifies the release; registry policy controls mutability.

**Arguments.** Destination image reference.

**Example.**

```
docker push registry.example.com/team/api:1.4.2
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 235. docker push — Push quietly

**Purpose.** Reduce progress output for automation where supported.

**Syntax.** `docker push [OPTIONS] NAME[:TAG]`

**Options used.** `-q, --quiet` suppresses progress details where available.

**Arguments.** Destination image reference.

**Example.**

```
docker push -q registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 236. docker save — Save one image to tar

**Purpose.** Export image layers and metadata for offline transfer.

**Syntax.** `docker save [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-o, --output FILE` writes an archive.

**Arguments.** One or more local image references.

**Example.**

```
docker save -o api-image.tar local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 237. docker save — Save multiple images

**Purpose.** Package several image references into one archive.

**Syntax.** `docker save [OPTIONS] IMAGE [IMAGE...]`

**Options used.** Each positional argument is an image reference.

**Arguments.** One or more local image references.

**Example.**

```
docker save -o release-images.tar local/api:1.0.0 local/worker:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 238. docker save — Save and gzip through shell

**Purpose.** Compress an image archive using a host utility.

**Syntax.** `docker save [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `gzip` and redirection are host-shell features.

**Arguments.** One or more local image references.

**Example.**

```
docker save local/api:1.0.0 | gzip > api-image.tar.gz
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 239. docker load — Load an image archive

**Purpose.** Restore an archive created by `docker save`.

**Syntax.** `docker load [OPTIONS]`

**Options used.** `-i, --input FILE` selects archive input.

**Arguments.** Use `-i FILE` or standard input.

**Example.**

```
docker load -i api-image.tar
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 240. docker load — Load a compressed archive

**Purpose.** Decompress a tar archive through the host shell.

**Syntax.** `docker load [OPTIONS]`

**Options used.** `gzip -dc` emits the archive to stdin.

**Arguments.** Use `-i FILE` or standard input.

**Example.**

```
gzip -dc api-image.tar.gz | docker load
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 241. docker load — Load quietly

**Purpose.** Reduce output during scripted image loading.

**Syntax.** `docker load [OPTIONS]`

**Options used.** `-q, --quiet` suppresses progress where supported.

**Arguments.** Use `-i FILE` or standard input.

**Example.**

```
docker load --quiet -i api-image.tar
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 242. docker history — Show image layer history

**Purpose.** Review layer commands and approximate sizes.

**Syntax.** `docker history [OPTIONS] IMAGE`

**Options used.** No option required.

**Arguments.** One image reference.

**Example.**

```
docker history local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 243. docker history — Show full created-by commands

**Purpose.** Avoid truncation when reviewing build steps.

**Syntax.** `docker history [OPTIONS] IMAGE`

**Options used.** `--no-trunc` prints full IDs and command text.

**Arguments.** One image reference.

**Example.**

```
docker history --no-trunc local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 244. docker history — Format history output

**Purpose.** Select command and size fields.

**Syntax.** `docker history [OPTIONS] IMAGE`

**Options used.** `--format TEMPLATE` selects fields.

**Arguments.** One image reference.

**Example.**

```
docker history --format '{{.CreatedBy}} {{.Size}}' local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 245. docker search — Search public repositories

**Purpose.** Find image repositories by keyword.

**Syntax.** `docker search [OPTIONS] TERM`

**Options used.** TERM is the search phrase.

**Arguments.** TERM is a repository search term.

**Example.**

```
docker search nginx
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 246. docker search — Limit search results

**Purpose.** Reduce output to a shortlist.

**Syntax.** `docker search [OPTIONS] TERM`

**Options used.** `--limit N` caps returned results where supported.

**Arguments.** TERM is a repository search term.

**Example.**

```
docker search --limit 10 postgres
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 247. docker search — Filter automated builds

**Purpose.** Use a supported filter to narrow results.

**Syntax.** `docker search [OPTIONS] TERM`

**Options used.** `--filter` accepts documented search filters; availability can vary.

**Arguments.** TERM is a repository search term.

**Example.**

```
docker search --filter is-automated=true nginx
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 248. docker search — Filter official images

**Purpose.** Request official-image results where supported by the search API.

**Syntax.** `docker search [OPTIONS] TERM`

**Options used.** `--filter is-official=true` narrows results where supported.

**Arguments.** TERM is a repository search term.

**Example.**

```
docker search --filter is-official=true alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 249. docker image inspect — Inspect image metadata

**Purpose.** Read ID, architecture, labels, and config.

**Syntax.** `docker image inspect [OPTIONS] IMAGE [IMAGE...]`

**Options used.** No option required.

**Arguments.** One or more image references.

**Example.**

```
docker image inspect nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 250. docker image inspect — Read image architecture

**Purpose.** Extract OS and architecture.

**Syntax.** `docker image inspect [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-f` selects metadata fields.

**Arguments.** One or more image references.

**Example.**

```
docker image inspect -f '{{.Os}}/{{.Architecture}}' nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 251. docker image inspect — Read image labels

**Purpose.** Extract image labels as JSON.

**Syntax.** `docker image inspect [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-f` and `json` format the label map.

**Arguments.** One or more image references.

**Example.**

```
docker image inspect -f '{{json .Config.Labels}}' local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 252. docker image inspect — Read entrypoint and CMD

**Purpose.** Inspect default executable and arguments.

**Syntax.** `docker image inspect [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-f` selects entrypoint and command fields.

**Arguments.** One or more image references.

**Example.**

```
docker image inspect -f '{{json .Config.Entrypoint}} {{json .Config.Cmd}}' nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 253. docker manifest — Inspect a remote manifest

**Purpose.** Read manifest or manifest-list JSON.

**Syntax.** `docker manifest COMMAND [OPTIONS]`

**Options used.** `inspect` fetches registry metadata.

**Arguments.** Subcommands manage registry manifests and manifest lists.

**Example.**

```
docker manifest inspect nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 254. docker manifest — Inspect verbose platform metadata

**Purpose.** Review platform entries in a manifest list.

**Syntax.** `docker manifest COMMAND [OPTIONS]`

**Options used.** `--verbose` adds platform/image metadata where available.

**Arguments.** Subcommands manage registry manifests and manifest lists.

**Example.**

```
docker manifest inspect --verbose nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 255. docker manifest — Create a manifest list

**Purpose.** Create a local list from platform-specific image tags.

**Syntax.** `docker manifest COMMAND [OPTIONS]`

**Options used.** `create TARGET SOURCE...` builds a local manifest list.

**Arguments.** Subcommands manage registry manifests and manifest lists.

**Example.**

```
docker manifest create registry.example.com/team/app:1.0 \
registry.example.com/team/app:1.0-amd64 registry.example.com/team/app:1.0-arm64
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 256. docker manifest — Push a manifest list

**Purpose.** Publish a prepared manifest list.

**Syntax.** `docker manifest COMMAND [OPTIONS]`

**Options used.** Argument is the target manifest reference.

**Arguments.** Subcommands manage registry manifests and manifest lists.

**Example.**

```
docker manifest push registry.example.com/team/app:1.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 257. docker manifest — Annotate a platform entry

**Purpose.** Set platform metadata for a source in a manifest list.

**Syntax.** `docker manifest COMMAND [OPTIONS]`

**Options used.** `--os` and `--arch` specify platform metadata.

**Arguments.** Subcommands manage registry manifests and manifest lists.

**Example.**

```
docker manifest annotate registry.example.com/team/app:1.0 \
registry.example.com/team/app:1.0-arm64 --os linux --arch arm64
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 258. docker login — Login to Docker Hub

**Purpose.** Authenticate to the default registry.

**Syntax.** `docker login [OPTIONS] [SERVER]`

**Options used.** No server argument targets Docker Hub by default.

**Arguments.** Optional registry server; credentials may be prompted or read from stdin.

**Example.**

```
docker login
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 259. docker login — Login to a private registry

**Purpose.** Authenticate to a specific registry hostname.

**Syntax.** `docker login [OPTIONS] [SERVER]`

**Options used.** Server argument selects registry.

**Arguments.** Optional registry server; credentials may be prompted or read from stdin.

**Example.**

```
docker login registry.example.com
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 260. docker login — Use password from stdin

**Purpose.** Avoid placing the secret in command arguments.

**Syntax.** `docker login [OPTIONS] [SERVER]`

**Options used.** `--username` selects account; `--password-stdin` reads secret from stdin.

**Arguments.** Optional registry server; credentials may be prompted or read from stdin.

**Example.**

```
printf '%s' "$REGISTRY_TOKEN" | docker login registry.example.com --username deploy \
--password-stdin
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 261. docker login — Specify username and prompt

**Purpose.** Choose an account and let CLI request credentials.

**Syntax.** `docker login [OPTIONS] [SERVER]`

**Options used.** `-u, --username USER` sets account name.

**Arguments.** Optional registry server; credentials may be prompted or read from stdin.

**Example.**

```
docker login --username deploy registry.example.com
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 262. docker logout — Logout from default registry

**Purpose.** Remove locally stored authentication for default registry.

**Syntax.** `docker logout [SERVER]`

**Options used.** No argument targets default registry.

**Arguments.** Optional registry server.

**Example.**

```
docker logout
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 263. docker logout — Logout from private registry

**Purpose.** Remove credentials for a registry hostname.

**Syntax.** `docker logout [SERVER]`

**Options used.** Argument selects registry entry.

**Arguments.** Optional registry server.

**Example.**

```
docker logout registry.example.com
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

# 04 · Build, Dockerfile & BuildKit

49 reference entries. Follow the examples in order and substitute environment-specific values.

### 264. docker build — Build and tag from current directory

**Purpose.** Build the Dockerfile in the current context.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `-t, --tag` assigns repository/tag; `.` is build context.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 265. docker build — Use alternate Dockerfile

**Purpose.** Select a Dockerfile separate from the default.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `-f, --file FILE` selects Dockerfile; final `.` remains context.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build -f docker/Dockerfile -t local/api:test .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 266. docker build — Disable cache

**Purpose.** Force build steps to rerun instead of reusing cache.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--no-cache` disables build cache reuse.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --no-cache -t local/api:clean .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 267. docker build — Refresh base images

**Purpose.** Attempt to pull referenced base images before building.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--pull` attempts to pull base images.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --pull -t local/api:latest .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 268. docker build — Pass build argument

**Purpose.** Set a Dockerfile ARG value.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--build-arg NAME=VALUE` sets a build-time argument.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --build-arg GO_VERSION=1.23 -t local/api:go123 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 269. docker build — Build named target stage

**Purpose.** Build a selected stage in a multi-stage Dockerfile.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--target NAME` selects a stage declared with `AS NAME`.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --target test -t local/api:test .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 270. docker build — Plain progress output

**Purpose.** Make build steps easier to read in CI logs.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--progress=plain` requests plain progress where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --progress=plain -t local/api:ci .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 271. docker build — Quiet build

**Purpose.** Print minimal output and image ID on success.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `-q, --quiet` suppresses build output.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build -q -t local/api:quiet .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 272. docker build — Add image label

**Purpose.** Attach metadata to the built image.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--label KEY=VALUE` adds image metadata.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --label org.opencontainers.image.source=https://example.com/repo -t \
local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 273. docker build — Build for target platform

**Purpose.** Select an OS/architecture output platform.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--platform OS/ARCH[/VARIANT]` selects platform.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build --platform linux/arm64 -t local/api:arm64 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 274. docker build — Build from remote Git URL

**Purpose.** Use a remote repository as build context.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** URL is the context; pin commit/URL for reproducibility.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker build https://github.com/docker/docs.git
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 275. docker build — Build with stdin Dockerfile

**Purpose.** Provide a small Dockerfile through stdin.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `-` marks stdin build input in supported mode; `printf` is host shell.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
printf 'FROM alpine:3.20\nCMD ["echo","hello"]\n' | docker build -t local/hello -
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 276. docker build — Set memory for build step

**Purpose.** Apply a resource limit to supported BuildKit build steps.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--resource memory=2g` configures build-step memory where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --resource memory=2g -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 277. docker build — Build with explicit network mode

**Purpose.** Select network mode for RUN instructions where supported.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--network` selects networking mode for build steps.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --network=host -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 278. docker build — Write image ID to file

**Purpose.** Save build output identity for automation.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--iidfile FILE` writes image ID to a file.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --iidfile ./image-id.txt -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 279. docker build — Write build metadata to file

**Purpose.** Save build metadata for CI artifacts.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--metadata-file FILE` writes build result metadata.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --metadata-file ./build-metadata.json -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 280. docker build — Build with provenance attestation

**Purpose.** Request provenance metadata for the build.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--provenance` is shorthand for provenance attestation where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --provenance -t registry.example.com/team/api:1.0 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 281. docker build — Build with SBOM attestation

**Purpose.** Request an SBOM attestation for the image.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--sbom` requests an SBOM attestation where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --sbom -t registry.example.com/team/api:1.0 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 282. docker build — Load result into local image store

**Purpose.** Make a Buildx result available to local `docker run`.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--load` exports result to local Docker image store; commonly single-platform.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --load -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 283. docker build — Push result to registry

**Purpose.** Build and publish directly to a registry.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--push` exports result to registry; `-t` sets image reference.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --push -t registry.example.com/team/api:1.0 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 284. docker build — Use secret mount during build

**Purpose.** Expose a secret to a BuildKit RUN step without ordinary ARG/ENV persistence.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--secret id=ID,src=FILE` supplies a BuildKit secret; Dockerfile must mount it.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --secret id=npmrc,src=$HOME/.npmrc -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 285. docker build — Forward SSH agent during build

**Purpose.** Allow a BuildKit step to authenticate to a private source.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--ssh default` forwards configured SSH agent/socket.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --ssh default -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 286. docker build — Import registry cache

**Purpose.** Reuse cache from a registry backend.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--cache-from` imports cache from a specified backend/reference.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --cache-from type=registry,ref=registry.example.com/team/api:cache \
-t local/api:ci .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 287. docker build — Export registry cache

**Purpose.** Publish cache for future CI builds.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--cache-to` exports cache; `mode=max` retains more intermediate-stage cache.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --cache-to \
type=registry,ref=registry.example.com/team/api:cache,mode=max -t local/api:ci .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 288. docker build — Check Dockerfile without full build

**Purpose.** Run supported BuildKit Dockerfile checks.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--check` requests check mode where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --check .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 289. docker build — Build only a named stage

**Purpose.** Use Buildx to target a build stage.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--target` selects a named Dockerfile stage.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --target runtime -t local/api:runtime .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 290. docker build — Build from named additional context

**Purpose.** Pass an additional context for COPY --from usage.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--build-context NAME=PATH` adds a named build context.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --build-context docs=./docs -t local/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 291. docker build — Set build output directory

**Purpose.** Export build artifacts to a local directory.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** `--output type=local,dest=DIR` exports filesystem artifacts.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build --output type=local,dest=./out .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 292. docker build — Set multiple output destinations

**Purpose.** Export one build result to more than one output where supported.

**Syntax.** `docker build [OPTIONS] PATH | URL | -`

**Options used.** Repeated `-o, --output` configures multiple outputs where supported.

**Arguments.** Build context is local path, URL, or stdin marker.

**Example.**

```
docker buildx build -o type=local,dest=./out -o \
type=registry,ref=registry.example.com/team/api:dev .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 293. docker buildx — List builders

**Purpose.** Show available builder instances and selected builder.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** No option required.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 294. docker buildx — Inspect current builder

**Purpose.** Read driver, endpoints, and supported platforms.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** No option required.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx inspect
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 295. docker buildx — Bootstrap builder

**Purpose.** Start/initialize the selected builder before inspecting.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--bootstrap` starts/initializes the builder.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx inspect --bootstrap
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 296. docker buildx — Create named builder

**Purpose.** Create an isolated BuildKit builder.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--name` names builder; `--driver` selects supported driver.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx create --name ci-builder --driver docker-container
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 297. docker buildx — Create and select builder

**Purpose.** Create, select, and initialize a builder.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--use` selects builder; `--bootstrap` starts it.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx create --name ci-builder --use --bootstrap
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 298. docker buildx — Select builder

**Purpose.** Make an existing builder current.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** Argument is existing builder name.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx use ci-builder
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 299. docker buildx — Remove builder

**Purpose.** Delete a builder no longer needed.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** Argument is builder name.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx rm ci-builder
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 300. docker buildx — Show Buildx version

**Purpose.** Check plugin version.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** No positional arguments.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx version
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 301. docker buildx — Inspect registry image platforms

**Purpose.** Read manifest/platform metadata for a remote image.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** Argument is registry image reference.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx imagetools inspect nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 302. docker buildx — Build multi-platform image

**Purpose.** Build for amd64 and arm64 and push a manifest list.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--platform` accepts comma-separated platforms; `--push` publishes result.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx build --platform linux/amd64,linux/arm64 --push -t \
registry.example.com/team/api:1.0 .
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 303. docker buildx — Prune build cache

**Purpose.** Remove reclaimable cache from selected builder.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--all`, `--filter`, and `--keep-storage` can adjust scope where supported.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 304. docker buildx — Inspect build cache usage

**Purpose.** Review cache records and their sizes.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** No option required.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx du
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 305. docker buildx — Print Bake configuration

**Purpose.** Resolve and print Bake targets without building.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--print` prints resolved build definition.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx bake --print
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 306. docker buildx — List Bake targets

**Purpose.** List targets declared in Bake definition.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--list=targets` requests target list where supported.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx bake --list=targets
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 307. docker buildx — Run Bake targets

**Purpose.** Execute a multi-target build graph.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--file` selects a Bake definition.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx bake --file docker-bake.hcl
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 308. docker buildx — Create builder from context

**Purpose.** Use an existing Docker context as a builder endpoint.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** First argument is context/endpoint; `--name` names builder.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx create staging --name remote-builder
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 309. docker buildx — Set builder platforms

**Purpose.** Configure a builder node's target platform where supported.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--platform` declares supported platforms for the node.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx create --name arm-builder --platform linux/arm64
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 310. docker buildx — Append builder node

**Purpose.** Add another context/endpoint to an existing multi-node builder.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--append` adds a node to the named builder.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx create staging-arm --name multi-builder --append
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 311. docker buildx — Stop a builder

**Purpose.** Stop a builder without deleting its definition.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** Optional builder name selects target.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx stop ci-builder
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 312. docker buildx — Inspect raw registry manifest

**Purpose.** Fetch raw manifest JSON for troubleshooting.

**Syntax.** `docker buildx COMMAND [OPTIONS]`

**Options used.** `--raw` requests raw manifest output where supported.

**Arguments.** COMMAND is a Buildx subcommand; Buildx extends builds with BuildKit.

**Example.**

```
docker buildx imagetools inspect --raw nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

# 06 · Networking

43 reference entries. Follow the examples in order and substitute environment-specific values.

### 313. docker network — List networks

**Purpose.** Show network names, drivers, and IDs.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `-q` prints IDs; `--filter` narrows results.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 314. docker network — Create a user-defined bridge

**Purpose.** Create isolated local network with container-name DNS.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Argument is new network name; local Engine defaults commonly use bridge driver.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 315. docker network — Create bridge with subnet

**Purpose.** Choose address range for a bridge network.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--driver` selects driver; `--subnet` specifies CIDR.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --driver bridge --subnet 172.28.0.0/16 app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 316. docker network — Set gateway

**Purpose.** Choose gateway within the network subnet.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--gateway IP` must belong to the subnet.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --subnet 172.28.0.0/16 --gateway 172.28.0.1 app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 317. docker network — Create internal network

**Purpose.** Restrict external connectivity for this network according to driver behavior.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--internal` creates an internal network.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --internal backend-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 318. docker network — Create labeled network

**Purpose.** Attach inventory metadata.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--label KEY=VALUE` adds metadata.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --label team=platform app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 319. docker network — Inspect network

**Purpose.** Read driver, IPAM, options, and endpoints.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Argument is network name/ID.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network inspect app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 320. docker network — Extract attached containers

**Purpose.** Print endpoint metadata for a network.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `-f` selects the `Containers` field.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network inspect -f '{{json .Containers}}' app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 321. docker network — Connect a container

**Purpose.** Attach an existing container to another network.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Arguments are network then container.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network connect app-net api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 322. docker network — Connect with DNS alias

**Purpose.** Add a network-scoped alias.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--alias NAME` adds an endpoint alias.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network connect --alias api.internal app-net api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 323. docker network — Connect with static IP

**Purpose.** Request a fixed IPv4 address on a network with IPAM.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--ip` selects endpoint IPv4 address.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network connect --ip 172.28.0.10 app-net api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 324. docker network — Disconnect a container

**Purpose.** Remove its endpoint from a network.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Arguments are network then container.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network disconnect app-net api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 325. docker network — Force disconnect endpoint

**Purpose.** Force endpoint removal where supported.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `-f, --force` forces disconnection where supported.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network disconnect --force app-net api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 326. docker network — Remove a network

**Purpose.** Delete a network without active endpoints.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Argument is network name/ID.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network rm app-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 327. docker network — Prune unused networks

**Purpose.** Remove unused network definitions.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `-f` skips confirmation; filters may narrow scope.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 328. docker network — Prune networks by age

**Purpose.** Limit network cleanup by supported age filter.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--filter until=24h` applies a cutoff.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network prune --filter 'until=24h'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 329. docker network — Filter networks by driver

**Purpose.** List only bridge networks.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--filter driver=DRIVER` selects driver.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network ls --filter driver=bridge
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 330. docker network — Filter networks by label

**Purpose.** Find networks with an environment label.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--filter label=KEY=VALUE` matches labels.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network ls --filter label=environment=dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 331. docker network — Create IPv6 network

**Purpose.** Enable IPv6 on a network when daemon configuration supports it.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--ipv6` enables IPv6 for the network where supported.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --ipv6 ipv6-net
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 332. docker network — Create attachable overlay

**Purpose.** Create an overlay network that standalone containers can join.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--driver overlay` selects overlay; `--attachable` permits standalone containers.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --driver overlay --attachable app-overlay
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 333. docker network — Create encrypted overlay

**Purpose.** Request encrypted overlay traffic where supported.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--opt encrypted` requests overlay encryption.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network create --driver overlay --opt encrypted secure-overlay
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 334. docker network — List network IDs only

**Purpose.** Output only IDs for scripts.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `-q, --quiet` prints IDs.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network ls -q
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 335. docker network — Format network inventory

**Purpose.** Print network name and driver fields.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** `--format TEMPLATE` uses Go templates.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network ls --format '{{.Name}} {{.Driver}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 336. docker network — Inspect multiple networks

**Purpose.** Compare multiple network configurations.

**Syntax.** `docker network COMMAND [OPTIONS]`

**Options used.** Each argument is a network name/ID.

**Arguments.** COMMAND is `ls`, `create`, `inspect`, `connect`, `disconnect`, `prune`, or `rm`.

**Example.**

```
docker network inspect bridge host none
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 337. docker volume — Create named volume

**Purpose.** Create Docker-managed persistent storage.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** Argument is volume name; `--driver` selects driver.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume create app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 338. docker volume — List volumes

**Purpose.** Inventory Docker-managed volumes.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `-q` prints names; `--filter` narrows results.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 339. docker volume — Inspect volume

**Purpose.** Read driver, mountpoint, labels, and options.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** Argument is volume name.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume inspect app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 340. docker volume — Extract mountpoint

**Purpose.** Print mountpoint where the Engine exposes it.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `-f` selects mountpoint field.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume inspect -f '{{.Mountpoint}}' app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 341. docker volume — Create volume with label

**Purpose.** Attach metadata for inventory/backup automation.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--label KEY=VALUE` adds metadata.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume create --label backup=daily app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 342. docker volume — Create using local driver

**Purpose.** Choose the local volume driver explicitly.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--driver NAME` selects driver.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume create --driver local app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 343. docker volume — Mount volume read-only

**Purpose.** Give a container read-only access to shared data.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `readonly` marks the mount read-only.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker run --rm --mount type=volume,src=app-data,dst=/data,readonly alpine:3.20 ls /data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 344. docker volume — Bind mount source directory

**Purpose.** Expose a host directory inside a container.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `type=bind`, `src`, and `dst` define host/container paths.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker run --rm --mount type=bind,src="$PWD",dst=/workspace alpine:3.20 ls /workspace
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 345. docker volume — Bind mount read-only

**Purpose.** Share config without allowing container writes through the mount.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `readonly` makes the mount read-only.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker run --rm --mount type=bind,src="$PWD/config",dst=/config,readonly alpine:3.20 \
ls /config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 346. docker volume — Use tmpfs with mount options

**Purpose.** Create ephemeral storage for temporary files.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--tmpfs PATH:OPTIONS` sets temporary mount options.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker run --rm --tmpfs /tmp:rw,noexec,nosuid,size=64m alpine:3.20 df -h /tmp
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 347. docker volume — Use tmpfs mount syntax

**Purpose.** Specify tmpfs through key-value mount syntax.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `type=tmpfs`, `dst`, and `tmpfs-size` define the mount where supported.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker run --rm --mount type=tmpfs,dst=/run,tmpfs-size=16m alpine:3.20 df -h /run
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 348. docker volume — Remove a volume

**Purpose.** Delete a specific unused volume.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** Volume must not be in use.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume rm app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 349. docker volume — Prune unused volumes

**Purpose.** Remove eligible unused volumes.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `-f` skips confirmation; `-a` broadens scope where supported.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 350. docker volume — Prune all unused volumes

**Purpose.** Expand cleanup to all unused volumes where supported.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--all` broadens eligible set on versions that support it.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume prune --all
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 351. docker volume — Prune with force

**Purpose.** Run volume cleanup without an interactive prompt.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--force` skips confirmation.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume prune --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 352. docker volume — Create driver-specific bind volume

**Purpose.** Configure local driver with a host path using driver options.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--opt KEY=VALUE` is repeatable; meaning depends on volume driver.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume create --driver local --opt type=none --opt o=bind --opt \
device=/srv/app-data app-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 353. docker volume — Create multiple labels

**Purpose.** Add environment and owner metadata.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** Each `--label` adds a metadata key/value.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume create --label environment=staging --label owner=platform staging-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 354. docker volume — Filter volumes by driver

**Purpose.** List volumes created by local driver.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--filter driver=DRIVER` narrows results.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume ls --filter driver=local
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 355. docker volume — Filter volumes by name

**Purpose.** Find volumes matching a name filter.

**Syntax.** `docker volume COMMAND [OPTIONS]`

**Options used.** `--filter name=VALUE` filters names.

**Arguments.** COMMAND is create, ls, inspect, rm, or prune.

**Example.**

```
docker volume ls --filter name=app
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

# 08 · Docker Compose

70 reference entries. Follow the examples in order and substitute environment-specific values.

## 356. docker compose — Show Compose help

**Purpose.** List Compose subcommands and global options.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--help` prints installed Compose plugin usage.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 357. docker compose — Print Compose version

**Purpose.** Verify Compose V2 plugin version.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No positional arguments.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose version
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 358. docker compose — Use alternate Compose file

**Purpose.** Select a non-default Compose file.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-f, --file FILE` selects a file; repeated files merge in order.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose -f compose.prod.yaml config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 359. docker compose — Merge base and override files

**Purpose.** Apply environment-specific overrides.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Repeated `-f` files merge in supplied order.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose -f compose.yaml -f compose.prod.yaml config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 360. docker compose — Set project name

**Purpose.** Isolate resource names for an environment.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-p, --project-name NAME` overrides project name.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose -p demo up -d
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 361. docker compose — Use env file for interpolation

**Purpose.** Load variables from a selected file.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--env-file FILE` sets interpolation source.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose --env-file .env.staging config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 362. docker compose — Enable profile

**Purpose.** Include services assigned to a named profile.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--profile NAME` enables a profile.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose --profile debug up -d
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 363. docker compose — Validate and render config

**Purpose.** Resolve interpolation and print normalized model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No option required; `--quiet` validates without printing where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 364. docker compose — List service names

**Purpose.** Print service names from the resolved model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--services` lists service names.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config --services
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 365. docker compose — List image references

**Purpose.** Extract image names from Compose model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--images` lists image references where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config --images
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 366. docker compose — List declared profiles

**Purpose.** List profile names from the Compose model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--profiles` requests profile list where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config --profiles
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 367. docker compose — Render JSON config

**Purpose.** Print resolved Compose model as JSON where supported.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--format json` selects JSON output on supported versions.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config --format json
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 368. docker compose — Validate quietly

**Purpose.** Check configuration without printing full model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--quiet` validates silently where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose config --quiet
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 369. docker compose — List projects

**Purpose.** Show Compose projects known to the current context.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No option required.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 370. docker compose — List stopped projects too

**Purpose.** Include stopped projects where supported.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--all` includes stopped projects on supported versions.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose ls --all
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 371. docker compose — List project containers

**Purpose.** Show service container state.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No option required.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose ps
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 372. docker compose — Include stopped service containers

**Purpose.** Show all containers including stopped ones.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-a` includes stopped containers.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose ps -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 373. docker compose — Start services detached

**Purpose.** Create/recreate services and leave them running.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-d, --detach` backgrounds services.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 374. docker compose — Build before startup

**Purpose.** Build images then start services.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--build` builds before startup; `-d` detaches.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up --build -d
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 375. docker compose — Force recreation

**Purpose.** Replace containers even if Compose sees no changes.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--force-recreate` recreates service containers.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d --force-recreate
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 376. docker compose — Skip dependency startup

**Purpose.** Start only selected service without starting dependencies.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--no-deps` skips dependency startup; `api` is a service name.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d --no-deps api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 377. docker compose — Start selected services

**Purpose.** Operate on a subset of the project.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service names select targets.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d api worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 378. docker compose — Wait for running/healthy

**Purpose.** Wait for services to start or become healthy where supported.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--wait` waits for running/healthy state and implies detached mode on supported versions.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d --wait
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 379. docker compose — Wait with timeout

**Purpose.** Limit readiness wait duration.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--wait-timeout SECONDS` bounds wait where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d --wait --wait-timeout 120
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 380. docker compose — Abort when a container exits

**Purpose.** Stop the project when one container exits, useful for test jobs.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--abort-on-container-exit` stops containers when one exits.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up --abort-on-container-exit
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 381. docker compose — Build all services

**Purpose.** Build services with build definitions.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--no-cache` disables cache; `--pull` attempts to refresh bases.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose build
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 382. docker compose — Build one service

**Purpose.** Build a specific service image.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name selects build target.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose build api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 383. docker compose — Build without cache

**Purpose.** Re-run build steps without cache reuse.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--no-cache` disables cache.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose build --no-cache api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 384. docker compose — Pull base images while building

**Purpose.** Attempt to refresh referenced base images.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--pull` attempts to pull newer base images.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose build --pull api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 385. docker compose — Use plain build progress

**Purpose.** Improve CI log readability.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--progress plain` requests plain progress output where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose build --progress plain api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 386. docker compose — Pull all service images

**Purpose.** Download images declared by services.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No option required.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose pull
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 387. docker compose — Pull one service image

**Purpose.** Fetch only the image for a selected service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name selects target.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose pull api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 388. docker compose — Start existing containers

**Purpose.** Start previously created containers without recreating.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Optional service names narrow scope.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose start
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 389. docker compose — Stop services but retain containers

**Purpose.** Stop project containers without removing them.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-t, --timeout SECONDS` sets grace period.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose stop
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 390. docker compose — Restart a service

**Purpose.** Restart an existing service container.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name narrows scope; restart does not necessarily apply config changes.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose restart api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 391. docker compose — Tear down project

**Purpose.** Remove project containers and networks, retaining volumes by default.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--remove-orphans` removes orphan containers; `-v` deletes volumes.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose down
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 392. docker compose — Remove orphan containers

**Purpose.** Remove containers for services absent from current model.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--remove-orphans` removes orphan project containers.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose down --remove-orphans
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 393. docker compose — Remove project volumes

**Purpose.** Remove project-associated named and anonymous volumes.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-v, --volumes` removes eligible volumes.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose down --volumes
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 394. docker compose — Remove local images on teardown

**Purpose.** Remove local-tagged images during project teardown.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--rmi local` removes local images without custom tags.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose down --rmi local
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 395. docker compose — Follow service logs

**Purpose.** Stream logs from a selected service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-f, --follow` streams output.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose logs -f api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 396. docker compose — Show last 100 log lines

**Purpose.** Limit initial log output.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--tail N` limits history per service.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose logs --tail 100
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 397. docker compose — Add timestamps to logs

**Purpose.** Correlate output with incident timeline.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--timestamps` adds timestamps.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose logs --timestamps api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 398. docker compose — Show logs since duration

**Purpose.** Limit log output to recent time window.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--since DURATION` sets lower time bound.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose logs --since 15m api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 399. docker compose — Execute interactive command

**Purpose.** Run command inside a running service container.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `exec` targets service container; `-T` disables TTY in scripts.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose exec api sh
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 400. docker compose — Execute without TTY

**Purpose.** Make output suitable for redirection/CI.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-T` disables pseudo-TTY allocation.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose exec -T api printenv
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 401. docker compose — Execute as non-root user

**Purpose.** Run a command as selected UID/GID.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--user UID[:GID]` overrides exec user.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose exec --user 10001:10001 api id
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 402. docker compose — Execute in working directory

**Purpose.** Run a command from app directory.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--workdir DIR` sets process working directory.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose exec --workdir /app api pwd
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 403. docker compose — Run one-off service command

**Purpose.** Create a one-off container using service config.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `run SERVICE COMMAND...` creates one-off container; `--rm` removes it after exit.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose run --rm api python -m pytest
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 404. docker compose — Run one-off with service ports

**Purpose.** Publish declared service ports for a one-off container.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--service-ports` publishes configured ports.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose run --service-ports api python -m http.server 8000
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 405. docker compose — Override one-off entrypoint

**Purpose.** Launch a shell instead of configured entrypoint.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--entrypoint` overrides service entrypoint for this run.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose run --rm --entrypoint sh api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 406. docker compose — Scale a service

**Purpose.** Request three replicas of a service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--scale SERVICE=NUM` sets replica count.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up -d --scale worker=3
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 407. docker compose — Copy file from service

**Purpose.** Copy an artifact from service container to host.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Paths use `SERVICE:PATH` for container side.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose cp api:/tmp/report.txt ./report.txt
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 408. docker compose — Copy directory to service

**Purpose.** Copy host directory into service container.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Host path is source; service path is destination.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose cp ./fixtures api:/tmp/fixtures
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 409. docker compose — Copy with archive semantics

**Purpose.** Request archive ownership/permission behavior.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-a` requests archive behavior where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose cp -a ./artifact api:/tmp/artifact
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 410. docker compose — List service images

**Purpose.** Show images associated with project containers.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-q` prints IDs where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose images
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 411. docker compose — Show service processes

**Purpose.** List processes for Compose services.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Optional service name narrows scope.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose top
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 412. docker compose — Show one service process list

**Purpose.** Inspect processes for one service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name selects target.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose top api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 413. docker compose — Monitor service stats

**Purpose.** Watch CPU/memory usage for project containers.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No option required.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose stats
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 414. docker compose — Take stats snapshot

**Purpose.** Print one resource sample.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--no-stream` requests a single sample where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose stats --no-stream
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 415. docker compose — Find published service port

**Purpose.** Print host mapping for a service port.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Arguments are service and private port.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose port web 80
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 416. docker compose — Kill all Compose containers

**Purpose.** Send SIGKILL to project service containers.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Default signal is SIGKILL.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose kill
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 417. docker compose — Send signal to one service

**Purpose.** Send SIGTERM to a selected service container.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `-s, --signal SIGNAL` selects signal.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose kill -s SIGTERM api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 418. docker compose — Pause all services

**Purpose.** Freeze project container processes.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No service argument targets project.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose pause
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 419. docker compose — Pause one service

**Purpose.** Freeze a selected service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name narrows scope.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose pause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 420. docker compose — Resume all services

**Purpose.** Resume paused project containers.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** No service argument targets project.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose unpause
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 421. docker compose — Resume one service

**Purpose.** Resume a selected service.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Service name narrows scope.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose unpause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 422. docker compose — Watch source changes

**Purpose.** Apply configured watch rules during development.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** Requires supported Compose version and watch/develop config.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose watch api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 423. docker compose — Start services with watch

**Purpose.** Start project and activate configured watch workflow.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--watch` enables watch workflow on supported versions.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose up --watch
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 424. docker compose — Use dry-run preview

**Purpose.** Preview planned actions without applying them where supported.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--dry-run` is a global option on supported Compose versions.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose --dry-run up -d
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 425. docker compose — Set parallelism

**Purpose.** Limit parallel execution for supported Compose operations.

**Syntax.** `docker compose [OPTIONS] COMMAND [ARGS...]`

**Options used.** `--parallel N` controls max parallelism where supported.

**Arguments.** Global options include `-f`, `-p`, `--profile`, `--env-file`, `--project-directory`, and `--dry-run`.

**Example.**

```
docker compose --parallel 4 pull
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

# 09 · Contexts & remote Engines

12 reference entries. Follow the examples in order and substitute environment-specific values.

### 426. docker context — List contexts

**Purpose.** Show named Engine endpoints and current selection.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `*` marks current context; `--format` customizes output.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 427. docker context — Show current context

**Purpose.** Print the default context name.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** No positional arguments.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context show
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 428. docker context — Inspect context endpoint

**Purpose.** Read endpoint and TLS metadata.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** Argument is context name.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context inspect staging
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 429. docker context — Create SSH context

**Purpose.** Define a remote Engine endpoint over SSH.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `create NAME` names context; `--docker host=ENDPOINT` sets endpoint.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context create staging --docker host=ssh://deploy@example.com
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 430. docker context — Select context

**Purpose.** Change default target Engine for future commands.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** Argument is an existing context name.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context use staging
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 431. docker context — Use context for one command

**Purpose.** Target a remote Engine without changing default.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `--context NAME` is a global per-invocation option.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker --context staging ps -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 432. docker context — Export context

**Purpose.** Package context metadata for transfer/backup.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `-o FILE` selects export output where supported.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context export staging -o staging.dockercontext
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 433. docker context — Import context

**Purpose.** Restore context metadata under a new name.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** Arguments are new context name and export file.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context import staging-copy staging.dockercontext
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 434. docker context — Remove context definition

**Purpose.** Delete local context metadata.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** Argument is context name; current context generally cannot be removed while selected.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context rm staging-copy
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 435. docker context — Update context description

**Purpose.** Change descriptive metadata where supported.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `--description TEXT` updates description where supported.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker context update --description 'Staging Engine' staging
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 436. docker context — Inspect remote daemon

**Purpose.** Query info through a named context.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `--context` selects remote Engine for invocation.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker --context staging info
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 437. docker context — Use SSH endpoint directly

**Purpose.** Connect through SSH transport for one command.

**Syntax.** `docker context COMMAND [OPTIONS]`

**Options used.** `-H, --host` selects endpoint; SSH transport requires remote Docker access.

**Arguments.** Commands include ls, create, use, inspect, show, export, import, rm, and update.

**Example.**

```
docker -H ssh://deploy@example.com ps
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

# 10 · Swarm, services, nodes & stacks

87 reference entries. Follow the examples in order and substitute environment-specific values.

## 438. docker swarm — Initialize a swarm

**Purpose.** Make current Engine a single-node manager.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--advertise-addr` selects manager address.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 439. docker swarm — Initialize with advertised address

**Purpose.** Set address other nodes use to reach manager.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--advertise-addr IP|INTERFACE[:PORT]` chooses advertised endpoint.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init --advertise-addr 192.0.2.10
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 440. docker swarm — Initialize with autolock

**Purpose.** Require unlock key after manager restart.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--autolock` enables manager autolocking.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init --autolock
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 441. docker swarm — Leave swarm

**Purpose.** Remove current node from membership.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--force` can bypass manager leave safeguard.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm leave
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 442. docker swarm — Force manager to leave

**Purpose.** Force a manager node to leave.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--force` bypasses normal manager-leave safeguard.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm leave --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 443. docker swarm — Show worker join token

**Purpose.** Retrieve token for worker nodes.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `worker` selects role; `-q` prints token only.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm join-token worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 444. docker swarm — Show manager join token

**Purpose.** Retrieve token for manager nodes.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `manager` selects role.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm join-token manager
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 445. docker swarm — Rotate worker token

**Purpose.** Invalidate old worker join token and issue a new one.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--rotate` rotates token.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm join-token --rotate worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 446. docker swarm — Display swarm CA

**Purpose.** Inspect current root CA certificate metadata.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** No option required to display current CA.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm ca
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 447. docker swarm — Rotate swarm CA

**Purpose.** Rotate root CA using documented workflow.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--rotate` requests CA rotation.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm ca --rotate
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 448. docker swarm — Unlock a manager

**Purpose.** Unlock manager after autolock restart.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** Requires stored unlock key.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm unlock
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 449. docker swarm — Display unlock key

**Purpose.** Retrieve manager unlock key.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** No option required; `--rotate` replaces key.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm unlock-key
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 450. docker swarm — Rotate unlock key

**Purpose.** Generate a new unlock key.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--rotate` replaces active unlock key.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm unlock-key --rotate
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 451. docker swarm — Set task history limit

**Purpose.** Control retained completed task records.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--task-history-limit N` sets task history retention.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm update --task-history-limit 10
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 452. docker swarm — Tune dispatcher heartbeat

**Purpose.** Set manager-worker dispatcher heartbeat period.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--dispatcher-heartbeat DURATION` changes heartbeat interval.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm update --dispatcher-heartbeat 5s
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 453. docker swarm — Set default address pool

**Purpose.** Configure overlay network address pool at initialization.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--default-addr-pool CIDR` and mask length set generated subnet ranges.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init --default-addr-pool 10.20.0.0/16 --default-addr-pool-mask-length 24
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 454. docker swarm — Set data-path address

**Purpose.** Select overlay data-plane address where supported.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--data-path-addr` selects address/interface on supported API versions.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init --data-path-addr 192.0.2.11
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 455. docker swarm — Set manager listen address

**Purpose.** Choose address/port for manager control traffic.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--listen-addr IP:PORT` selects listen endpoint.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm init --listen-addr 192.0.2.10:2377
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 456. docker swarm — Join as worker

**Purpose.** Join an existing swarm as worker.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** `--token` supplies worker token; final argument is manager endpoint.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm join --token WORKER_TOKEN 192.0.2.10:2377
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 457. docker swarm — Join as manager

**Purpose.** Join with manager token.

**Syntax.** `docker swarm COMMAND [OPTIONS]`

**Options used.** Manager token grants control-plane role.

**Arguments.** Swarm commands manage a cluster; many require manager role.

**Example.**

```
docker swarm join --token MANAGER_TOKEN 192.0.2.10:2377
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 458. docker service — List services

**Purpose.** Show desired replicas and current service state.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--filter` narrows by supported metadata.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 459. docker service — Create replicated web service

**Purpose.** Deploy three replicas and publish port.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--name`, `--replicas`, and `-p` set service name, replica count, and published port.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name web --replicas 3 -p 8080:80 nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 460. docker service — Create global agent

**Purpose.** Run one task on each eligible node.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--mode global` requests one task per eligible node.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name node-agent --mode global alpine:3.20 sleep 3600
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 461. docker service — Set service environment

**Purpose.** Configure task environment variable.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--env KEY=VALUE` sets environment for tasks.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --env APP_MODE=prod nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 462. docker service — Mount volume in service

**Purpose.** Attach persistent storage to service tasks.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--mount` specifies mount type/source/target.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name db --mount \
type=volume,source=dbdata,target=/var/lib/postgresql/data postgres:16
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 463. docker service — Inspect service JSON

**Purpose.** Read full service specification.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--pretty` formats summary; `--format` selects fields.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service inspect web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 464. docker service — Inspect service pretty output

**Purpose.** Get a readable summary.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--pretty` selects human-readable summary.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service inspect --pretty web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 465. docker service — List service tasks

**Purpose.** Inspect task placement and current state.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--no-trunc` shows full task IDs/errors.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service ps web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 466. docker service — Show full task errors

**Purpose.** Avoid truncation when diagnosing scheduling/startup.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--no-trunc` prints full IDs and error details.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service ps --no-trunc web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 467. docker service — Scale service to five replicas

**Purpose.** Change desired replica count.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `SERVICE=REPLICAS` sets desired replica count.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service scale web=5
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 468. docker service — Update service image

**Purpose.** Roll out a new image reference.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--image IMAGE` updates task image.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --image nginx:1.27-alpine web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 469. docker service — Update replica count

**Purpose.** Set desired replicas through update.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--replicas N` changes replica count.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --replicas 2 web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 470. docker service — Add environment variable

**Purpose.** Add/update a service environment entry.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--env-add KEY=VALUE` updates environment.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --env-add LOG_LEVEL=info web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 471. docker service — Remove environment variable

**Purpose.** Remove an environment entry.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--env-rm KEY` removes named variable.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --env-rm LOG_LEVEL web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 472. docker service — Remove published port

**Purpose.** Remove a matching published port specification.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--publish-rm` removes a port specification; match syntax to installed version.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --publish-rm 8080:80 web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 473. docker service — Force rolling update

**Purpose.** Replace tasks even when service spec appears unchanged.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--force` forces task replacement according to update policy.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --force web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 474. docker service — Rollback service

**Purpose.** Return to prior service spec when available.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** Uses stored previous service spec if rollback is available.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service rollback web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 475. docker service — Remove service

**Purpose.** Delete service and stop its tasks.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** Argument is service name/ID.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service rm web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 476. docker service — Follow service logs

**Purpose.** Stream aggregated task logs where driver supports it.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--follow` streams; logging driver support varies.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service logs --follow web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 477. docker service — Show recent service logs

**Purpose.** Limit output to last 100 lines.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--tail N` limits initial history.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service logs --tail 100 web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 478. docker service — Constrain placement by node label

**Purpose.** Schedule tasks only on nodes matching a label constraint.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--constraint` adds placement rule; nodes need matching label.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --constraint 'node.labels.region==east' nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 479. docker service — Set CPU/memory limits

**Purpose.** Set per-task resource ceilings.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--limit-cpu` and `--limit-memory` cap task resources.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --limit-cpu 0.50 --limit-memory 256M nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 480. docker service — Set restart condition

**Purpose.** Choose when Swarm restarts failed tasks.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--restart-condition` accepts documented values such as `none`, `on-failure`, `any`.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name worker --restart-condition on-failure alpine:3.20 sleep 3600
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 481. docker service — Set DNS round-robin endpoint mode

**Purpose.** Use DNSRR service discovery where supported.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--endpoint-mode dnsrr` selects DNS round-robin mode.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --endpoint-mode dnsrr nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 482. docker service — Set update parallelism

**Purpose.** Control tasks updated concurrently.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--update-parallelism N` sets concurrent update count.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --replicas 4 --update-parallelism 1 nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 483. docker service — Set update delay

**Purpose.** Insert a delay between update batches.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--update-delay DURATION` sets delay.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --replicas 4 --update-delay 10s nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 484. docker service — Configure rollback-on-failure

**Purpose.** Choose rollback as failure action where supported.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--update-failure-action` sets documented update failure action.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service create --name api --update-failure-action rollback nginx:alpine
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 485. docker service — Add published port

**Purpose.** Add a port to an existing service.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--publish-add` accepts a port specification.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --publish-add published=8081,target=80,protocol=tcp,mode=ingress web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 486. docker service — Add secret to service

**Purpose.** Grant service access to a Swarm secret.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--secret-add` adds a secret reference.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --secret-add db_password web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 487. docker service — Remove secret from service

**Purpose.** Remove a secret reference.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--secret-rm NAME` removes secret reference.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --secret-rm old_api_key web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 488. docker service — Add config to service

**Purpose.** Mount a Swarm config into task containers.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--config-add` adds config mount.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --config-add source=nginx_conf,target=/etc/nginx/nginx.conf web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 489. docker service — Add placement preference

**Purpose.** Prefer spreading tasks across zones.

**Syntax.** `docker service COMMAND [OPTIONS]`

**Options used.** `--placement-pref-add` adds a scheduling preference.

**Arguments.** Service commands generally require a Swarm manager.

**Example.**

```
docker service update --placement-pref-add 'spread=node.labels.zone' web
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 490. docker node — List nodes

**Purpose.** Show node IDs, hostnames, availability, and role.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `-q` prints IDs; `--filter` narrows results.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 491. docker node — Inspect node

**Purpose.** Read node metadata and status.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--pretty` formats a summary.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node inspect worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 492. docker node — Pretty node summary

**Purpose.** Read a concise human-readable node report.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--pretty` requests summary output.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node inspect --pretty worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 493. docker node — List node tasks

**Purpose.** See tasks placed on a node.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--no-trunc` shows full IDs/errors.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node ps worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 494. docker node — Drain node

**Purpose.** Prevent new tasks and reschedule eligible service tasks.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--availability drain` drains the node.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node update --availability drain worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 495. docker node — Reactivate node

**Purpose.** Allow new tasks to be scheduled on a node.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--availability active` makes node schedulable.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node update --availability active worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 496. docker node — Pause scheduling

**Purpose.** Prevent new task assignment without draining current tasks.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--availability pause` prevents new task placement.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node update --availability pause worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 497. docker node — Add node label

**Purpose.** Add a placement metadata label.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--label-add KEY=VALUE` adds node label.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node update --label-add region=east worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 498. docker node — Remove node label

**Purpose.** Remove a scheduling label.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--label-rm KEY` removes label.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node update --label-rm region worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 499. docker node — Promote worker to manager

**Purpose.** Promote a worker node.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** Argument is node name/ID.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node promote worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 500. docker node — Demote manager to worker

**Purpose.** Change manager role to worker.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** Argument is node name/ID.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node demote manager-2
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 501. docker node — Remove node record

**Purpose.** Remove a node from cluster inventory.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** Node should have left/decommissioned first.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node rm worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 502. docker node — Force-remove down node

**Purpose.** Remove a down node record when normal safeguards block removal.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--force` permits removal of a down node in supported cases.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node rm --force worker-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 503. docker node — Filter node list by role

**Purpose.** List manager nodes using supported filter.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--filter role=manager` narrows node list.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node ls --filter role=manager
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 504. docker node — Filter node list by availability

**Purpose.** Find drained nodes.

**Syntax.** `docker node COMMAND [OPTIONS]`

**Options used.** `--filter availability=drain` filters availability.

**Arguments.** Node management normally requires manager role.

**Example.**

```
docker node ls --filter availability=drain
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 505. docker stack — List stacks

**Purpose.** Show deployed stack names and service counts.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** No option required.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 506. docker stack — Deploy stack

**Purpose.** Deploy services from stack file.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** `-c, --compose-file FILE` selects file; final argument is stack name.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack deploy -c stack.yml demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 507. docker stack — Deploy with override file

**Purpose.** Merge base and production stack files.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** Repeated `-c` files merge in order.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack deploy -c stack.yml -c stack.prod.yml demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 508. docker stack — List stack services

**Purpose.** List services belonging to a stack.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** Stack name is positional.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack services demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 509. docker stack — List stack tasks

**Purpose.** Inspect task states for a stack.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** `--no-trunc` displays full errors.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack ps demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 510. docker stack — Remove a stack

**Purpose.** Remove stack services and stop their tasks.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** Argument is stack name.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack rm demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 511. docker stack — Render stack configuration

**Purpose.** Display resolved stack model before deploy.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** `-c` selects stack file.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack config -c stack.yml
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 512. docker stack — Deploy with registry auth

**Purpose.** Pass registry auth details to Swarm agents for private image pulls.

**Syntax.** `docker stack COMMAND [OPTIONS]`

**Options used.** `--with-registry-auth` propagates registry auth to agents.

**Arguments.** Stack operations use a Compose-like file and Swarm manager.

**Example.**

```
docker stack deploy --with-registry-auth -c stack.yml demo
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 513. docker secret — Create secret from file

**Purpose.** Store sensitive data as a Swarm secret.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** First argument is secret name; second is file or stdin source.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
docker secret create db_password ./db_password.txt
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 514. docker secret — Create secret from stdin

**Purpose.** Supply secret bytes through stdin.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** `-` reads stdin; `printf` is a shell utility.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
printf '%s' "$DB_PASSWORD" | docker secret create db_password -
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 515. docker secret — List secrets

**Purpose.** Inventory names/IDs without exposing payloads.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** `--filter` can narrow metadata.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
docker secret ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 516. docker secret — Inspect secret metadata

**Purpose.** Read metadata without retrieving plaintext payload.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** Argument is secret name/ID.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
docker secret inspect db_password
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 517. docker secret — Remove unused secret

**Purpose.** Delete secret after service references are removed.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** Argument is secret name/ID; active references can block removal.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
docker secret rm old_api_key
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 518. docker secret — Remove several unused secrets

**Purpose.** Delete a reviewed set of unreferenced secrets.

**Syntax.** `docker secret COMMAND [OPTIONS]`

**Options used.** Multiple secret names/IDs are accepted.

**Arguments.** Swarm secrets are manager-managed and mounted to authorized service tasks.

**Example.**

```
docker secret rm old_key old_password
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 519. docker config — Create config from file

**Purpose.** Store a non-sensitive configuration file in Swarm.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** First argument is config name; second is file or stdin.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
docker config create nginx_conf ./nginx.conf
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 520. docker config — Create config from stdin

**Purpose.** Create a config from generated text.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** `-` reads stdin; shell `printf` generates content.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
printf 'worker_processes auto;\n' | docker config create nginx_conf -
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 521. docker config — List configs

**Purpose.** Inventory config names and IDs.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** `--filter` narrows metadata.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
docker config ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 522. docker config — Inspect config metadata

**Purpose.** Read metadata for a Swarm config.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** Argument is config name/ID.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
docker config inspect nginx_conf
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 523. docker config — Remove unused config

**Purpose.** Delete a config no longer referenced by services.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** Remove service references first.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
docker config rm old_nginx_conf
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

### 524. docker config — Remove several configs

**Purpose.** Delete reviewed configs no longer in use.

**Syntax.** `docker config COMMAND [OPTIONS]`

**Options used.** Multiple config names/IDs are accepted.

**Arguments.** Swarm configs store non-secret configuration for services.

**Example.**

```
docker config rm old_nginx old_app
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

# 11 · Maintenance, security, plugins & advanced tooling

78 reference entries. Follow the examples in order and substitute environment-specific values.

### 525. docker system prune — Prune unused resources

**Purpose.** Remove eligible stopped containers, unused networks, dangling images, and build cache.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** Confirmation is shown by default.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove resources and possibly volumes. Review inventory and backups first.**

### 526. docker system prune — Include all unused images

**Purpose.** Expand image cleanup beyond dangling images.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `-a, --all` includes all unused images.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove resources and possibly volumes. Review inventory and backups first.**

## 527. docker system prune — Include unused volumes

**Purpose.** Expand cleanup to eligible unused volumes where supported.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `--volumes` includes eligible unused volumes.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --volumes
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove resources and possibly volumes. Review inventory and backups first.**

## 528. docker system prune — Skip confirmation

**Purpose.** Run a reviewed cleanup non-interactively.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `-f, --force` suppresses confirmation.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove resources and possibly volumes. Review inventory and backups first.**

## 529. docker system prune — Prune with age filter

**Purpose.** Apply an age cutoff to eligible resources where supported.

**Syntax.** `docker system prune [OPTIONS]`

**Options used.** `--filter until=...` applies supported time filter.

**Arguments.** No positional arguments.

**Example.**

```
docker system prune --filter 'until=168h'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: can remove resources and possibly volumes. Review inventory and backups first.**

## 530. docker builder — Show builder commands

**Purpose.** Discover builder-related commands.

**Syntax.** `docker builder COMMAND [OPTIONS]`

**Options used.** `--help` prints available subcommands.

**Arguments.** Legacy/compatibility builder namespace; Buildx is preferred for advanced builds.

**Example.**

```
docker builder --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 531. docker builder — Prune builder cache

**Purpose.** Remove unused legacy builder cache.

**Syntax.** `docker builder COMMAND [OPTIONS]`

**Options used.** `-f` skips confirmation; filters can limit scope.

**Arguments.** Legacy/compatibility builder namespace; Buildx is preferred for advanced builds.

**Example.**

```
docker builder prune
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 532. docker builder — Prune all builder cache

**Purpose.** Expand cleanup to all unused build cache.

**Syntax.** `docker builder COMMAND [OPTIONS]`

**Options used.** `--all` broadens cache cleanup where supported.

**Arguments.** Legacy/compatibility builder namespace; Buildx is preferred for advanced builds.

**Example.**

```
docker builder prune --all
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 533. docker builder — Skip cache prune prompt

**Purpose.** Run reviewed cache cleanup in automation.

**Syntax.** `docker builder COMMAND [OPTIONS]`

**Options used.** `--force` skips confirmation.

**Arguments.** Legacy/compatibility builder namespace; Buildx is preferred for advanced builds.

**Example.**

```
docker builder prune --force
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 534. docker plugin — List installed plugins

**Purpose.** Show installed plugin names and enabled state.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** `-q` prints concise identifiers.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin ls
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 535. docker plugin — Inspect plugin configuration

**Purpose.** Read mounts, capabilities, and settings.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** Argument is plugin name/ID.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin inspect vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 536. docker plugin — Install plugin

**Purpose.** Install a plugin from a registry.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** Plugin name/reference is positional; permissions may be requested.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin install vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 537. docker plugin — Install and grant requested permissions

**Purpose.** Accept all plugin permission requests.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** `--grant-all-permissions` accepts requested permissions.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin install --grant-all-permissions vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 538. docker plugin — Enable plugin

**Purpose.** Activate an installed plugin.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** Argument is plugin name/ID.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin enable vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 539. docker plugin — Disable plugin

**Purpose.** Deactivate a plugin.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** Argument is plugin name/ID; active use can block disable.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin disable vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 540. docker plugin — Force-disable plugin

**Purpose.** Request forced disable where supported.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** `--force` requests forced disable where supported.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin disable --force vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 541. docker plugin — Remove plugin

**Purpose.** Uninstall a plugin.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** Plugin generally must be disabled/not in use.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin rm vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 542. docker plugin — Inspect plugin settings as JSON

**Purpose.** Extract plugin settings for review.

**Syntax.** `docker plugin COMMAND [OPTIONS]`

**Options used.** `--format` selects settings as JSON.

**Arguments.** Engine plugin support varies by platform and Engine version.

**Example.**

```
docker plugin inspect --format '{{json .Settings}}' vieux/sshfs
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH RISK: plugins can request host mounts/capabilities; review privileges before install/enable.**

## 543. docker scout — Show Scout help

**Purpose.** Discover available image analysis commands.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** `--help` lists supported operations.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 544. docker scout — Quick image posture summary

**Purpose.** Review high-level vulnerabilities and package posture.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** Image reference is the analysis target.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout quickview local/api:dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 545. docker scout — List image CVEs

**Purpose.** Show known vulnerabilities associated with image packages.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** `cves` selects vulnerability report.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout cves local/api:dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 546. docker scout — Compare image versions

**Purpose.** Compare package/vulnerability state between image references.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** `--to` selects comparison target in supported versions.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout compare local/api:old --to local/api:new
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 547. docker scout — Show remediation recommendations

**Purpose.** Review available base-image/package update suggestions.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** Image reference selects analysis target; options vary by version.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout recommendations local/api:dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 548. docker scout — Quickview remote image

**Purpose.** Analyze an image in a registry by reference.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** Fully qualified image reference selects registry target.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout quickview registry.example.com/team/api:1.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 549. docker scout — Scan newly built image

**Purpose.** Run an image security report immediately after local build.

**Syntax.** `docker scout COMMAND [OPTIONS]`

**Options used.** Image must exist locally or be accessible to the Scout integration.

**Arguments.** Optional Docker Scout plugin/integration; availability and options depend on installation/version.

**Example.**

```
docker scout cves local/api:dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Scan results depend on image metadata and vulnerability database freshness. Review advisory details and test remediation changes.

**Related.** Related: `docker image inspect`, `docker buildx build --sbom`, official Docker Scout docs

**Safety.** Routine operation; verify the selected daemon/context and target.

## 550. docker trust — Show trust help

**Purpose.** Check whether legacy trust commands are available.

**Syntax.** `docker trust COMMAND [OPTIONS]`

**Options used.** `--help` lists installed trust operations.

**Arguments.** Legacy/optional Docker Content Trust command family; availability depends on CLI and registry setup.

**Example.**

```
docker trust --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH SENSITIVITY: protect signing keys and verify the artifact before signing or revoking trust metadata.**

## 551. docker trust — Inspect trust metadata

**Purpose.** Review signing metadata for a repository.

**Syntax.** `docker trust COMMAND [OPTIONS]`

**Options used.** Argument is repository reference.

**Arguments.** Legacy/optional Docker Content Trust command family; availability depends on CLI and registry setup.

**Example.**

```
docker trust inspect registry.example.com/team/api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH SENSITIVITY: protect signing keys and verify the artifact before signing or revoking trust metadata.**

## 552. docker trust — Sign an image tag

**Purpose.** Sign a repository tag using configured trust keys.

**Syntax.** `docker trust COMMAND [OPTIONS]`

**Options used.** Argument is image repository:tag; signing setup is required.

**Arguments.** Legacy/optional Docker Content Trust command family; availability depends on CLI and registry setup.

**Example.**

```
docker trust sign registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH SENSITIVITY: protect signing keys and verify the artifact before signing or revoking trust metadata.**

## 553. docker trust — Revoke a trust signature

**Purpose.** Remove a signature from a tag where supported by configured trust workflow.

**Syntax.** `docker trust COMMAND [OPTIONS]`

**Options used.** Argument is repository:tag; operation requires signing credentials.

**Arguments.** Legacy/optional Docker Content Trust command family; availability depends on CLI and registry setup.

**Example.**

```
docker trust revoke registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. HIGH SENSITIVITY: protect signing keys and verify the artifact before signing or revoking trust metadata.**

## 554. docker checkpoint — Check checkpoint availability

**Purpose.** Discover whether this Engine exposes checkpoint commands.

**Syntax.** `docker checkpoint COMMAND [OPTIONS]`

**Options used.** `--help` reports installed command surface.

**Arguments.** Checkpoint support is experimental/runtime/kernel-dependent and may be unavailable.

**Example.**

```
docker checkpoint --help
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Failures often indicate unsupported runtime, kernel features, or container configuration.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Experimental/limited feature; checkpoint portability and consistency are not guaranteed.

## 555. docker checkpoint — Create container checkpoint

**Purpose.** Capture process/container state where configured runtime supports it.

**Syntax.** `docker checkpoint COMMAND [OPTIONS]`

**Options used.** Arguments are container name/ID and checkpoint name.

**Arguments.** Checkpoint support is experimental/runtime/kernel-dependent and may be unavailable.

**Example.**

```
docker checkpoint create api checkpoint-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Failures often indicate unsupported runtime, kernel features, or container configuration.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Experimental/limited feature; checkpoint portability and consistency are not guaranteed.

## 556. docker checkpoint — List checkpoints

**Purpose.** List saved checkpoints for a container where supported.

**Syntax.** `docker checkpoint COMMAND [OPTIONS]`

**Options used.** Argument is container name/ID.

**Arguments.** Checkpoint support is experimental/runtime/kernel-dependent and may be unavailable.

**Example.**

```
docker checkpoint ls api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Failures often indicate unsupported runtime, kernel features, or container configuration.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Experimental/limited feature; checkpoint portability and consistency are not guaranteed.

## 557. docker checkpoint — Remove checkpoint

**Purpose.** Delete a saved checkpoint.

**Syntax.** `docker checkpoint COMMAND [OPTIONS]`

**Options used.** Arguments identify container and checkpoint.

**Arguments.** Checkpoint support is experimental/runtime/kernel-dependent and may be unavailable.

**Example.**

```
docker checkpoint rm api checkpoint-1
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** Failures often indicate unsupported runtime, kernel features, or container configuration.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Experimental/limited feature; checkpoint portability and consistency are not guaranteed.

## 558. docker context ls — Format context inventory

**Purpose.** Print context name and endpoint fields.

**Syntax.** `docker context ls [OPTIONS]`

**Options used.** `--format TEMPLATE` uses Go templates.

**Arguments.** No positional arguments.

**Example.**

```
docker context ls --format '{{.Name}} {{.DockerEndpoint}}'
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 559. docker context ls — Filter context names

**Purpose.** Narrow context list by supported name filter.

**Syntax.** `docker context ls [OPTIONS]`

**Options used.** `--filter name=VALUE` filters context list where supported.

**Arguments.** No positional arguments.

**Example.**

```
docker context ls --filter name=staging
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 560. docker container ls — List latest container record

**Purpose.** Show the latest created container.

**Syntax.** `docker container ls [OPTIONS]`

**Options used.** `-l` selects latest; `-a` includes stopped containers.

**Arguments.** No positional arguments.

**Example.**

```
docker container ls -l -a
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 561. docker container ls — Filter by ancestor image

**Purpose.** Find containers using an image or descendant.

**Syntax.** `docker container ls [OPTIONS]`

**Options used.** `--filter ancestor=IMAGE` selects image ancestry.

**Arguments.** No positional arguments.

**Example.**

```
docker container ls -a --filter ancestor=alpine:3.20
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 562. docker container inspect — Inspect status and exit code

**Purpose.** Extract current status and exit code.

**Syntax.** `docker container inspect [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-f` selects state fields.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker container inspect -f '{{.State.Status}} {{.State.ExitCode}}' failed-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 563. docker container inspect — Inspect configured command

**Purpose.** Read entrypoint and CMD arrays.

**Syntax.** `docker container inspect [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `-f` formats configuration fields.

**Arguments.** One or more container names/IDs.

**Example.**

```
docker container inspect -f '{{json .Config.Entrypoint}} {{json .Config.Cmd}}' api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 564. docker container logs — Read recent logs using namespace

**Purpose.** Retrieve recent captured output through the namespaced command.

**Syntax.** `docker container logs [OPTIONS] CONTAINER`

**Options used.** `--tail 50` limits history.

**Arguments.** Container name/ID.

**Example.**

```
docker container logs --tail 50 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 565. docker container exec — Execute command using namespace

**Purpose.** Run a process through the explicit container command family.

**Syntax.** `docker container exec [OPTIONS] CONTAINER COMMAND [ARG...]`

**Options used.** Command runs inside the container.

**Arguments.** Running container and executable.

**Example.**

```
docker container exec api id
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 566. docker container rm — Remove via namespaced command

**Purpose.** Delete a stopped container using the explicit namespace.

**Syntax.** `docker container rm [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Target must be stopped unless force is used.

**Arguments.** Container names/IDs.

**Example.**

```
docker container rm old-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 567. docker container start — Start via namespaced command

**Purpose.** Start an existing container.

**Syntax.** `docker container start [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Existing container names/IDs.

**Example.**

```
docker container start api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 568. docker container stop — Stop with explicit timeout

**Purpose.** Set grace period through namespaced command.

**Syntax.** `docker container stop [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--timeout 20` sets graceful-stop period.

**Arguments.** Running container names/IDs.

**Example.**

```
docker container stop --timeout 20 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 569. docker container restart — Restart via namespace

**Purpose.** Restart an existing container.

**Syntax.** `docker container restart [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** Optional timeout controls graceful stop where supported.

**Arguments.** Container names/IDs.

**Example.**

```
docker container restart api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 570. docker container pause — Pause via namespace

**Purpose.** Freeze container processes.

**Syntax.** `docker container pause CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Container names/IDs.

**Example.**

```
docker container pause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 571. docker container unpause — Resume via namespace

**Purpose.** Resume paused container processes.

**Syntax.** `docker container unpause CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Container names/IDs.

**Example.**

```
docker container unpause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 572. docker container update — Update CPU limit via namespace

**Purpose.** Set CPU quota through container command family.

**Syntax.** `docker container update [OPTIONS] CONTAINER [CONTAINER...]`

**Options used.** `--cpus 1.0` sets CPU quota where supported.

**Arguments.** Container names/IDs.

**Example.**

```
docker container update --cpus 1.0 api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 573. docker container wait — Wait via namespace

**Purpose.** Block until a container exits and print exit code.

**Syntax.** `docker container wait CONTAINER [CONTAINER...]`

**Options used.** No option required.

**Arguments.** Container names/IDs.

**Example.**

```
docker container wait batch-job
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 574. docker image pull — Pull through image namespace

**Purpose.** Download image using explicit image command family.

**Syntax.** `docker image pull [OPTIONS] NAME[:TAG|@DIGEST]`

**Options used.** Argument is image reference.

**Arguments.** Image reference.

**Example.**

```
docker image pull alpine:3.20
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 575. docker image push — Push through image namespace

**Purpose.** Publish an image using explicit image command family.

**Syntax.** `docker image push [OPTIONS] NAME[:TAG]`

**Options used.** Destination reference must be accessible and authenticated.

**Arguments.** Image reference.

**Example.**

```
docker image push registry.example.com/team/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 576. docker image save — Save through image namespace

**Purpose.** Export image archive through explicit image family.

**Syntax.** `docker image save [OPTIONS] IMAGE [IMAGE...]`

**Options used.** `-o` selects archive output.

**Arguments.** One or more image references.

**Example.**

```
docker image save -o api.tar local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 577. docker image load — Load through image namespace

**Purpose.** Restore an image archive through explicit image family.

**Syntax.** `docker image load [OPTIONS]`

**Options used.** `-i` selects archive input.

**Arguments.** Archive file or stdin.

**Example.**

```
docker image load -i api.tar
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 578. docker image tag — Tag through image namespace

**Purpose.** Add a local image reference using explicit image family.

**Syntax.** `docker image tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`

**Options used.** Source and target are positional references.

**Arguments.** Source and target references.

**Example.**

```
docker image tag local/api:dev local/api:1.0.0
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 579. docker image history — Review full image history

**Purpose.** Print full created-by commands through image namespace.

**Syntax.** `docker image history [OPTIONS] IMAGE`

**Options used.** `--no-trunc` disables truncation.

**Arguments.** Image reference.

**Example.**

```
docker image history --no-trunc local/api:dev
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 580. docker volume inspect — Inspect multiple volumes

**Purpose.** Compare metadata for multiple volumes.

**Syntax.** `docker volume inspect [OPTIONS] VOLUME [VOLUME...]`

**Options used.** Each argument is a volume name/ID.

**Arguments.** One or more volume names.

**Example.**

```
docker volume inspect app-data db-data
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 581. docker network inspect — Inspect multiple networks

**Purpose.** Compare configuration for several networks.

**Syntax.** `docker network inspect [OPTIONS] NETWORK [NETWORK...]`

**Options used.** Each argument is a network name/ID.

**Arguments.** One or more network names/IDs.

**Example.**

```
docker network inspect bridge host none
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 582. docker compose rm — Remove stopped Compose containers

**Purpose.** Remove stopped service containers.

**Syntax.** `docker compose rm [OPTIONS] [SERVICE...]`

**Options used.** `-f` skips prompt; `-v` removes anonymous volumes.

**Arguments.** Optional service names.

**Example.**

```
docker compose rm
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 583. docker compose rm — Remove one stopped service container

**Purpose.** Remove a specific service container.

**Syntax.** `docker compose rm [OPTIONS] [SERVICE...]`

**Options used.** Service name narrows target.

**Arguments.** Optional service names.

**Example.**

```
docker compose rm api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 584. docker compose rm — Force removal without prompt

**Purpose.** Remove selected stopped container without confirmation.

**Syntax.** `docker compose rm [OPTIONS] [SERVICE...]`

**Options used.** `--force` skips confirmation.

**Arguments.** Optional service names.

**Example.**

```
docker compose rm --force api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 585. docker compose kill — Kill all Compose containers

**Purpose.** Send default SIGKILL to project containers.

**Syntax.** `docker compose kill [OPTIONS] [SERVICE...]`

**Options used.** No signal option means default kill signal.

**Arguments.** Optional service names.

**Example.**

```
docker compose kill
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 586. docker compose kill — Send SIGTERM to one service

**Purpose.** Send chosen signal to selected service container.

**Syntax.** `docker compose kill [OPTIONS] [SERVICE...]`

**Options used.** `-s, --signal SIGNAL` selects signal.

**Arguments.** Optional service names.

**Example.**

```
docker compose kill -s SIGTERM api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 587. docker compose pause — Pause all services

**Purpose.** Freeze running project container processes.

**Syntax.** `docker compose pause [SERVICE...]`

**Options used.** No service argument targets project.

**Arguments.** Optional service names.

**Example.**

```
docker compose pause
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 588. docker compose pause — Pause one service

**Purpose.** Freeze one selected service.

**Syntax.** `docker compose pause [SERVICE...]`

**Options used.** Service name narrows scope.

**Arguments.** Optional service names.

**Example.**

```
docker compose pause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 589. docker compose unpause — Resume all services

**Purpose.** Resume paused project containers.

**Syntax.** `docker compose unpause [SERVICE...]`

**Options used.** No service argument targets project.

**Arguments.** Optional service names.

**Example.**

```
docker compose unpause
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 590. docker compose unpause — Resume one service

**Purpose.** Resume a selected service.

**Syntax.** `docker compose unpause [SERVICE...]`

**Options used.** Service name narrows scope.

**Arguments.** Optional service names.

**Example.**

```
docker compose unpause worker
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 591. docker compose port — Find published service port

**Purpose.** Show host mapping for a service port.

**Syntax.** `docker compose port SERVICE PRIVATE_PORT`

**Options used.** Arguments are service and private port.

**Arguments.** Service name and private port.

**Example.**

```
docker compose port web 80
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 592. docker compose port — Find DNS UDP mapping

**Purpose.** Check a configured DNS port mapping.

**Syntax.** `docker compose port SERVICE PRIVATE_PORT`

**Options used.** Service/port select mapping; protocol follows Compose config.

**Arguments.** Service name and private port.

**Example.**

```
docker compose port dns 53
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 593. docker compose down — Remove project containers and networks

**Purpose.** Tear down project resources while retaining named volumes by default.

**Syntax.** `docker compose down [OPTIONS]`

**Options used.** Default behavior keeps named volumes.

**Arguments.** No positional arguments.

**Example.**

```
docker compose down
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 594. docker compose down — Remove project images too

**Purpose.** Remove eligible local images during teardown.

**Syntax.** `docker compose down [OPTIONS]`

**Options used.** `--rmi local` removes local images without custom tags.

**Arguments.** No positional arguments.

**Example.**

```
docker compose down --rmi local
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety. DESTRUCTIVE: verify the exact target and backups before running.**

## 595. docker compose exec — Run as non-root user

**Purpose.** Execute a command as selected UID/GID.

**Syntax.** `docker compose exec [OPTIONS] SERVICE COMMAND [ARGS...]`

**Options used.** `--user UID[:GID]` overrides exec user.

**Arguments.** Service and command.

**Example.**

```
docker compose exec --user 10001:10001 api id
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 596. docker compose exec — Set working directory

**Purpose.** Run a command from selected path.

**Syntax.** `docker compose exec [OPTIONS] SERVICE COMMAND [ARGS...]`

**Options used.** `--workdir DIR` sets working directory.

**Arguments.** Service and command.

**Example.**

```
docker compose exec --workdir /app api pwd
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 597. docker compose build — Pull base image while building

**Purpose.** Attempt to refresh referenced base images.

**Syntax.** `docker compose build [OPTIONS] [SERVICE...]`

**Options used.** `--pull` attempts to pull newer base images.

**Arguments.** Optional service names.

**Example.**

```
docker compose build --pull api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 598. docker compose logs — Show logs until cutoff

**Purpose.** Limit log query by upper time bound where supported.

**Syntax.** `docker compose logs [OPTIONS] [SERVICE...]`

**Options used.** `--until` sets end time; replace illustrative timestamp.

**Arguments.** Optional service names.

**Example.**

```
docker compose logs --until 2025-01-01T10:15:00Z api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

## 599. docker compose cp — Copy directory from host

**Purpose.** Copy host directory into service container.

**Syntax.** `docker compose cp [OPTIONS] SRC_PATH DEST_PATH`

**Options used.** Host path is source; service path is destination.

**Arguments.** One side uses `SERVICE:PATH`.

**Example.**

```
docker compose cp ./fixtures api:/tmp/fixtures
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 600. docker compose cp — Copy with archive semantics

**Purpose.** Request archive ownership/permission semantics.

**Syntax.** `docker compose cp [OPTIONS] SRC_PATH DEST_PATH`

**Options used.** `-a` requests archive behavior where supported.

**Arguments.** One side uses `SERVICE:PATH`.

**Example.**

```
docker compose cp -a ./artifact api:/tmp/artifact
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 601. docker compose watch — Watch one development service

**Purpose.** Apply configured watch actions for a service.

**Syntax.** `docker compose watch [OPTIONS] [SERVICE...]`

**Options used.** Requires supported Compose version and develop/watch rules.

**Arguments.** Optional service names and watch configuration.

**Example.**

```
docker compose watch api
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

### 602. docker compose watch — Start services with watch

**Purpose.** Bring up services and activate configured watch workflow.

**Syntax.** `docker compose watch [OPTIONS] [SERVICE...]`

**Options used.** `--watch` enables watch mode where supported.

**Arguments.** Optional service names and watch configuration.

**Example.**

```
docker compose up --watch
```

**Expected result.** Docker performs the requested operation and prints status/output; exact output varies by Engine, CLI, image, and platform.

**Troubleshooting.** If it fails, check the active context (`docker context show`), target name/ID, permissions, and the command-specific `--help` output.

**Related.** Related: inspect the resource, review logs/state, and consult the corresponding official Docker CLI reference.

**Safety.** Routine operation; verify the selected daemon/context and target.

# 12 · Dockerfile syntax (not CLI commands)

20 reference entries. Follow the examples in order and substitute environment-specific values.

## 603. Dockerfile: FROM — FROM

**Purpose.** Selects the base image/stage.

**Syntax.** `FROM [--platform=PLATFORM] IMAGE[:TAG|@DIGEST] [AS NAME]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
FROM golang:1.23 AS build
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Use trusted, maintained bases; pin digests for reproducibility.

## 604. Dockerfile: RUN — RUN

**Purpose.** Executes build-time commands and creates a layer.

**Syntax.** `RUN COMMAND | RUN ["executable","arg",...]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
RUN apk add --no-cache ca-certificates
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Combine related package actions and clean caches in the same layer.

## 605. Dockerfile: CMD — CMD

**Purpose.** Defines default command/arguments at container start.

**Syntax.** `CMD ["executable","arg",...] | CMD COMMAND`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
CMD ["./server"]
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Exec form usually provides predictable signal behavior.

## 606. Dockerfile: ENTRYPOINT — ENTRYPOINT

**Purpose.** Defines the main executable.

**Syntax.** `ENTRYPOINT ["executable","arg",...] | ENTRYPOINT COMMAND`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
ENTRYPOINT ["/app"]
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Understand how runtime arguments combine with CMD/ENTRYPOINT.

## 607. Dockerfile: COPY — COPY

**Purpose.** Copies context/stage files into image.

**Syntax.** `COPY [OPTIONS] SRC... DEST`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
COPY --chown=10001:10001 . /app
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety. Keep .dockerignore accurate and never copy local secrets.**

## 608. Dockerfile: ADD — ADD

**Purpose.** Copies files and has additional extraction/remote-source behavior.

**Syntax.** `ADD [OPTIONS] SRC... DEST`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
ADD assets.tar /opt/assets/
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Prefer COPY unless ADD-specific behavior is needed.

## 609. Dockerfile: WORKDIR — WORKDIR

**Purpose.** Sets working directory for later instructions.

**Syntax.** `WORKDIR /absolute/or/relative/path`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
WORKDIR /app
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Prefer explicit absolute paths.

## 610. Dockerfile: ENV — ENV

**Purpose.** Sets image/runtime environment variables.

**Syntax.** `ENV KEY=VALUE ...`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
ENV NODE_ENV=production
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety. Values persist in image config; not for secrets.**

## 611. Dockerfile: ARG — ARG

**Purpose.** Declares build-time argument.

**Syntax.** `ARG NAME[=DEFAULT]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
ARG GO_VERSION=1.23
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety. Not secret storage; values can leak into metadata/logs.**

## 612. Dockerfile: EXPOSE — EXPOSE

**Purpose.** Documents intended container port; does not publish host port.

**Syntax.** `EXPOSE PORT[/PROTOCOL] ...`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
EXPOSE 8080/tcp
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Use `docker run -p` or Compose `ports` to publish.

## 613. Dockerfile: USER — USER

**Purpose.** Sets user for later build steps and default runtime process.

**Syntax.** `USER USER[:GROUP] | USER UID[:GID]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
USER 10001:10001
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Ensure writable paths have appropriate ownership.

## 614. Dockerfile: VOLUME — VOLUME

**Purpose.** Declares mount points intended for persistent/external data.

**Syntax.** `VOLUME ["/data"]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
VOLUME ["/var/lib/app"]
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Understand anonymous volume behavior and lifecycle.

## 615. Dockerfile: HEALTHCHECK — HEALTHCHECK

**Purpose.** Defines a health probe.

**Syntax.** `HEALTHCHECK [OPTIONS] CMD command | HEALTHCHECK NONE`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
HEALTHCHECK --interval=30s CMD curl -f http://localhost/ || exit 1
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Ensure probe tool exists and checks readiness.

## 616. Dockerfile: STOPSIGNAL — STOPSIGNAL

**Purpose.** Sets signal sent during container stop.

**Syntax.** `STOPSIGNAL SIGNAL`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
STOPSIGNAL SIGTERM
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Application must handle the signal.

## 617. Dockerfile: SHELL — SHELL

**Purpose.** Changes shell for shell-form RUN/CMD/ENTRYPOINT.

**Syntax.** `SHELL ["executable","parameters"]`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
SHELL ["/bin/sh","-c"]
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Does not affect exec-form JSON arrays.

## 618. Dockerfile: LABEL — LABEL

**Purpose.** Adds image metadata.

**Syntax.** `LABEL KEY=VALUE ...`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
LABEL org.opencontainers.image.title="API"
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety. Labels are inspectable; do not store credentials.**

### 619. Dockerfile: ONBUILD — ONBUILD

**Purpose.** Registers a trigger for downstream builds.

**Syntax.** `ONBUILD INSTRUCTION`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
ONBUILD COPY . /src
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Use sparingly; downstream behavior can surprise.

### 620. Dockerfile: Multi-stage build — Multi-stage build

**Purpose.** Separates build tools from runtime files.

**Syntax.** `FROM IMAGE AS build ... FROM IMAGE AS runtime`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
COPY --from=build /out/app /usr/local/bin/app
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Copy only runtime artifacts into final stage.

### 621. Dockerfile: BuildKit secret mount — BuildKit secret mount

**Purpose.** Exposes a secret to one supported build step.

**Syntax.** `RUN --mount=type=secret,id=TOKEN command`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety. Never print or copy secret contents into layers.**

### 622. Dockerfile: BuildKit cache mount — BuildKit cache mount

**Purpose.** Provides a persistent build cache mount.

**Syntax.** `RUN --mount=type=cache,target=/root/.cache command`

**Options used.** Dockerfile instruction, not an executable Docker CLI command.

**Arguments.** Instruction-specific arguments; source paths are normally relative to the build context.

**Example.**

```
RUN --mount=type=cache,target=/root/.cache go build ./...
```

**Expected result.** Affects image construction, runtime defaults, or image metadata when the Dockerfile is built.

**Troubleshooting.** Check the Dockerfile reference and selected frontend/version; instruction semantics can depend on frontend support.

**Related.** Related: `docker build`, `docker buildx build`, `.dockerignore`

**Safety.** Cache improves performance but should not affect correctness.

# 13 · Practical combined workflows (CLI recipes)

8 reference entries. Follow the examples in order and substitute environment-specific values.

## 623. Workflow: Verify a fresh install — Verify a fresh install

**Purpose.** Confirm client/server connectivity, inspect Engine information, and run the test image.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker version
docker info
docker run --rm hello-world
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 624. Workflow: Run and troubleshoot a web server — Run and troubleshoot a web server

**Purpose.** Uses loopback binding for local testing; check port conflicts and logs if startup fails.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker pull nginx:alpine
docker run -d --name web -p 127.0.0.1:8080:80 nginx:alpine
docker ps
docker port web
docker logs --tail 50 web
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 625. Workflow: Build, tag, and publish — Build, tag, and publish

**Purpose.** Authenticate before push and inspect the image for accidental secrets.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker build -t local/api:1.0.0 .
docker image inspect local/api:1.0.0
docker tag local/api:1.0.0 registry.example.com/team/api:1.0.0
docker push registry.example.com/team/api:1.0.0
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 626. Workflow: Create a network and persistent volume — Create a network and persistent volume

**Purpose.** Use the image-documented data path and backup persistent data.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker network create app-net
docker volume create db-data
docker run -d --name db --network app-net --mount \
type=volume,src=db-data,dst=/var/lib/postgresql/data postgres:16
docker network inspect app-net
docker volume inspect db-data
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 627. Workflow: Diagnose a failed container — Diagnose a failed container

**Purpose.** Combine state, logs, events, and writable-layer changes; `docker diff` excludes mounted-volume contents.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker ps -a
docker logs --timestamps --tail 200 failed-job
docker inspect -f '{{.State.Status}} {{.State.ExitCode}} {{.State.Error}}' failed-job
docker events --filter type=container --since 15m
docker diff failed-job
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 628. Workflow: Compose deployment and teardown — Compose deployment and teardown

**Purpose.** Do not use `down --volumes` unless deleting data is intentional.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker compose config --quiet
docker compose pull
docker compose up -d --wait
docker compose ps
docker compose logs -f --tail 100
docker compose down
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 629. Workflow: Review before cleanup — Review before cleanup

**Purpose.** Review retention requirements and backup state before confirming prune.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker system df -v
docker ps -a
docker image ls
docker volume ls
docker network ls
docker system prune
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

## 630. Workflow: Swarm deployment verification — Swarm deployment verification

**Purpose.** Replace placeholder address with a reachable IP; keep join tokens secret and preserve manager quorum.

**Syntax.** `A sequence of Docker CLI commands executed in order.`

**Options used.** Each command has its own flags; see the corresponding entry for definitions.

**Arguments.** Replace placeholder names, registry hostnames, paths, tags, and IP addresses with environment-specific values.

**Example.**

```
docker swarm init --advertise-addr <reachable-manager-ip>
docker swarm join-token worker
docker node ls
docker stack deploy -c stack.yml demo
docker stack services demo
docker stack ps demo
```

**Expected result.** The sequence performs the described workflow if prerequisites and permissions are satisfied.

**Troubleshooting.** Run commands one at a time during troubleshooting; inspect exit status and use the related command entries for details.

**Related.** Related: all command families used in the recipe.

**Safety.** Review effects before running in production; workflows may create resources, publish ports, deploy services, or remove resources.

# Appendix A · Quick option glossary

| Option | Meaning |
|---|---|
| `-d`, `--detach` | Run a container in the background. |
| `-i`, `--interactive` | Keep standard input open. |
| `-t`, `--tty` | Allocate a pseudo-terminal. |
| `-p`, `--publish` | Publish/map a container port to the host. |
| `-P`, `--publish-all` | Publish image-declared ports to dynamic host ports. |
| `-v`, `--volume` | Mount a volume or bind path using short syntax. |
| `--mount` | Specify mount type and attributes in key-value syntax. |
| `-e`, `--env` | Set an environment variable. |
| `--env-file` | Read environment entries from a file. |
| `--name` | Assign a unique name. |
| `--network` | Select network or network mode. |
| `--restart` | Set container restart policy. |
| `--memory` | Set memory limit where supported. |
| `--cpus` | Set CPU quota in CPU units where supported. |
| `--filter` | Filter command results using command-specific syntax. |
| `-q`, `--quiet` | Reduce output; exact meaning is command-specific. |
| `--format` / `-f` | Format output using Go templates in commands that support it; `-f` has other meanings in some commands. |
| `--force` / `-f` | Skip prompts or force an operation; meaning is command-specific. |
| `--no-cache` | Disable build cache reuse. |
| `--pull` | Attempt to pull newer referenced images. |
| `--platform` | Select OS/architecture/variant where supported. |
| `--secret` | Expose a BuildKit secret to a supported build step; not a generic runtime flag. |
| `--ssh` | Forward SSH agent/key to a BuildKit step. |
| `--target` | Select a build stage or command-specific target. |
| `--since`, `--until` | Limit logs/events to a time range where supported. |
| `--timeout`, `-t` | Set a timeout in commands that define one; short flags are not globally interchangeable. |
| `--read-only` | Make container root filesystem read-only. |
| `--cap-drop`, `--cap-add` | Remove/add Linux capabilities on supported Linux runtimes. |
| `--security-opt` | Set supported runtime security options. |
| `--label` | Attach metadata; labels are not secret storage. |
| `--no-trunc` | Avoid truncating IDs/output in supported commands. |
| `--rm` | Remove a container after its main process exits or remove a one-off Compose container. |

# Appendix B · Glossary

| Term | Definition |
|---|---|
| **Image** | Filesystem/configuration template used to create containers; tags are mutable references. |
| **Container** | A running or stopped instance of an image with runtime configuration and a writable layer. |
| **Engine/daemon** | Docker service that manages images, containers, networks, volumes, and related resources. |
| **CLI** | Command-line client that sends requests to the Engine API. |
| **Registry** | Service that stores/distributes image manifests and layers. |
| **Tag** | Mutable human-readable image reference such as `1.4.2` or `latest`. |
| **Digest** | Content-addressed identifier, commonly `sha256:...`. |
| **Build context** | Files/metadata made available to a build, controlled by context path and `.dockerignore`. |
| **Dockerfile** | Text recipe used to build an image. |
| **BuildKit/Buildx** | Build engine and CLI plugin for caching, exporters, multi-platform builds, and attestations. |
| **Compose** | Declarative model for multi-container applications. |
| **Volume** | Docker-managed persistent storage independent of container lifecycle. |
| **Bind mount** | Mount mapping a host path into a container. |
| **Network** | Docker-managed connectivity boundary. |
| **Swarm** | Built-in clustering/orchestration mode for services, tasks, nodes, secrets, and stacks. |
| **Service** | Swarm desired-state definition that schedules and replaces tasks. |
| **Task** | Scheduled instance of a service assigned to a node. |
| **Health check** | Probe and timing policy used to report container health. |
| **Capability** | Fine-grained Linux privilege that can be granted/dropped. |
| **Secret** | Swarm-managed sensitive value mounted to authorized service tasks. |
| **Config** | Swarm-managed non-secret configuration object mounted to tasks. |
| **Context** | Named CLI endpoint/configuration selecting local or remote Engine. |
| **Prune** | Cleanup operation that removes resources considered unused. |
| **OCI** | Open Container Initiative standards for images and runtimes. |
| **SBOM** | Software Bill of Materials: inventory of components in an artifact. |
| **Provenance** | Metadata describing how an artifact was produced. |

# Appendix C · Official references and version checks

These official references are the primary starting points for verifying current syntax and platform/version availability. Local CLI help is the final practical check for the installed environment.

**Docker CLI reference**
https://docs.docker.com/reference/cli/docker/

**Docker run**
https://docs.docker.com/reference/cli/docker/container/run/

**Docker Buildx**
https://docs.docker.com/reference/cli/docker/buildx/

**Buildx build options**
https://docs.docker.com/reference/cli/docker/buildx/build/

**Docker Compose CLI**
https://docs.docker.com/reference/cli/docker/compose/

**Compose file reference**
https://docs.docker.com/reference/compose-file/

**Dockerfile reference**
https://docs.docker.com/reference/dockerfile/

**Docker networking**
https://docs.docker.com/engine/network/

**Docker volumes**
https://docs.docker.com/engine/storage/volumes/

**Docker Engine security**
https://docs.docker.com/engine/security/

**Docker Swarm**
https://docs.docker.com/engine/swarm/

**Swarm CLI reference**
https://docs.docker.com/reference/cli/docker/swarm/

**Docker contexts**
https://docs.docker.com/engine/manage-resources/contexts/

**Docker daemon configuration**
https://docs.docker.com/engine/daemon/

**Docker Scout**
https://docs.docker.com/scout/

**Before production use: check `docker version`; verify `docker context show`; inspect resolved Compose configuration; test destructive operations on non-production resources; and validate backup/restore procedures.**

# Appendix D · Command inventory and alphabetical index

The inventory below lists all 630 numbered entries. Page numbers were captured from the generated PDF; Dockerfile instructions and combined workflows are labeled separately from executable CLI entries.

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 001 | docker version — Compare client and server versions | 01 · Fundamentals, installation & CLI | 4 |
| 002 | docker version — Extract client version as JSON | 01 · Fundamentals, installation & CLI | 4 |
| 003 | docker version — Extract server API version | 01 · Fundamentals, installation & CLI | 4 |
| 004 | docker version — Compare client and server API values | 01 · Fundamentals, installation & CLI | 5 |
| 005 | docker version — Check server reachability | 01 · Fundamentals, installation & CLI | 5 |
| 006 | docker info — Display Engine configuration | 01 · Fundamentals, installation & CLI | 5 |
| 007 | docker info — Extract storage driver | 01 · Fundamentals, installation & CLI | 5 |
| 008 | docker info — List configured runtimes | 01 · Fundamentals, installation & CLI | 6 |
| 009 | docker info — Inspect security options | 01 · Fundamentals, installation & CLI | 6 |
| 010 | docker info — Check daemon root directory | 01 · Fundamentals, installation & CLI | 6 |
| 011 | docker --help — List top-level commands | 01 · Fundamentals, installation & CLI | 6 |
| 012 | docker --help — Inspect run options | 01 · Fundamentals, installation & CLI | 7 |
| 013 | docker --help — Inspect Compose options | 01 · Fundamentals, installation & CLI | 7 |
| 014 | docker --help — Use a specific client config directory | 01 · Fundamentals, installation & CLI | 7 |
| 015 | docker --help — Target a named context for one command | 01 · Fundamentals, installation & CLI | 7 |
| 016 | docker --help — Target a Docker host endpoint | 01 · Fundamentals, installation & CLI | 8 |
| 017 | docker system df — Summarize Docker disk usage | 01 · Fundamentals, installation & CLI | 8 |
| 018 | docker system df — Show verbose resource usage | 01 · Fundamentals, installation & CLI | 8 |
| 019 | docker events — Stream all Engine events | 01 · Fundamentals, installation & CLI | 8 |
| 020 | docker events — Filter to container events | 01 · Fundamentals, installation & CLI | 9 |
| 021 | docker events — Filter to die events | 01 · Fundamentals, installation & CLI | 9 |
| 022 | docker events — Limit events to recent interval | 01 · Fundamentals, installation & CLI | 9 |
| 023 | docker events — Bound event query with until | 01 · Fundamentals, installation & CLI | 9 |
| 024 | docker events — Filter by image reference | 01 · Fundamentals, installation & CLI | 10 |
| 025 | docker system prune — Prune unused resources conservatively | 01 · Fundamentals, installation & CLI | 10 |
| 026 | docker system prune — Include all unused images | 01 · Fundamentals, installation & CLI | 10 |
| 027 | docker system prune — Skip confirmation in automation | 01 · Fundamentals, installation & CLI | 10 |
| 028 | docker system prune — Prune with age filter | 01 · Fundamentals, installation & CLI | 11 |
| 029 | docker system prune — Include unused volumes | 01 · Fundamentals, installation & CLI | 11 |
| 030 | docker run — Detached service | 02 · Containers: run, create & lifecycle | 11 |
| 031 | docker run — Auto-remove one-shot task | 02 · Containers: run, create & lifecycle | 12 |
| 032 | docker run — Interactive shell | 02 · Containers: run, create & lifecycle | 12 |
| 033 | docker run — Publish a host port | 02 · Containers: run, create & lifecycle | 12 |
| 034 | docker run — Bind published port to loopback | 02 · Containers: run, create & lifecycle | 12 |
| 035 | docker run — Publish all exposed ports | 02 · Containers: run, create & lifecycle | 13 |
| 036 | docker run — Set one environment variable | 02 · Containers: run, create & lifecycle | 13 |
| 037 | docker run — Read environment from file | 02 · Containers: run, create & lifecycle | 13 |
| 038 | docker run — Mount a named volume | 02 · Containers: run, create & lifecycle | 13 |
| 039 | docker run — Use explicit volume mount syntax | 02 · Containers: run, create & lifecycle | 14 |
| 040 | docker run — Connect to custom network | 02 · Containers: run, create & lifecycle | 14 |
| 041 | docker run — Set restart policy | 02 · Containers: run, create & lifecycle | 14 |
| 042 | docker run — Limit memory | 02 · Containers: run, create & lifecycle | 14 |
| 043 | docker run — Limit CPU quota | 02 · Containers: run, create & lifecycle | 15 |
| 044 | docker run — Pin to selected CPUs | 02 · Containers: run, create & lifecycle | 15 |
| 045 | docker run — Read-only root filesystem | 02 · Containers: run, create & lifecycle | 15 |
| 046 | docker run — Run as non-root UID | 02 · Containers: run, create & lifecycle | 15 |
| 047 | docker run — Set working directory | 02 · Containers: run, create & lifecycle | 16 |
| 048 | docker run — Override entrypoint | 02 · Containers: run, create & lifecycle | 16 |
| 049 | docker run — Override default command | 02 · Containers: run, create & lifecycle | 16 |
| 050 | docker run — Add a host mapping | 02 · Containers: run, create & lifecycle | 16 |
| 051 | docker run — Set DNS resolver | 02 · Containers: run, create & lifecycle | 17 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 052 | docker run — Set DNS search domain | 02 · Containers: run, create & lifecycle | 17 |
| 053 | docker run — Set hostname | 02 · Containers: run, create & lifecycle | 17 |
| 054 | docker run — Add metadata labels | 02 · Containers: run, create & lifecycle | 17 |
| 055 | docker run — Configure health check | 02 · Containers: run, create & lifecycle | 18 |
| 056 | docker run — Disable inherited health check | 02 · Containers: run, create & lifecycle | 18 |
| 057 | docker run — Set stop grace period | 02 · Containers: run, create & lifecycle | 18 |
| 058 | docker run — Set PID limit | 02 · Containers: run, create & lifecycle | 18 |
| 059 | docker run — Drop all capabilities and add one | 02 · Containers: run, create & lifecycle | 19 |
| 060 | docker run — Enable no-new-privileges | 02 · Containers: run, create & lifecycle | 19 |
| 061 | docker run — Use host networking | 02 · Containers: run, create & lifecycle | 19 |
| 062 | docker run — Increase shared memory | 02 · Containers: run, create & lifecycle | 19 |
| 063 | docker run — Use init process | 02 · Containers: run, create & lifecycle | 20 |
| 064 | docker run — Select image platform | 02 · Containers: run, create & lifecycle | 20 |
| 065 | docker run — Select logging driver | 02 · Containers: run, create & lifecycle | 20 |
| 066 | docker run — Configure JSON log rotation | 02 · Containers: run, create & lifecycle | 20 |
| 067 | docker run — Request GPU devices | 02 · Containers: run, create & lifecycle | 21 |
| 068 | docker run — Constrain memory and swap together | 02 · Containers: run, create & lifecycle | 21 |
| 069 | docker run — Use a deterministic name | 02 · Containers: run, create & lifecycle | 21 |
| 070 | docker run — Run a shell expression in container | 02 · Containers: run, create & lifecycle | 21 |
| 071 | docker run — Set custom stop signal | 02 · Containers: run, create & lifecycle | 22 |
| 072 | docker run — Configure OOM-killer behavior | 02 · Containers: run, create & lifecycle | 22 |
| 073 | docker run — Expose an internal port in metadata | 02 · Containers: run, create & lifecycle | 22 |
| 074 | docker run — Set domain name | 02 · Containers: run, create & lifecycle | 22 |
| 075 | docker run — Set a MAC address | 02 · Containers: run, create & lifecycle | 23 |
| 076 | docker run — Set a sysctl | 02 · Containers: run, create & lifecycle | 23 |
| 077 | docker run — Set an IPC namespace mode | 02 · Containers: run, create & lifecycle | 23 |
| 078 | docker run — Set PID namespace mode | 02 · Containers: run, create & lifecycle | 23 |
| 079 | docker run — Set user namespace mode | 02 · Containers: run, create & lifecycle | 24 |
| 080 | docker run — Limit block I/O read rate | 02 · Containers: run, create & lifecycle | 24 |
| 081 | docker run — Limit number of open files | 02 · Containers: run, create & lifecycle | 24 |
| 082 | docker run — Set cgroup parent | 02 · Containers: run, create & lifecycle | 24 |
| 083 | docker run — Use a custom runtime | 02 · Containers: run, create & lifecycle | 25 |
| 084 | docker run — Set detach key sequence | 02 · Containers: run, create & lifecycle | 25 |
| 085 | docker run — Disable signal proxying | 02 · Containers: run, create & lifecycle | 25 |
| 086 | docker run — Write container ID to a file | 02 · Containers: run, create & lifecycle | 25 |
| 087 | docker run — Set container restart maximum | 02 · Containers: run, create & lifecycle | 26 |
| 088 | docker run — Mount a bind path with explicit syntax | 02 · Containers: run, create & lifecycle | 26 |
| 089 | docker run — Mount bind path read-only | 02 · Containers: run, create & lifecycle | 26 |
| 090 | docker run — Use tmpfs with options | 02 · Containers: run, create & lifecycle | 26 |
| 091 | docker run — Use custom entrypoint and args | 02 · Containers: run, create & lifecycle | 27 |
| 092 | docker run — Set environment variable from host | 02 · Containers: run, create & lifecycle | 27 |
| 093 | docker ps — List running containers | 02 · Containers: run, create & lifecycle | 27 |
| 094 | docker ps — Include stopped containers | 02 · Containers: run, create & lifecycle | 27 |
| 095 | docker ps — Print container IDs only | 02 · Containers: run, create & lifecycle | 28 |
| 096 | docker ps — Filter by name | 02 · Containers: run, create & lifecycle | 28 |
| 097 | docker ps — Filter by image ancestor | 02 · Containers: run, create & lifecycle | 28 |
| 098 | docker ps — Filter by status | 02 · Containers: run, create & lifecycle | 28 |
| 099 | docker ps — Filter by label | 02 · Containers: run, create & lifecycle | 29 |
| 100 | docker ps — Show most recently created | 02 · Containers: run, create & lifecycle | 29 |
| 101 | docker ps — Show last five containers | 02 · Containers: run, create & lifecycle | 29 |
| 102 | docker ps — Format rows as JSON | 02 · Containers: run, create & lifecycle | 29 |
| 103 | docker ps — Disable output truncation | 02 · Containers: run, create & lifecycle | 30 |
| 104 | docker ps — Filter by published port | 02 · Containers: run, create & lifecycle | 30 |
| 105 | docker ps — Filter by network | 02 · Containers: run, create & lifecycle | 30 |
| 106 | docker ps — Filter by exit code | 02 · Containers: run, create & lifecycle | 30 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 107 | docker ps — Filter by health status | 02 · Containers: run, create & lifecycle | 31 |
| 108 | docker logs — Read captured logs | 02 · Containers: run, create & lifecycle | 31 |
| 109 | docker logs — Follow logs live | 02 · Containers: run, create & lifecycle | 31 |
| 110 | docker logs — Show last 100 lines | 02 · Containers: run, create & lifecycle | 31 |
| 111 | docker logs — Show logs from last ten minutes | 02 · Containers: run, create & lifecycle | 32 |
| 112 | docker logs — Include timestamps | 02 · Containers: run, create & lifecycle | 32 |
| 113 | docker logs — Follow recent lines with timestamps | 02 · Containers: run, create & lifecycle | 32 |
| 114 | docker logs — Bound logs by start/end time | 02 · Containers: run, create & lifecycle | 32 |
| 115 | docker logs — Include logging metadata | 02 · Containers: run, create & lifecycle | 33 |
| 116 | docker logs — Read logs from stopped container | 02 · Containers: run, create & lifecycle | 33 |
| 117 | docker logs — Follow only errors by shell filter | 02 · Containers: run, create & lifecycle | 33 |
| 118 | docker exec — Run a diagnostic command | 02 · Containers: run, create & lifecycle | 33 |
| 119 | docker exec — Open an interactive shell | 02 · Containers: run, create & lifecycle | 34 |
| 120 | docker exec — Run as a specific user | 02 · Containers: run, create & lifecycle | 34 |
| 121 | docker exec — Set working directory | 02 · Containers: run, create & lifecycle | 34 |
| 122 | docker exec — Set temporary environment variable | 02 · Containers: run, create & lifecycle | 34 |
| 123 | docker exec — Load environment file | 02 · Containers: run, create & lifecycle | 35 |
| 124 | docker exec — Run detached exec task | 02 · Containers: run, create & lifecycle | 35 |
| 125 | docker exec — Set detach keys | 02 · Containers: run, create & lifecycle | 35 |
| 126 | docker exec — Check database readiness | 02 · Containers: run, create & lifecycle | 35 |
| 127 | docker exec — Check filesystem usage | 02 · Containers: run, create & lifecycle | 36 |
| 128 | docker exec — Inspect environment safely by key | 02 · Containers: run, create & lifecycle | 36 |
| 129 | docker exec — Run command as root explicitly | 02 · Containers: run, create & lifecycle | 36 |
| 130 | docker inspect — Inspect full container JSON | 02 · Containers: run, create & lifecycle | 36 |
| 131 | docker inspect — Extract container IP address | 02 · Containers: run, create & lifecycle | 37 |
| 132 | docker inspect — Inspect health status | 02 · Containers: run, create & lifecycle | 37 |
| 133 | docker inspect — Inspect restart policy | 02 · Containers: run, create & lifecycle | 37 |
| 134 | docker inspect — Inspect mount points | 02 · Containers: run, create & lifecycle | 37 |
| 135 | docker inspect — Inspect labels | 02 · Containers: run, create & lifecycle | 38 |
| 136 | docker inspect — Inspect image entrypoint and CMD | 02 · Containers: run, create & lifecycle | 38 |
| 137 | docker inspect — Disambiguate object type | 02 · Containers: run, create & lifecycle | 38 |
| 138 | docker create — Create without starting | 02 · Containers: run, create & lifecycle | 38 |
| 139 | docker create — Create with port mapping | 02 · Containers: run, create & lifecycle | 39 |
| 140 | docker create — Create with persistent volume | 02 · Containers: run, create & lifecycle | 39 |
| 141 | docker create — Create with environment | 02 · Containers: run, create & lifecycle | 39 |
| 142 | docker create — Create on custom network | 02 · Containers: run, create & lifecycle | 39 |
| 143 | docker create — Create read-only root filesystem | 02 · Containers: run, create & lifecycle | 40 |
| 144 | docker create — Create with memory limit | 02 · Containers: run, create & lifecycle | 40 |
| 145 | docker create — Create a one-shot task | 02 · Containers: run, create & lifecycle | 40 |
| 146 | docker create — Create with restart policy | 02 · Containers: run, create & lifecycle | 40 |
| 147 | docker create — Create with labels | 02 · Containers: run, create & lifecycle | 41 |
| 148 | docker start — Start a stopped container | 02 · Containers: run, create & lifecycle | 41 |
| 149 | docker start — Start and attach output | 02 · Containers: run, create & lifecycle | 41 |
| 150 | docker start — Start with stdin attached | 02 · Containers: run, create & lifecycle | 41 |
| 151 | docker start — Start several containers | 02 · Containers: run, create & lifecycle | 42 |
| 152 | docker stop — Gracefully stop a container | 02 · Containers: run, create & lifecycle | 42 |
| 153 | docker stop — Set a 30-second timeout | 02 · Containers: run, create & lifecycle | 42 |
| 154 | docker stop — Stop a group of containers | 02 · Containers: run, create & lifecycle | 42 |
| 155 | docker stop — Use short timeout for a disposable task | 02 · Containers: run, create & lifecycle | 43 |
| 156 | docker restart — Restart a container | 02 · Containers: run, create & lifecycle | 43 |
| 157 | docker restart — Restart with longer timeout | 02 · Containers: run, create & lifecycle | 43 |
| 158 | docker restart — Restart multiple containers | 02 · Containers: run, create & lifecycle | 43 |
| 159 | docker kill — Force-stop a stuck container | 02 · Containers: run, create & lifecycle | 44 |
| 160 | docker kill — Send SIGTERM | 02 · Containers: run, create & lifecycle | 44 |
| 161 | docker kill — Send HUP for supported reload | 02 · Containers: run, create & lifecycle | 44 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 162 | docker rm — Remove a stopped container | 02 · Containers: run, create & lifecycle | 45 |
| 163 | docker rm — Force-remove a running container | 02 · Containers: run, create & lifecycle | 45 |
| 164 | docker rm — Remove associated anonymous volumes | 02 · Containers: run, create & lifecycle | 45 |
| 165 | docker rm — Remove several stopped containers | 02 · Containers: run, create & lifecycle | 46 |
| 166 | docker pause — Freeze one container | 02 · Containers: run, create & lifecycle | 46 |
| 167 | docker pause — Pause a group | 02 · Containers: run, create & lifecycle | 46 |
| 168 | docker unpause — Resume one container | 02 · Containers: run, create & lifecycle | 46 |
| 169 | docker unpause — Resume a group | 02 · Containers: run, create & lifecycle | 47 |
| 170 | docker rename — Rename a container | 02 · Containers: run, create & lifecycle | 47 |
| 171 | docker rename — Rename a stopped container | 02 · Containers: run, create & lifecycle | 47 |
| 172 | docker update — Change restart policy | 02 · Containers: run, create & lifecycle | 47 |
| 173 | docker update — Change memory limit | 02 · Containers: run, create & lifecycle | 48 |
| 174 | docker update — Change CPU quota | 02 · Containers: run, create & lifecycle | 48 |
| 175 | docker update — Set PID limit | 02 · Containers: run, create & lifecycle | 48 |
| 176 | docker update — Update CPU shares | 02 · Containers: run, create & lifecycle | 48 |
| 177 | docker update — Update memory reservation | 02 · Containers: run, create & lifecycle | 49 |
| 178 | docker update — Update stop timeout | 02 · Containers: run, create & lifecycle | 49 |
| 179 | docker wait — Wait for one job exit code | 02 · Containers: run, create & lifecycle | 49 |
| 180 | docker wait — Wait for multiple jobs | 02 · Containers: run, create & lifecycle | 49 |
| 181 | docker top — List container processes | 02 · Containers: run, create & lifecycle | 50 |
| 182 | docker top — Show PID, parent PID, and command | 02 · Containers: run, create & lifecycle | 50 |
| 183 | docker port — List all published mappings | 02 · Containers: run, create & lifecycle | 50 |
| 184 | docker port — Find a TCP mapping | 02 · Containers: run, create & lifecycle | 50 |
| 185 | docker stats — Monitor all running containers | 02 · Containers: run, create & lifecycle | 51 |
| 186 | docker stats — Take one snapshot | 02 · Containers: run, create & lifecycle | 51 |
| 187 | docker stats — Monitor one container | 02 · Containers: run, create & lifecycle | 51 |
| 188 | docker stats — Format selected fields | 02 · Containers: run, create & lifecycle | 51 |
| 189 | docker cp — Copy file from container to host | 02 · Containers: run, create & lifecycle | 52 |
| 190 | docker cp — Copy directory into container | 02 · Containers: run, create & lifecycle | 52 |
| 191 | docker cp — Copy with archive semantics | 02 · Containers: run, create & lifecycle | 52 |
| 192 | docker cp — Copy from stopped container | 02 · Containers: run, create & lifecycle | 52 |
| 193 | docker diff — List writable-layer changes | 02 · Containers: run, create & lifecycle | 53 |
| 194 | docker diff — Filter changes to configuration paths | 02 · Containers: run, create & lifecycle | 53 |
| 195 | docker attach — Attach to main process streams | 02 · Containers: run, create & lifecycle | 53 |
| 196 | docker attach — Customize detach keys | 02 · Containers: run, create & lifecycle | 53 |
| 197 | docker commit — Commit filesystem snapshot | 02 · Containers: run, create & lifecycle | 54 |
| 198 | docker commit — Add commit message | 02 · Containers: run, create & lifecycle | 54 |
| 199 | docker commit — Add author metadata | 02 · Containers: run, create & lifecycle | 54 |
| 200 | docker export — Export root filesystem to tar | 02 · Containers: run, create & lifecycle | 54 |
| 201 | docker export — Export and gzip through shell | 02 · Containers: run, create & lifecycle | 55 |
| 202 | docker import — Import rootfs archive | 02 · Containers: run, create & lifecycle | 55 |
| 203 | docker import — Import with default command | 02 · Containers: run, create & lifecycle | 55 |
| 204 | docker container prune — Remove all stopped containers | 02 · Containers: run, create & lifecycle | 55 |
| 205 | docker container prune — Prune stopped containers older than 24h | 02 · Containers: run, create & lifecycle | 56 |
| 206 | docker container prune — Skip prune confirmation | 02 · Containers: run, create & lifecycle | 56 |
| 207 | docker image ls — List local images | 03 · Images, registries & distribution | 56 |
| 208 | docker image ls — Include intermediate image records | 03 · Images, registries & distribution | 57 |
| 209 | docker image ls — Print image IDs only | 03 · Images, registries & distribution | 57 |
| 210 | docker image ls — Filter dangling images | 03 · Images, registries & distribution | 57 |
| 211 | docker image ls — Filter repository name | 03 · Images, registries & distribution | 57 |
| 212 | docker image ls — Show full image IDs | 03 · Images, registries & distribution | 58 |
| 213 | docker image ls — Format image listing | 03 · Images, registries & distribution | 58 |
| 214 | docker image ls — Filter images by label | 03 · Images, registries & distribution | 58 |
| 215 | docker image ls — Filter by reference pattern | 03 · Images, registries & distribution | 58 |
| 216 | docker image ls — List image digests | 03 · Images, registries & distribution | 59 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 217 | docker pull — Pull a tagged image | 03 · Images, registries & distribution | 59 |
| 218 | docker pull — Pull a specific platform | 03 · Images, registries & distribution | 59 |
| 219 | docker pull — Pull quietly | 03 · Images, registries & distribution | 59 |
| 220 | docker pull — Pull from private registry | 03 · Images, registries & distribution | 60 |
| 221 | docker pull — Pull by digest | 03 · Images, registries & distribution | 60 |
| 222 | docker image rm — Remove one tagged reference | 03 · Images, registries & distribution | 60 |
| 223 | docker image rm — Remove by image ID | 03 · Images, registries & distribution | 60 |
| 224 | docker image rm — Force removal of reference | 03 · Images, registries & distribution | 61 |
| 225 | docker image rm — Remove multiple references | 03 · Images, registries & distribution | 61 |
| 226 | docker image prune — Remove dangling images | 03 · Images, registries & distribution | 61 |
| 227 | docker image prune — Remove all unused images | 03 · Images, registries & distribution | 62 |
| 228 | docker image prune — Prune images older than a cutoff | 03 · Images, registries & distribution | 62 |
| 229 | docker image prune — Skip image-prune confirmation | 03 · Images, registries & distribution | 62 |
| 230 | docker tag — Tag a local image | 03 · Images, registries & distribution | 62 |
| 231 | docker tag — Prepare a registry tag | 03 · Images, registries & distribution | 63 |
| 232 | docker tag — Create a latest alias | 03 · Images, registries & distribution | 63 |
| 233 | docker push — Push a tagged image | 03 · Images, registries & distribution | 63 |
| 234 | docker push — Push a versioned release | 03 · Images, registries & distribution | 63 |
| 235 | docker push — Push quietly | 03 · Images, registries & distribution | 64 |
| 236 | docker save — Save one image to tar | 03 · Images, registries & distribution | 64 |
| 237 | docker save — Save multiple images | 03 · Images, registries & distribution | 64 |
| 238 | docker save — Save and gzip through shell | 03 · Images, registries & distribution | 64 |
| 239 | docker load — Load an image archive | 03 · Images, registries & distribution | 65 |
| 240 | docker load — Load a compressed archive | 03 · Images, registries & distribution | 65 |
| 241 | docker load — Load quietly | 03 · Images, registries & distribution | 65 |
| 242 | docker history — Show image layer history | 03 · Images, registries & distribution | 65 |
| 243 | docker history — Show full created-by commands | 03 · Images, registries & distribution | 66 |
| 244 | docker history — Format history output | 03 · Images, registries & distribution | 66 |
| 245 | docker search — Search public repositories | 03 · Images, registries & distribution | 66 |
| 246 | docker search — Limit search results | 03 · Images, registries & distribution | 66 |
| 247 | docker search — Filter automated builds | 03 · Images, registries & distribution | 67 |
| 248 | docker search — Filter official images | 03 · Images, registries & distribution | 67 |
| 249 | docker image inspect — Inspect image metadata | 03 · Images, registries & distribution | 67 |
| 250 | docker image inspect — Read image architecture | 03 · Images, registries & distribution | 67 |
| 251 | docker image inspect — Read image labels | 03 · Images, registries & distribution | 68 |
| 252 | docker image inspect — Read entrypoint and CMD | 03 · Images, registries & distribution | 68 |
| 253 | docker manifest — Inspect a remote manifest | 03 · Images, registries & distribution | 68 |
| 254 | docker manifest — Inspect verbose platform metadata | 03 · Images, registries & distribution | 68 |
| 255 | docker manifest — Create a manifest list | 03 · Images, registries & distribution | 69 |
| 256 | docker manifest — Push a manifest list | 03 · Images, registries & distribution | 69 |
| 257 | docker manifest — Annotate a platform entry | 03 · Images, registries & distribution | 69 |
| 258 | docker login — Login to Docker Hub | 03 · Images, registries & distribution | 70 |
| 259 | docker login — Login to a private registry | 03 · Images, registries & distribution | 70 |
| 260 | docker login — Use password from stdin | 03 · Images, registries & distribution | 70 |
| 261 | docker login — Specify username and prompt | 03 · Images, registries & distribution | 70 |
| 262 | docker logout — Logout from default registry | 03 · Images, registries & distribution | 71 |
| 263 | docker logout — Logout from private registry | 03 · Images, registries & distribution | 71 |
| 264 | docker build — Build and tag from current directory | 04 · Build, Dockerfile & BuildKit | 71 |
| 265 | docker build — Use alternate Dockerfile | 04 · Build, Dockerfile & BuildKit | 72 |
| 266 | docker build — Disable cache | 04 · Build, Dockerfile & BuildKit | 72 |
| 267 | docker build — Refresh base images | 04 · Build, Dockerfile & BuildKit | 72 |
| 268 | docker build — Pass build argument | 04 · Build, Dockerfile & BuildKit | 72 |
| 269 | docker build — Build named target stage | 04 · Build, Dockerfile & BuildKit | 73 |
| 270 | docker build — Plain progress output | 04 · Build, Dockerfile & BuildKit | 73 |
| 271 | docker build — Quiet build | 04 · Build, Dockerfile & BuildKit | 73 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 272 | docker build — Add image label | 04 · Build, Dockerfile & BuildKit | 73 |
| 273 | docker build — Build for target platform | 04 · Build, Dockerfile & BuildKit | 74 |
| 274 | docker build — Build from remote Git URL | 04 · Build, Dockerfile & BuildKit | 74 |
| 275 | docker build — Build with stdin Dockerfile | 04 · Build, Dockerfile & BuildKit | 74 |
| 276 | docker build — Set memory for build step | 04 · Build, Dockerfile & BuildKit | 74 |
| 277 | docker build — Build with explicit network mode | 04 · Build, Dockerfile & BuildKit | 75 |
| 278 | docker build — Write image ID to file | 04 · Build, Dockerfile & BuildKit | 75 |
| 279 | docker build — Write build metadata to file | 04 · Build, Dockerfile & BuildKit | 75 |
| 280 | docker build — Build with provenance attestation | 04 · Build, Dockerfile & BuildKit | 75 |
| 281 | docker build — Build with SBOM attestation | 04 · Build, Dockerfile & BuildKit | 76 |
| 282 | docker build — Load result into local image store | 04 · Build, Dockerfile & BuildKit | 76 |
| 283 | docker build — Push result to registry | 04 · Build, Dockerfile & BuildKit | 76 |
| 284 | docker build — Use secret mount during build | 04 · Build, Dockerfile & BuildKit | 76 |
| 285 | docker build — Forward SSH agent during build | 04 · Build, Dockerfile & BuildKit | 77 |
| 286 | docker build — Import registry cache | 04 · Build, Dockerfile & BuildKit | 77 |
| 287 | docker build — Export registry cache | 04 · Build, Dockerfile & BuildKit | 77 |
| 288 | docker build — Check Dockerfile without full build | 04 · Build, Dockerfile & BuildKit | 78 |
| 289 | docker build — Build only a named stage | 04 · Build, Dockerfile & BuildKit | 78 |
| 290 | docker build — Build from named additional context | 04 · Build, Dockerfile & BuildKit | 78 |
| 291 | docker build — Set build output directory | 04 · Build, Dockerfile & BuildKit | 78 |
| 292 | docker build — Set multiple output destinations | 04 · Build, Dockerfile & BuildKit | 79 |
| 293 | docker buildx — List builders | 04 · Build, Dockerfile & BuildKit | 79 |
| 294 | docker buildx — Inspect current builder | 04 · Build, Dockerfile & BuildKit | 79 |
| 295 | docker buildx — Bootstrap builder | 04 · Build, Dockerfile & BuildKit | 79 |
| 296 | docker buildx — Create named builder | 04 · Build, Dockerfile & BuildKit | 80 |
| 297 | docker buildx — Create and select builder | 04 · Build, Dockerfile & BuildKit | 80 |
| 298 | docker buildx — Select builder | 04 · Build, Dockerfile & BuildKit | 80 |
| 299 | docker buildx — Remove builder | 04 · Build, Dockerfile & BuildKit | 80 |
| 300 | docker buildx — Show Buildx version | 04 · Build, Dockerfile & BuildKit | 81 |
| 301 | docker buildx — Inspect registry image platforms | 04 · Build, Dockerfile & BuildKit | 81 |
| 302 | docker buildx — Build multi-platform image | 04 · Build, Dockerfile & BuildKit | 81 |
| 303 | docker buildx — Prune build cache | 04 · Build, Dockerfile & BuildKit | 82 |
| 304 | docker buildx — Inspect build cache usage | 04 · Build, Dockerfile & BuildKit | 82 |
| 305 | docker buildx — Print Bake configuration | 04 · Build, Dockerfile & BuildKit | 82 |
| 306 | docker buildx — List Bake targets | 04 · Build, Dockerfile & BuildKit | 82 |
| 307 | docker buildx — Run Bake targets | 04 · Build, Dockerfile & BuildKit | 83 |
| 308 | docker buildx — Create builder from context | 04 · Build, Dockerfile & BuildKit | 83 |
| 309 | docker buildx — Set builder platforms | 04 · Build, Dockerfile & BuildKit | 83 |
| 310 | docker buildx — Append builder node | 04 · Build, Dockerfile & BuildKit | 83 |
| 311 | docker buildx — Stop a builder | 04 · Build, Dockerfile & BuildKit | 84 |
| 312 | docker buildx — Inspect raw registry manifest | 04 · Build, Dockerfile & BuildKit | 84 |
| 313 | docker network — List networks | 06 · Networking | 84 |
| 314 | docker network — Create a user-defined bridge | 06 · Networking | 85 |
| 315 | docker network — Create bridge with subnet | 06 · Networking | 85 |
| 316 | docker network — Set gateway | 06 · Networking | 85 |
| 317 | docker network — Create internal network | 06 · Networking | 85 |
| 318 | docker network — Create labeled network | 06 · Networking | 86 |
| 319 | docker network — Inspect network | 06 · Networking | 86 |
| 320 | docker network — Extract attached containers | 06 · Networking | 86 |
| 321 | docker network — Connect a container | 06 · Networking | 86 |
| 322 | docker network — Connect with DNS alias | 06 · Networking | 87 |
| 323 | docker network — Connect with static IP | 06 · Networking | 87 |
| 324 | docker network — Disconnect a container | 06 · Networking | 87 |
| 325 | docker network — Force disconnect endpoint | 06 · Networking | 87 |
| 326 | docker network — Remove a network | 06 · Networking | 88 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 327 | docker network — Prune unused networks | 06 · Networking | 88 |
| 328 | docker network — Prune networks by age | 06 · Networking | 88 |
| 329 | docker network — Filter networks by driver | 06 · Networking | 88 |
| 330 | docker network — Filter networks by label | 06 · Networking | 89 |
| 331 | docker network — Create IPv6 network | 06 · Networking | 89 |
| 332 | docker network — Create attachable overlay | 06 · Networking | 89 |
| 333 | docker network — Create encrypted overlay | 06 · Networking | 89 |
| 334 | docker network — List network IDs only | 06 · Networking | 90 |
| 335 | docker network — Format network inventory | 06 · Networking | 90 |
| 336 | docker network — Inspect multiple networks | 06 · Networking | 90 |
| 337 | docker volume — Create named volume | 06 · Networking | 90 |
| 338 | docker volume — List volumes | 06 · Networking | 91 |
| 339 | docker volume — Inspect volume | 06 · Networking | 91 |
| 340 | docker volume — Extract mountpoint | 06 · Networking | 91 |
| 341 | docker volume — Create volume with label | 06 · Networking | 91 |
| 342 | docker volume — Create using local driver | 06 · Networking | 92 |
| 343 | docker volume — Mount volume read-only | 06 · Networking | 92 |
| 344 | docker volume — Bind mount source directory | 06 · Networking | 92 |
| 345 | docker volume — Bind mount read-only | 06 · Networking | 92 |
| 346 | docker volume — Use tmpfs with mount options | 06 · Networking | 93 |
| 347 | docker volume — Use tmpfs mount syntax | 06 · Networking | 93 |
| 348 | docker volume — Remove a volume | 06 · Networking | 93 |
| 349 | docker volume — Prune unused volumes | 06 · Networking | 93 |
| 350 | docker volume — Prune all unused volumes | 06 · Networking | 94 |
| 351 | docker volume — Prune with force | 06 · Networking | 94 |
| 352 | docker volume — Create driver-specific bind volume | 06 · Networking | 94 |
| 353 | docker volume — Create multiple labels | 06 · Networking | 95 |
| 354 | docker volume — Filter volumes by driver | 06 · Networking | 95 |
| 355 | docker volume — Filter volumes by name | 06 · Networking | 95 |
| 356 | docker compose — Show Compose help | 08 · Docker Compose | 96 |
| 357 | docker compose — Print Compose version | 08 · Docker Compose | 96 |
| 358 | docker compose — Use alternate Compose file | 08 · Docker Compose | 96 |
| 359 | docker compose — Merge base and override files | 08 · Docker Compose | 96 |
| 360 | docker compose — Set project name | 08 · Docker Compose | 97 |
| 361 | docker compose — Use env file for interpolation | 08 · Docker Compose | 97 |
| 362 | docker compose — Enable profile | 08 · Docker Compose | 97 |
| 363 | docker compose — Validate and render config | 08 · Docker Compose | 97 |
| 364 | docker compose — List service names | 08 · Docker Compose | 98 |
| 365 | docker compose — List image references | 08 · Docker Compose | 98 |
| 366 | docker compose — List declared profiles | 08 · Docker Compose | 98 |
| 367 | docker compose — Render JSON config | 08 · Docker Compose | 98 |
| 368 | docker compose — Validate quietly | 08 · Docker Compose | 99 |
| 369 | docker compose — List projects | 08 · Docker Compose | 99 |
| 370 | docker compose — List stopped projects too | 08 · Docker Compose | 99 |
| 371 | docker compose — List project containers | 08 · Docker Compose | 99 |
| 372 | docker compose — Include stopped service containers | 08 · Docker Compose | 100 |
| 373 | docker compose — Start services detached | 08 · Docker Compose | 100 |
| 374 | docker compose — Build before startup | 08 · Docker Compose | 100 |
| 375 | docker compose — Force recreation | 08 · Docker Compose | 100 |
| 376 | docker compose — Skip dependency startup | 08 · Docker Compose | 101 |
| 377 | docker compose — Start selected services | 08 · Docker Compose | 101 |
| 378 | docker compose — Wait for running/healthy | 08 · Docker Compose | 101 |
| 379 | docker compose — Wait with timeout | 08 · Docker Compose | 101 |
| 380 | docker compose — Abort when a container exits | 08 · Docker Compose | 102 |
| 381 | docker compose — Build all services | 08 · Docker Compose | 102 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 382 | docker compose — Build one service | 08 · Docker Compose | 102 |
| 383 | docker compose — Build without cache | 08 · Docker Compose | 102 |
| 384 | docker compose — Pull base images while building | 08 · Docker Compose | 103 |
| 385 | docker compose — Use plain build progress | 08 · Docker Compose | 103 |
| 386 | docker compose — Pull all service images | 08 · Docker Compose | 103 |
| 387 | docker compose — Pull one service image | 08 · Docker Compose | 103 |
| 388 | docker compose — Start existing containers | 08 · Docker Compose | 104 |
| 389 | docker compose — Stop services but retain containers | 08 · Docker Compose | 104 |
| 390 | docker compose — Restart a service | 08 · Docker Compose | 104 |
| 391 | docker compose — Tear down project | 08 · Docker Compose | 104 |
| 392 | docker compose — Remove orphan containers | 08 · Docker Compose | 105 |
| 393 | docker compose — Remove project volumes | 08 · Docker Compose | 105 |
| 394 | docker compose — Remove local images on teardown | 08 · Docker Compose | 105 |
| 395 | docker compose — Follow service logs | 08 · Docker Compose | 105 |
| 396 | docker compose — Show last 100 log lines | 08 · Docker Compose | 106 |
| 397 | docker compose — Add timestamps to logs | 08 · Docker Compose | 106 |
| 398 | docker compose — Show logs since duration | 08 · Docker Compose | 106 |
| 399 | docker compose — Execute interactive command | 08 · Docker Compose | 106 |
| 400 | docker compose — Execute without TTY | 08 · Docker Compose | 107 |
| 401 | docker compose — Execute as non-root user | 08 · Docker Compose | 107 |
| 402 | docker compose — Execute in working directory | 08 · Docker Compose | 107 |
| 403 | docker compose — Run one-off service command | 08 · Docker Compose | 107 |
| 404 | docker compose — Run one-off with service ports | 08 · Docker Compose | 108 |
| 405 | docker compose — Override one-off entrypoint | 08 · Docker Compose | 108 |
| 406 | docker compose — Scale a service | 08 · Docker Compose | 108 |
| 407 | docker compose — Copy file from service | 08 · Docker Compose | 108 |
| 408 | docker compose — Copy directory to service | 08 · Docker Compose | 109 |
| 409 | docker compose — Copy with archive semantics | 08 · Docker Compose | 109 |
| 410 | docker compose — List service images | 08 · Docker Compose | 109 |
| 411 | docker compose — Show service processes | 08 · Docker Compose | 109 |
| 412 | docker compose — Show one service process list | 08 · Docker Compose | 110 |
| 413 | docker compose — Monitor service stats | 08 · Docker Compose | 110 |
| 414 | docker compose — Take stats snapshot | 08 · Docker Compose | 110 |
| 415 | docker compose — Find published service port | 08 · Docker Compose | 110 |
| 416 | docker compose — Kill all Compose containers | 08 · Docker Compose | 111 |
| 417 | docker compose — Send signal to one service | 08 · Docker Compose | 111 |
| 418 | docker compose — Pause all services | 08 · Docker Compose | 111 |
| 419 | docker compose — Pause one service | 08 · Docker Compose | 111 |
| 420 | docker compose — Resume all services | 08 · Docker Compose | 112 |
| 421 | docker compose — Resume one service | 08 · Docker Compose | 112 |
| 422 | docker compose — Watch source changes | 08 · Docker Compose | 112 |
| 423 | docker compose — Start services with watch | 08 · Docker Compose | 112 |
| 424 | docker compose — Use dry-run preview | 08 · Docker Compose | 113 |
| 425 | docker compose — Set parallelism | 08 · Docker Compose | 113 |
| 426 | docker context — List contexts | 09 · Contexts & remote Engines | 113 |
| 427 | docker context — Show current context | 09 · Contexts & remote Engines | 114 |
| 428 | docker context — Inspect context endpoint | 09 · Contexts & remote Engines | 114 |
| 429 | docker context — Create SSH context | 09 · Contexts & remote Engines | 114 |
| 430 | docker context — Select context | 09 · Contexts & remote Engines | 114 |
| 431 | docker context — Use context for one command | 09 · Contexts & remote Engines | 115 |
| 432 | docker context — Export context | 09 · Contexts & remote Engines | 115 |
| 433 | docker context — Import context | 09 · Contexts & remote Engines | 115 |
| 434 | docker context — Remove context definition | 09 · Contexts & remote Engines | 115 |
| 435 | docker context — Update context description | 09 · Contexts & remote Engines | 116 |
| 436 | docker context — Inspect remote daemon | 09 · Contexts & remote Engines | 116 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 437 | docker context — Use SSH endpoint directly | 09 · Contexts & remote Engines | 116 |
| 438 | docker swarm — Initialize a swarm | 10 · Swarm, services, nodes & stacks | 117 |
| 439 | docker swarm — Initialize with advertised address | 10 · Swarm, services, nodes & stacks | 117 |
| 440 | docker swarm — Initialize with autolock | 10 · Swarm, services, nodes & stacks | 117 |
| 441 | docker swarm — Leave swarm | 10 · Swarm, services, nodes & stacks | 118 |
| 442 | docker swarm — Force manager to leave | 10 · Swarm, services, nodes & stacks | 118 |
| 443 | docker swarm — Show worker join token | 10 · Swarm, services, nodes & stacks | 118 |
| 444 | docker swarm — Show manager join token | 10 · Swarm, services, nodes & stacks | 119 |
| 445 | docker swarm — Rotate worker token | 10 · Swarm, services, nodes & stacks | 119 |
| 446 | docker swarm — Display swarm CA | 10 · Swarm, services, nodes & stacks | 119 |
| 447 | docker swarm — Rotate swarm CA | 10 · Swarm, services, nodes & stacks | 120 |
| 448 | docker swarm — Unlock a manager | 10 · Swarm, services, nodes & stacks | 120 |
| 449 | docker swarm — Display unlock key | 10 · Swarm, services, nodes & stacks | 120 |
| 450 | docker swarm — Rotate unlock key | 10 · Swarm, services, nodes & stacks | 121 |
| 451 | docker swarm — Set task history limit | 10 · Swarm, services, nodes & stacks | 121 |
| 452 | docker swarm — Tune dispatcher heartbeat | 10 · Swarm, services, nodes & stacks | 121 |
| 453 | docker swarm — Set default address pool | 10 · Swarm, services, nodes & stacks | 122 |
| 454 | docker swarm — Set data-path address | 10 · Swarm, services, nodes & stacks | 122 |
| 455 | docker swarm — Set manager listen address | 10 · Swarm, services, nodes & stacks | 122 |
| 456 | docker swarm — Join as worker | 10 · Swarm, services, nodes & stacks | 123 |
| 457 | docker swarm — Join as manager | 10 · Swarm, services, nodes & stacks | 123 |
| 458 | docker service — List services | 10 · Swarm, services, nodes & stacks | 123 |
| 459 | docker service — Create replicated web service | 10 · Swarm, services, nodes & stacks | 123 |
| 460 | docker service — Create global agent | 10 · Swarm, services, nodes & stacks | 124 |
| 461 | docker service — Set service environment | 10 · Swarm, services, nodes & stacks | 124 |
| 462 | docker service — Mount volume in service | 10 · Swarm, services, nodes & stacks | 124 |
| 463 | docker service — Inspect service JSON | 10 · Swarm, services, nodes & stacks | 124 |
| 464 | docker service — Inspect service pretty output | 10 · Swarm, services, nodes & stacks | 125 |
| 465 | docker service — List service tasks | 10 · Swarm, services, nodes & stacks | 125 |
| 466 | docker service — Show full task errors | 10 · Swarm, services, nodes & stacks | 125 |
| 467 | docker service — Scale service to five replicas | 10 · Swarm, services, nodes & stacks | 125 |
| 468 | docker service — Update service image | 10 · Swarm, services, nodes & stacks | 126 |
| 469 | docker service — Update replica count | 10 · Swarm, services, nodes & stacks | 126 |
| 470 | docker service — Add environment variable | 10 · Swarm, services, nodes & stacks | 126 |
| 471 | docker service — Remove environment variable | 10 · Swarm, services, nodes & stacks | 126 |
| 472 | docker service — Remove published port | 10 · Swarm, services, nodes & stacks | 127 |
| 473 | docker service — Force rolling update | 10 · Swarm, services, nodes & stacks | 127 |
| 474 | docker service — Rollback service | 10 · Swarm, services, nodes & stacks | 127 |
| 475 | docker service — Remove service | 10 · Swarm, services, nodes & stacks | 127 |
| 476 | docker service — Follow service logs | 10 · Swarm, services, nodes & stacks | 128 |
| 477 | docker service — Show recent service logs | 10 · Swarm, services, nodes & stacks | 128 |
| 478 | docker service — Constrain placement by node label | 10 · Swarm, services, nodes & stacks | 128 |
| 479 | docker service — Set CPU/memory limits | 10 · Swarm, services, nodes & stacks | 128 |
| 480 | docker service — Set restart condition | 10 · Swarm, services, nodes & stacks | 129 |
| 481 | docker service — Set DNS round-robin endpoint mode | 10 · Swarm, services, nodes & stacks | 129 |
| 482 | docker service — Set update parallelism | 10 · Swarm, services, nodes & stacks | 129 |
| 483 | docker service — Set update delay | 10 · Swarm, services, nodes & stacks | 129 |
| 484 | docker service — Configure rollback-on-failure | 10 · Swarm, services, nodes & stacks | 130 |
| 485 | docker service — Add published port | 10 · Swarm, services, nodes & stacks | 130 |
| 486 | docker service — Add secret to service | 10 · Swarm, services, nodes & stacks | 130 |
| 487 | docker service — Remove secret from service | 10 · Swarm, services, nodes & stacks | 130 |
| 488 | docker service — Add config to service | 10 · Swarm, services, nodes & stacks | 131 |
| 489 | docker service — Add placement preference | 10 · Swarm, services, nodes & stacks | 131 |
| 490 | docker node — List nodes | 10 · Swarm, services, nodes & stacks | 131 |
| 491 | docker node — Inspect node | 10 · Swarm, services, nodes & stacks | 131 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 492 | docker node — Pretty node summary | 10 · Swarm, services, nodes & stacks | 132 |
| 493 | docker node — List node tasks | 10 · Swarm, services, nodes & stacks | 132 |
| 494 | docker node — Drain node | 10 · Swarm, services, nodes & stacks | 132 |
| 495 | docker node — Reactivate node | 10 · Swarm, services, nodes & stacks | 132 |
| 496 | docker node — Pause scheduling | 10 · Swarm, services, nodes & stacks | 133 |
| 497 | docker node — Add node label | 10 · Swarm, services, nodes & stacks | 133 |
| 498 | docker node — Remove node label | 10 · Swarm, services, nodes & stacks | 133 |
| 499 | docker node — Promote worker to manager | 10 · Swarm, services, nodes & stacks | 133 |
| 500 | docker node — Demote manager to worker | 10 · Swarm, services, nodes & stacks | 134 |
| 501 | docker node — Remove node record | 10 · Swarm, services, nodes & stacks | 134 |
| 502 | docker node — Force-remove down node | 10 · Swarm, services, nodes & stacks | 134 |
| 503 | docker node — Filter node list by role | 10 · Swarm, services, nodes & stacks | 134 |
| 504 | docker node — Filter node list by availability | 10 · Swarm, services, nodes & stacks | 135 |
| 505 | docker stack — List stacks | 10 · Swarm, services, nodes & stacks | 135 |
| 506 | docker stack — Deploy stack | 10 · Swarm, services, nodes & stacks | 135 |
| 507 | docker stack — Deploy with override file | 10 · Swarm, services, nodes & stacks | 135 |
| 508 | docker stack — List stack services | 10 · Swarm, services, nodes & stacks | 136 |
| 509 | docker stack — List stack tasks | 10 · Swarm, services, nodes & stacks | 136 |
| 510 | docker stack — Remove a stack | 10 · Swarm, services, nodes & stacks | 136 |
| 511 | docker stack — Render stack configuration | 10 · Swarm, services, nodes & stacks | 136 |
| 512 | docker stack — Deploy with registry auth | 10 · Swarm, services, nodes & stacks | 137 |
| 513 | docker secret — Create secret from file | 10 · Swarm, services, nodes & stacks | 137 |
| 514 | docker secret — Create secret from stdin | 10 · Swarm, services, nodes & stacks | 137 |
| 515 | docker secret — List secrets | 10 · Swarm, services, nodes & stacks | 137 |
| 516 | docker secret — Inspect secret metadata | 10 · Swarm, services, nodes & stacks | 138 |
| 517 | docker secret — Remove unused secret | 10 · Swarm, services, nodes & stacks | 138 |
| 518 | docker secret — Remove several unused secrets | 10 · Swarm, services, nodes & stacks | 138 |
| 519 | docker config — Create config from file | 10 · Swarm, services, nodes & stacks | 138 |
| 520 | docker config — Create config from stdin | 10 · Swarm, services, nodes & stacks | 139 |
| 521 | docker config — List configs | 10 · Swarm, services, nodes & stacks | 139 |
| 522 | docker config — Inspect config metadata | 10 · Swarm, services, nodes & stacks | 139 |
| 523 | docker config — Remove unused config | 10 · Swarm, services, nodes & stacks | 139 |
| 524 | docker config — Remove several configs | 10 · Swarm, services, nodes & stacks | 140 |
| 525 | docker system prune — Prune unused resources | 11 · Maintenance, security, plugins & advanced tooling | 140 |
| 526 | docker system prune — Include all unused images | 11 · Maintenance, security, plugins & advanced tooling | 140 |
| 527 | docker system prune — Include unused volumes | 11 · Maintenance, security, plugins & advanced tooling | 141 |
| 528 | docker system prune — Skip confirmation | 11 · Maintenance, security, plugins & advanced tooling | 141 |
| 529 | docker system prune — Prune with age filter | 11 · Maintenance, security, plugins & advanced tooling | 141 |
| 530 | docker builder — Show builder commands | 11 · Maintenance, security, plugins & advanced tooling | 141 |
| 531 | docker builder — Prune builder cache | 11 · Maintenance, security, plugins & advanced tooling | 142 |
| 532 | docker builder — Prune all builder cache | 11 · Maintenance, security, plugins & advanced tooling | 142 |
| 533 | docker builder — Skip cache prune prompt | 11 · Maintenance, security, plugins & advanced tooling | 142 |
| 534 | docker plugin — List installed plugins | 11 · Maintenance, security, plugins & advanced tooling | 143 |
| 535 | docker plugin — Inspect plugin configuration | 11 · Maintenance, security, plugins & advanced tooling | 143 |
| 536 | docker plugin — Install plugin | 11 · Maintenance, security, plugins & advanced tooling | 143 |
| 537 | docker plugin — Install and grant requested permissions | 11 · Maintenance, security, plugins & advanced tooling | 144 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 538 | docker plugin — Enable plugin | 11 · Maintenance, security, plugins & advanced tooling | 144 |
| 539 | docker plugin — Disable plugin | 11 · Maintenance, security, plugins & advanced tooling | 144 |
| 540 | docker plugin — Force-disable plugin | 11 · Maintenance, security, plugins & advanced tooling | 145 |
| 541 | docker plugin — Remove plugin | 11 · Maintenance, security, plugins & advanced tooling | 145 |
| 542 | docker plugin — Inspect plugin settings as JSON | 11 · Maintenance, security, plugins & advanced tooling | 145 |
| 543 | docker scout — Show Scout help | 11 · Maintenance, security, plugins & advanced tooling | 145 |
| 544 | docker scout — Quick image posture summary | 11 · Maintenance, security, plugins & advanced tooling | 146 |
| 545 | docker scout — List image CVEs | 11 · Maintenance, security, plugins & advanced tooling | 146 |
| 546 | docker scout — Compare image versions | 11 · Maintenance, security, plugins & advanced tooling | 146 |
| 547 | docker scout — Show remediation recommendations | 11 · Maintenance, security, plugins & advanced tooling | 146 |
| 548 | docker scout — Quickview remote image | 11 · Maintenance, security, plugins & advanced tooling | 147 |
| 549 | docker scout — Scan newly built image | 11 · Maintenance, security, plugins & advanced tooling | 147 |
| 550 | docker trust — Show trust help | 11 · Maintenance, security, plugins & advanced tooling | 147 |
| 551 | docker trust — Inspect trust metadata | 11 · Maintenance, security, plugins & advanced tooling | 147 |
| 552 | docker trust — Sign an image tag | 11 · Maintenance, security, plugins & advanced tooling | 148 |
| 553 | docker trust — Revoke a trust signature | 11 · Maintenance, security, plugins & advanced tooling | 148 |
| 554 | docker checkpoint — Check checkpoint availability | 11 · Maintenance, security, plugins & advanced tooling | 148 |
| 555 | docker checkpoint — Create container checkpoint | 11 · Maintenance, security, plugins & advanced tooling | 148 |
| 556 | docker checkpoint — List checkpoints | 11 · Maintenance, security, plugins & advanced tooling | 149 |
| 557 | docker checkpoint — Remove checkpoint | 11 · Maintenance, security, plugins & advanced tooling | 149 |
| 558 | docker context ls — Format context inventory | 11 · Maintenance, security, plugins & advanced tooling | 149 |
| 559 | docker context ls — Filter context names | 11 · Maintenance, security, plugins & advanced tooling | 149 |
| 560 | docker container ls — List latest container record | 11 · Maintenance, security, plugins & advanced tooling | 150 |
| 561 | docker container ls — Filter by ancestor image | 11 · Maintenance, security, plugins & advanced tooling | 150 |
| 562 | docker container inspect — Inspect status and exit code | 11 · Maintenance, security, plugins & advanced tooling | 150 |
| 563 | docker container inspect — Inspect configured command | 11 · Maintenance, security, plugins & advanced tooling | 150 |
| 564 | docker container logs — Read recent logs using namespace | 11 · Maintenance, security, plugins & advanced tooling | 151 |
| 565 | docker container exec — Execute command using namespace | 11 · Maintenance, security, plugins & advanced tooling | 151 |
| 566 | docker container rm — Remove via namespaced command | 11 · Maintenance, security, plugins & advanced tooling | 151 |
| 567 | docker container start — Start via namespaced command | 11 · Maintenance, security, plugins & advanced tooling | 151 |
| 568 | docker container stop — Stop with explicit timeout | 11 · Maintenance, security, plugins & advanced tooling | 152 |
| 569 | docker container restart — Restart via namespace | 11 · Maintenance, security, plugins & advanced tooling | 152 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 570 | docker container pause — Pause via namespace | 11 · Maintenance, security, plugins & advanced tooling | 152 |
| 571 | docker container unpause — Resume via namespace | 11 · Maintenance, security, plugins & advanced tooling | 152 |
| 572 | docker container update — Update CPU limit via namespace | 11 · Maintenance, security, plugins & advanced tooling | 153 |
| 573 | docker container wait — Wait via namespace | 11 · Maintenance, security, plugins & advanced tooling | 153 |
| 574 | docker image pull — Pull through image namespace | 11 · Maintenance, security, plugins & advanced tooling | 153 |
| 575 | docker image push — Push through image namespace | 11 · Maintenance, security, plugins & advanced tooling | 153 |
| 576 | docker image save — Save through image namespace | 11 · Maintenance, security, plugins & advanced tooling | 154 |
| 577 | docker image load — Load through image namespace | 11 · Maintenance, security, plugins & advanced tooling | 154 |
| 578 | docker image tag — Tag through image namespace | 11 · Maintenance, security, plugins & advanced tooling | 154 |
| 579 | docker image history — Review full image history | 11 · Maintenance, security, plugins & advanced tooling | 154 |
| 580 | docker volume inspect — Inspect multiple volumes | 11 · Maintenance, security, plugins & advanced tooling | 155 |
| 581 | docker network inspect — Inspect multiple networks | 11 · Maintenance, security, plugins & advanced tooling | 155 |
| 582 | docker compose rm — Remove stopped Compose containers | 11 · Maintenance, security, plugins & advanced tooling | 155 |
| 583 | docker compose rm — Remove one stopped service container | 11 · Maintenance, security, plugins & advanced tooling | 155 |
| 584 | docker compose rm — Force removal without prompt | 11 · Maintenance, security, plugins & advanced tooling | 156 |
| 585 | docker compose kill — Kill all Compose containers | 11 · Maintenance, security, plugins & advanced tooling | 156 |
| 586 | docker compose kill — Send SIGTERM to one service | 11 · Maintenance, security, plugins & advanced tooling | 156 |
| 587 | docker compose pause — Pause all services | 11 · Maintenance, security, plugins & advanced tooling | 156 |
| 588 | docker compose pause — Pause one service | 11 · Maintenance, security, plugins & advanced tooling | 157 |
| 589 | docker compose unpause — Resume all services | 11 · Maintenance, security, plugins & advanced tooling | 157 |
| 590 | docker compose unpause — Resume one service | 11 · Maintenance, security, plugins & advanced tooling | 157 |
| 591 | docker compose port — Find published service port | 11 · Maintenance, security, plugins & advanced tooling | 157 |
| 592 | docker compose port — Find DNS UDP mapping | 11 · Maintenance, security, plugins & advanced tooling | 158 |
| 593 | docker compose down — Remove project containers and networks | 11 · Maintenance, security, plugins & advanced tooling | 158 |
| 594 | docker compose down — Remove project images too | 11 · Maintenance, security, plugins & advanced tooling | 158 |
| 595 | docker compose exec — Run as non-root user | 11 · Maintenance, security, plugins & advanced tooling | 158 |
| 596 | docker compose exec — Set working directory | 11 · Maintenance, security, plugins & advanced tooling | 159 |
| 597 | docker compose build — Pull base image while building | 11 · Maintenance, security, plugins & advanced tooling | 159 |
| 598 | docker compose logs — Show logs until cutoff | 11 · Maintenance, security, plugins & advanced tooling | 159 |
| 599 | docker compose cp — Copy directory from host | 11 · Maintenance, security, plugins & advanced tooling | 159 |
| 600 | docker compose cp — Copy with archive semantics | 11 · Maintenance, security, plugins & advanced tooling | 160 |
| 601 | docker compose watch — Watch one development service | 11 · Maintenance, security, plugins & advanced tooling | 160 |

| No. | Command / variation | Section | Page |
|---|---|---|---|
| 602 | docker compose watch — Start services with watch | 11 · Maintenance, security, plugins & advanced tooling | 160 |
| 603 | Dockerfile: FROM — FROM | 12 · Dockerfile syntax (not CLI commands) | 161 |
| 604 | Dockerfile: RUN — RUN | 12 · Dockerfile syntax (not CLI commands) | 161 |
| 605 | Dockerfile: CMD — CMD | 12 · Dockerfile syntax (not CLI commands) | 161 |
| 606 | Dockerfile: ENTRYPOINT — ENTRYPOINT | 12 · Dockerfile syntax (not CLI commands) | 161 |
| 607 | Dockerfile: COPY — COPY | 12 · Dockerfile syntax (not CLI commands) | 162 |
| 608 | Dockerfile: ADD — ADD | 12 · Dockerfile syntax (not CLI commands) | 162 |
| 609 | Dockerfile: WORKDIR — WORKDIR | 12 · Dockerfile syntax (not CLI commands) | 162 |
| 610 | Dockerfile: ENV — ENV | 12 · Dockerfile syntax (not CLI commands) | 162 |
| 611 | Dockerfile: ARG — ARG | 12 · Dockerfile syntax (not CLI commands) | 163 |
| 612 | Dockerfile: EXPOSE — EXPOSE | 12 · Dockerfile syntax (not CLI commands) | 163 |
| 613 | Dockerfile: USER — USER | 12 · Dockerfile syntax (not CLI commands) | 163 |
| 614 | Dockerfile: VOLUME — VOLUME | 12 · Dockerfile syntax (not CLI commands) | 163 |
| 615 | Dockerfile: HEALTHCHECK — HEALTHCHECK | 12 · Dockerfile syntax (not CLI commands) | 164 |
| 616 | Dockerfile: STOPSIGNAL — STOPSIGNAL | 12 · Dockerfile syntax (not CLI commands) | 164 |
| 617 | Dockerfile: SHELL — SHELL | 12 · Dockerfile syntax (not CLI commands) | 164 |
| 618 | Dockerfile: LABEL — LABEL | 12 · Dockerfile syntax (not CLI commands) | 164 |
| 619 | Dockerfile: ONBUILD — ONBUILD | 12 · Dockerfile syntax (not CLI commands) | 165 |
| 620 | Dockerfile: Multi-stage build — Multi-stage build | 12 · Dockerfile syntax (not CLI commands) | 165 |
| 621 | Dockerfile: BuildKit secret mount — BuildKit secret mount | 12 · Dockerfile syntax (not CLI commands) | 165 |
| 622 | Dockerfile: BuildKit cache mount — BuildKit cache mount | 12 · Dockerfile syntax (not CLI commands) | 165 |
| 623 | Workflow: Verify a fresh install — Verify a fresh install | 13 · Practical combined workflows (CLI recipes) | 166 |
| 624 | Workflow: Run and troubleshoot a web server — Run and troubleshoot a web server | 13 · Practical combined workflows (CLI recipes) | 166 |
| 625 | Workflow: Build, tag, and publish — Build, tag, and publish | 13 · Practical combined workflows (CLI recipes) | 166 |
| 626 | Workflow: Create a network and persistent volume — Create a network and persistent volume | 13 · Practical combined workflows (CLI recipes) | 167 |
| 627 | Workflow: Diagnose a failed container — Diagnose a failed container | 13 · Practical combined workflows (CLI recipes) | 167 |
| 628 | Workflow: Compose deployment and teardown — Compose deployment and teardown | 13 · Practical combined workflows (CLI recipes) | 167 |
| 629 | Workflow: Review before cleanup — Review before cleanup | 13 · Practical combined workflows (CLI recipes) | 168 |
| 630 | Workflow: Swarm deployment verification — Swarm deployment verification | 13 · Practical combined workflows (CLI recipes) | 168 |

# Alphabetical command-family index

**docker --help**
011 (p.6), 012 (p.7), 013 (p.7), 014 (p.7), 015 (p.7), 016 (p.8)

**docker attach**
195 (p.53), 196 (p.53)

**docker build**
264 (p.71), 265 (p.72), 266 (p.72), 267 (p.72), 268 (p.72), 269 (p.73), 270 (p.73), 271 (p.73), 272 (p.73), 273 (p.74), 274 (p.74), 275 (p.74), 276 (p.74), 277 (p.75), 278 (p.75), 279 (p.75), 280 (p.75), 281 (p.76), 282 (p.76), 283 (p.76), 284 (p.76), 285 (p.77), 286 (p.77), 287 (p.77), 288 (p.78), 289 (p.78), 290 (p.78), 291 (p.78), 292 (p.79)

**docker builder**
530 (p.141), 531 (p.142), 532 (p.142), 533 (p.142)

**docker buildx**
293 (p.79), 294 (p.79), 295 (p.79), 296 (p.80), 297 (p.80), 298 (p.80), 299 (p.80), 300 (p.81), 301 (p.81), 302 (p.81), 303 (p.82), 304 (p.82), 305 (p.82), 306 (p.82), 307 (p.83), 308 (p.83), 309 (p.83), 310 (p.83), 311 (p.84), 312 (p.84)

**docker checkpoint**
554 (p.148), 555 (p.148), 556 (p.149), 557 (p.149)

**docker commit**
197 (p.54), 198 (p.54), 199 (p.54)

**docker compose**
356 (p.96), 357 (p.96), 358 (p.96), 359 (p.96), 360 (p.97), 361 (p.97), 362 (p.97), 363 (p.97), 364 (p.98), 365 (p.98), 366 (p.98), 367 (p.98), 368 (p.99), 369 (p.99), 370 (p.99), 371 (p.99), 372 (p.100), 373 (p.100), 374 (p.100), 375 (p.100), 376 (p.101), 377 (p.101), 378 (p.101), 379 (p.101), 380 (p.102), 381 (p.102), 382 (p.102), 383 (p.102), 384 (p.103), 385 (p.103), 386 (p.103), 387 (p.103), 388 (p.104), 389 (p.104), 390 (p.104), 391 (p.104), 392 (p.105), 393 (p.105), 394 (p.105), 395 (p.105), 396 (p.106), 397 (p.106), 398 (p.106), 399 (p.106), 400 (p.107), 401 (p.107), 402 (p.107), 403 (p.107), 404 (p.108), 405 (p.108), 406 (p.108), 407 (p.108), 408 (p.109), 409 (p.109), 410 (p.109), 411 (p.109), 412 (p.110), 413 (p.110), 414 (p.110), 415 (p.110), 416 (p.111), 417 (p.111), 418 (p.111), 419 (p.111), 420 (p.112), 421 (p.112), 422 (p.112), 423 (p.112), 424 (p.113), 425 (p.113)

**docker compose build**
597 (p.159)

**docker compose cp**
599 (p.159), 600 (p.160)

**docker compose down**
593 (p.158), 594 (p.158)

**docker compose exec**
595 (p.158), 596 (p.159)

**docker compose kill**
585 (p.156), 586 (p.156)

**docker compose logs**
598 (p.159)

**docker compose pause**
587 (p.156), 588 (p.157)

**docker compose port**
591 (p.157), 592 (p.158)

**docker compose rm**
582 (p.155), 583 (p.155), 584 (p.156)

**docker compose unpause**
589 (p.157), 590 (p.157)

**docker compose watch**
601 (p.160), 602 (p.160)

**docker config**
519 (p.138), 520 (p.139), 521 (p.139), 522 (p.139), 523 (p.139), 524 (p.140)

**docker container exec**
565 (p.151)

**docker container inspect**
562 (p.150), 563 (p.150)

**docker container logs**
564 (p.151)

**docker container ls**
560 (p.150), 561 (p.150)

**docker container pause**
570 (p.152)

**docker container prune**
204 (p.55), 205 (p.56), 206 (p.56)

**docker container restart**
569 (p.152)

**docker container rm**
566 (p.151)

**docker container start**
567 (p.151)

**docker container stop**
568 (p.152)

**docker container unpause**
571 (p.152)

**docker container update**
572 (p.153)

**docker container wait**
573 (p.153)

**docker context**
426 (p.113), 427 (p.114), 428 (p.114), 429 (p.114), 430 (p.114), 431 (p.115), 432 (p.115), 433 (p.115), 434 (p.115), 435 (p.116), 436 (p.116), 437 (p.116)

**docker context ls**
558 (p.149), 559 (p.149)

**docker cp**
189 (p.52), 190 (p.52), 191 (p.52), 192 (p.52)

**docker create**
138 (p.38), 139 (p.39), 140 (p.39), 141 (p.39), 142 (p.39), 143 (p.40), 144 (p.40), 145 (p.40), 146 (p.40), 147 (p.41)

**docker diff**
193 (p.53), 194 (p.53)

**docker events**
019 (p.8), 020 (p.9), 021 (p.9), 022 (p.9), 023 (p.9), 024 (p.10)

**docker exec**
118 (p.33), 119 (p.34), 120 (p.34), 121 (p.34), 122 (p.34), 123 (p.35), 124 (p.35), 125 (p.35), 126 (p.35), 127 (p.36), 128 (p.36), 129 (p.36)

**docker export**
200 (p.54), 201 (p.55)

**docker history**
242 (p.65), 243 (p.66), 244 (p.66)

**docker image history**
579 (p.154)

**docker image inspect**
249 (p.67), 250 (p.67), 251 (p.68), 252 (p.68)

**docker image load**
577 (p.154)

**docker image ls**
207 (p.56), 208 (p.57), 209 (p.57), 210 (p.57), 211 (p.57), 212 (p.58), 213 (p.58), 214 (p.58), 215 (p.58), 216 (p.59)

**docker image prune**
226 (p.61), 227 (p.62), 228 (p.62), 229 (p.62)

**docker image pull**
574 (p.153)

**docker image push**
575 (p.153)

**docker image rm**
222 (p.60), 223 (p.60), 224 (p.61), 225 (p.61)

**docker image save**
576 (p.154)

**docker image tag**
578 (p.154)

**docker import**
202 (p.55), 203 (p.55)

**docker info**
006 (p.5), 007 (p.5), 008 (p.6), 009 (p.6), 010 (p.6)

**docker inspect**
130 (p.36), 131 (p.37), 132 (p.37), 133 (p.37), 134 (p.37), 135 (p.38), 136 (p.38), 137 (p.38)

**docker kill**
159 (p.44), 160 (p.44), 161 (p.44)

**docker load**
239 (p.65), 240 (p.65), 241 (p.65)

**docker login**
258 (p.70), 259 (p.70), 260 (p.70), 261 (p.70)

**docker logout**
262 (p.71), 263 (p.71)

**docker logs**
108 (p.31), 109 (p.31), 110 (p.31), 111 (p.32), 112 (p.32), 113 (p.32), 114 (p.32), 115 (p.33), 116 (p.33), 117 (p.33)

**docker manifest**
253 (p.68), 254 (p.68), 255 (p.69), 256 (p.69), 257 (p.69)

**docker network**
313 (p.84), 314 (p.85), 315 (p.85), 316 (p.85), 317 (p.85), 318 (p.86), 319 (p.86), 320 (p.86), 321 (p.86), 322 (p.87), 323 (p.87), 324 (p.87), 325 (p.87), 326 (p.88), 327 (p.88), 328 (p.88), 329 (p.88), 330 (p.89), 331 (p.89), 332 (p.89), 333 (p.89), 334 (p.90), 335 (p.90), 336 (p.90)

**docker network inspect**
581 (p.155)

**docker node**
490 (p.131), 491 (p.131), 492 (p.132), 493 (p.132), 494 (p.132), 495 (p.132), 496 (p.133), 497 (p.133), 498 (p.133), 499 (p.133), 500 (p.134), 501 (p.134), 502 (p.134), 503 (p.134), 504 (p.135)

**docker pause**
166 (p.46), 167 (p.46)

**docker plugin**
534 (p.143), 535 (p.143), 536 (p.143), 537 (p.144), 538 (p.144), 539 (p.144), 540 (p.145), 541 (p.145), 542 (p.145)

**docker port**
183 (p.50), 184 (p.50)

**docker ps**
093 (p.27), 094 (p.27), 095 (p.28), 096 (p.28), 097 (p.28), 098 (p.28), 099 (p.29), 100 (p.29), 101 (p.29), 102 (p.29), 103 (p.30), 104 (p.30), 105 (p.30), 106 (p.30), 107 (p.31)

**docker pull**
217 (p.59), 218 (p.59), 219 (p.59), 220 (p.60), 221 (p.60)

**docker push**
233 (p.63), 234 (p.63), 235 (p.64)

**docker rename**
170 (p.47), 171 (p.47)

**docker restart**
156 (p.43), 157 (p.43), 158 (p.43)

**docker rm**
162 (p.45), 163 (p.45), 164 (p.45), 165 (p.46)

**docker run**
030 (p.11), 031 (p.12), 032 (p.12), 033 (p.12), 034 (p.12), 035 (p.13), 036 (p.13), 037 (p.13), 038 (p.13), 039 (p.14), 040 (p.14), 041 (p.14), 042 (p.14), 043 (p.15), 044 (p.15), 045 (p.15), 046 (p.15), 047 (p.16), 048 (p.16), 049 (p.16), 050 (p.16), 051 (p.17), 052 (p.17), 053 (p.17), 054 (p.17), 055 (p.18), 056 (p.18), 057 (p.18), 058 (p.18), 059 (p.19), 060 (p.19), 061 (p.19), 062 (p.19), 063 (p.20), 064 (p.20), 065 (p.20), 066 (p.20), 067 (p.21), 068 (p.21), 069 (p.21), 070 (p.21), 071 (p.22), 072 (p.22), 073 (p.22), 074 (p.22), 075 (p.23), 076 (p.23), 077 (p.23), 078 (p.23), 079 (p.24), 080 (p.24), 081 (p.24), 082 (p.24), 083 (p.25), 084 (p.25), 085 (p.25), 086 (p.25), 087 (p.26), 088 (p.26), 089 (p.26), 090 (p.26), 091 (p.27), 092 (p.27)

**docker save**
236 (p.64), 237 (p.64), 238 (p.64)

**docker scout**
543 (p.145), 544 (p.146), 545 (p.146), 546 (p.146), 547 (p.146), 548 (p.147), 549 (p.147)

**docker search**
245 (p.66), 246 (p.66), 247 (p.67), 248 (p.67)

**docker secret**
513 (p.137), 514 (p.137), 515 (p.137), 516 (p.138), 517 (p.138), 518 (p.138)

**docker service**
458 (p.123), 459 (p.123), 460 (p.124), 461 (p.124), 462 (p.124), 463 (p.124), 464 (p.125), 465 (p.125), 466 (p.125), 467 (p.125), 468 (p.126), 469 (p.126), 470 (p.126), 471 (p.126), 472 (p.127), 473 (p.127), 474 (p.127), 475 (p.127), 476 (p.128), 477 (p.128), 478 (p.128), 479 (p.128), 480 (p.129), 481 (p.129), 482 (p.129), 483 (p.129), 484 (p.130), 485 (p.130), 486 (p.130), 487 (p.130), 488 (p.131), 489 (p.131)

**docker stack**
505 (p.135), 506 (p.135), 507 (p.135), 508 (p.136), 509 (p.136), 510 (p.136), 511 (p.136), 512 (p.137)

**docker start**
148 (p.41), 149 (p.41), 150 (p.41), 151 (p.42)

**docker stats**
185 (p.51), 186 (p.51), 187 (p.51), 188 (p.51)

**docker stop**
152 (p.42), 153 (p.42), 154 (p.42), 155 (p.43)

**docker swarm**
438 (p.117), 439 (p.117), 440 (p.117), 441 (p.118), 442 (p.118), 443 (p.118), 444 (p.119), 445 (p.119), 446 (p.119), 447 (p.120), 448 (p.120), 449 (p.120), 450 (p.121), 451 (p.121), 452 (p.121), 453 (p.122), 454 (p.122), 455 (p.122), 456 (p.123), 457 (p.123)

**docker system df**
017 (p.8), 018 (p.8)

**docker system prune**
025 (p.10), 026 (p.10), 027 (p.10), 028 (p.11), 029 (p.11), 525 (p.140), 526 (p.140), 527 (p.141), 528 (p.141), 529 (p.141)

**docker tag**
230 (p.62), 231 (p.63), 232 (p.63)

**docker top**
181 (p.50), 182 (p.50)

**docker trust**
550 (p.147), 551 (p.147), 552 (p.148), 553 (p.148)

**docker unpause**
168 (p.46), 169 (p.47)

**docker update**
172 (p.47), 173 (p.48), 174 (p.48), 175 (p.48), 176 (p.48), 177 (p.49), 178 (p.49)

**docker version**
001 (p.4), 002 (p.4), 003 (p.4), 004 (p.5), 005 (p.5)

**docker volume**
337 (p.90), 338 (p.91), 339 (p.91), 340 (p.91), 341 (p.91), 342 (p.92), 343 (p.92), 344 (p.92), 345 (p.92), 346 (p.93), 347 (p.93), 348 (p.93), 349 (p.93), 350 (p.94), 351 (p.94), 352 (p.94), 353 (p.95), 354 (p.95), 355 (p.95)

**docker volume inspect**
580 (p.155)

**docker wait**
179 (p.49), 180 (p.49)

**Dockerfile: ADD**
608 (p.162)

**Dockerfile: ARG**
611 (p.163)

**Dockerfile: BuildKit cache mount**
622 (p.165)

**Dockerfile: BuildKit secret mount**
621 (p.165)

**Dockerfile: CMD**
605 (p.161)

**Dockerfile: COPY**
607 (p.162)

**Dockerfile: ENTRYPOINT**
606 (p.161)

**Dockerfile: ENV**
610 (p.162)

**Dockerfile: EXPOSE**
612 (p.163)

**Dockerfile: FROM**
603 (p.161)

**Dockerfile: HEALTHCHECK**
615 (p.164)

**Dockerfile: LABEL**
618 (p.164)

**Dockerfile: Multi-stage build**
620 (p.165)

**Dockerfile: ONBUILD**
619 (p.165)

**Dockerfile: RUN**
604 (p.161)

**Dockerfile: SHELL**
617 (p.164)

**Dockerfile: STOPSIGNAL**
616 (p.164)

**Dockerfile: USER**
613 (p.163)

**Dockerfile: VOLUME**
614 (p.163)

**Dockerfile: WORKDIR**
609 (p.162)

**Workflow: Build, tag, and publish**
625 (p.166)

**Workflow: Compose deployment and teardown**
628 (p.167)

**Workflow: Create a network and persistent volume**
626 (p.167)

**Workflow: Diagnose a failed container**
627 (p.167)

**Workflow: Review before cleanup**
629 (p.168)

**Workflow: Run and troubleshoot a web server**
624 (p.166)

**Workflow: Swarm deployment verification**
630 (p.168)

**Workflow: Verify a fresh install**
623 (p.166)